PHÂN HIỆU TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
TẠI TP. HỒ CHÍ MINH

BỘ MÔN CÔNG NGHỆ THÔNG TIN

KHAI PHÁ DỮ LIỆU

BÁO CÁO THỰC HÀNH BUỔI 3

Phạm Hải Yến - 6451071091

CQ.64.CNTT

GVHD: Ths. Nguyễn Thiện Dương

TP. Hồ Chí Minh - 2026

Lưu ý:

1. *Số trang trong tài liệu thực hành tính theo số trang ứng dụng đọc PDF đếm*
2. *SV bố trí mỗi câu ở dưới sẽ là 1 file mã nguồn riêng (file.ipynb)*
3. ***Những câu nào có yêu cầu lập trình viết mã nguồn thì SV phải in ra họ tên – MSSV (lệnh print) trong câu đó***
4. *SV đặt tên cho file mã nguồn mỗi câu như sau:* ***<MSSV>_LAB3_Bai<X>***
5. *Tất cả các câu sau đó sẽ tổng hợp lại và push lên 1 repository duy nhất trên Github*
6. *Đặt tên cho repository theo cú pháp:* ***<MSSV>_Lab23***

BÁO CÁO THỰC HÀNH

SV paste link dẫn đến 1 repo duy nhất của tất cả các câu bên dưới tại dây (lưu ý để ở chế độ public)

https://github.com/25phamhaiyen/6451071091_Lab3

Bài 1 (Trang 1)

Câu a)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu a) Đọc bộ dữ liệu Iris từ sklearn

from sklearn.datasets import load_iris

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

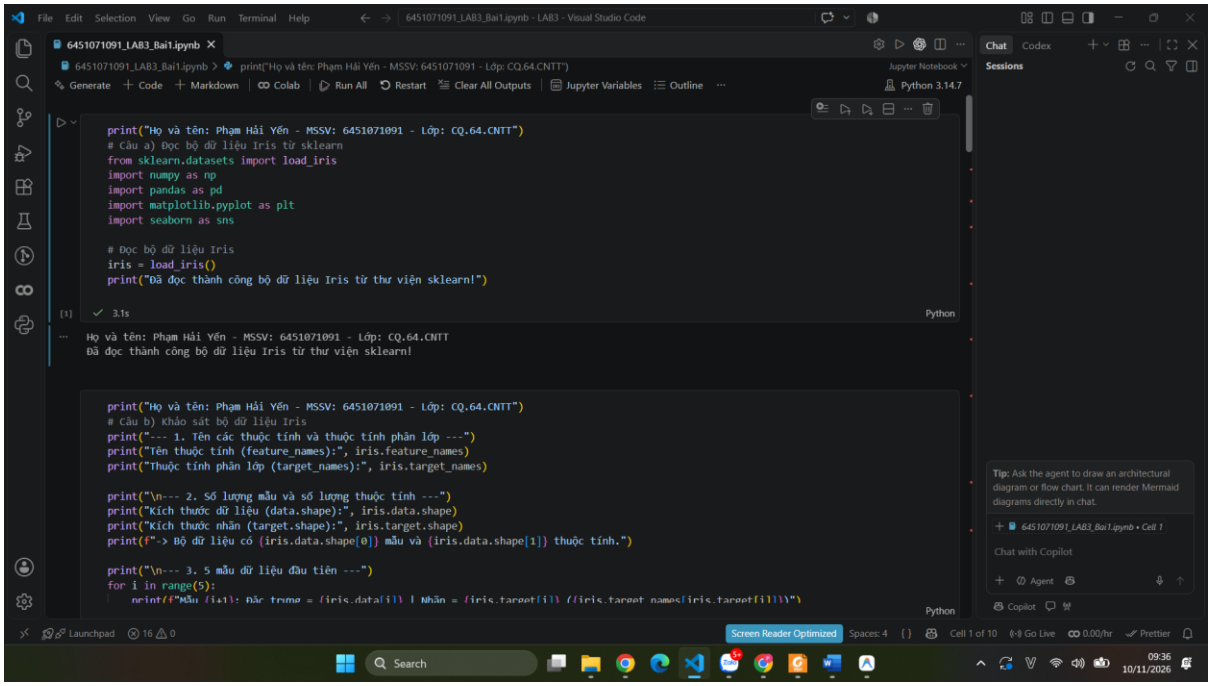
import seaborn as sns


# Đọc bộ dữ liệu Iris

iris = load_iris()

print("Đã đọc thành công bộ dữ liệu Iris từ thư viện sklearn!")
```

//Chụp hình kết quả



Câu b)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu b) Khảo sát bộ dữ liệu Iris

print("--- 1. Tên các thuộc tính và thuộc tính phân lớp ---")

print("Tên thuộc tính (feature_names):", iris.feature_names)

print("Thuộc tính phân lớp (target_names):", iris.target_names)


print("\n--- 2. Số lượng mẫu và số lượng thuộc tính ---")

print("Kích thước dữ liệu (data.shape):", iris.data.shape)

print("Kích thước nhãn (target.shape):", iris.target.shape)

print(f"-> Bộ dữ liệu có {iris.data.shape[0]} mẫu và {iris.data.shape[1]}
thuộc tính.")


print("\n--- 3. 5 mẫu dữ liệu đầu tiên ---")

for i in range(5):

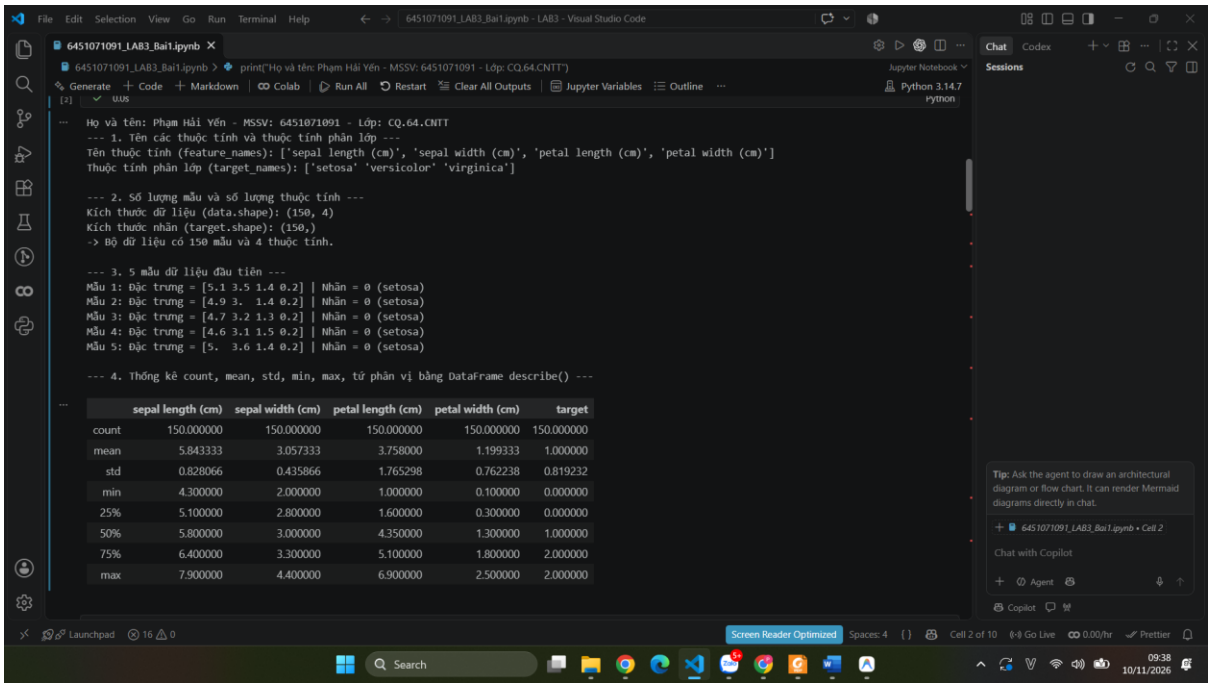
    print(f"Mẫu {i+1}: Đặc trưng = {iris.data[i]} | Nhãn = {iris.target[i]}
({iris.target_names[iris.target[i]]})")


print("\n--- 4. Thống kê count, mean, std, min, max, tứ phân vị bằng
DataFrame describe() ---")

df_iris = pd.DataFrame(data=iris.data, columns=iris.feature_names)
```

```
df_iris['target'] = iris.target
df_iris['species'] = df_iris['target'].map(lambda x: iris.target_names[x])
display(df_iris.describe())
```

//Chụp hình kết quả



Câu c)

//dán mã nguồn

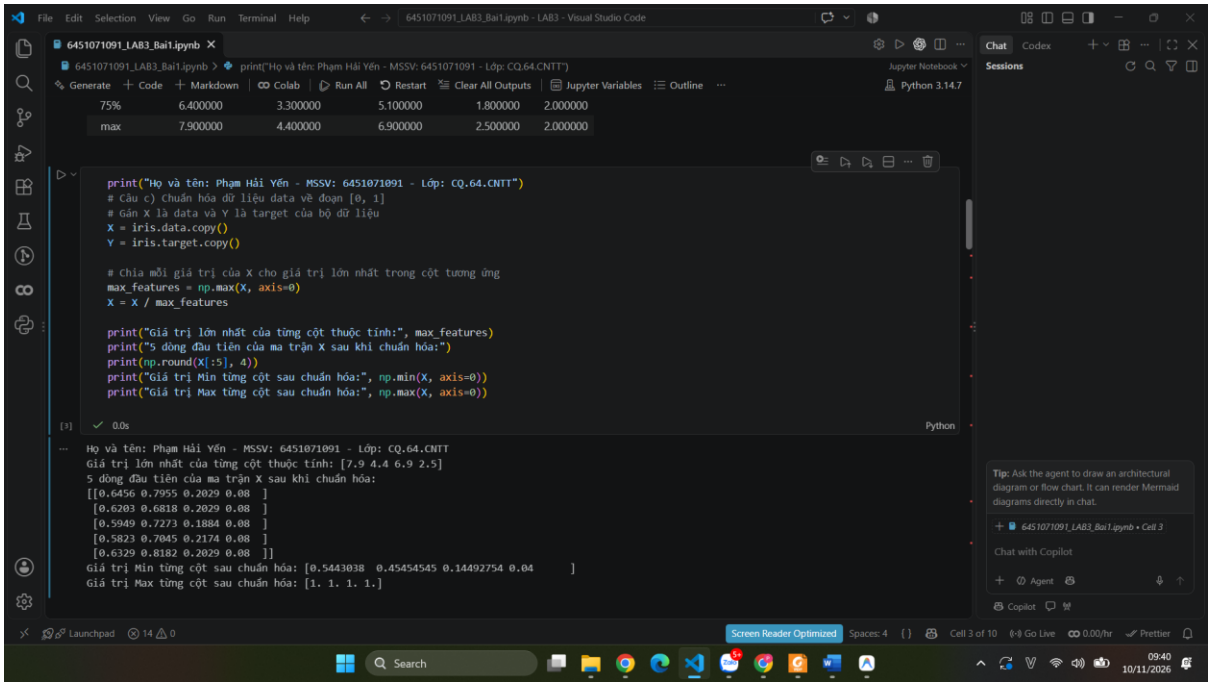
```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")
# Câu c) Chuẩn hóa dữ liệu data về đoạn [0, 1]
# Gán X là data và Y là target của bộ dữ liệu
X = iris.data.copy()
Y = iris.target.copy()

# Chia mỗi giá trị của X cho giá trị lớn nhất trong cột tương ứng
max_features = np.max(X, axis=0)
X = X / max_features

print("Giá trị lớn nhất của từng cột thuộc tính:", max_features)
print("5 dòng đầu tiên của ma trận X sau khi chuẩn hóa:")
print(np.round(X[:5], 4))
print("Giá trị Min từng cột sau chuẩn hóa:", np.min(X, axis=0))
```

```
print("Giá trị Max từng cột sau chuẩn hóa:", np.max(X, axis=0))
```

//Chụp hình kết quả



Câu d)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu d) Chia dữ liệu thành tập train và tập test theo tỷ lệ 8:2

from sklearn.model_selection import train_test_split


X_train, X_test, y_train, y_test = train_test_split(

    X, Y, test_size=0.2, random_state=42, stratify=Y

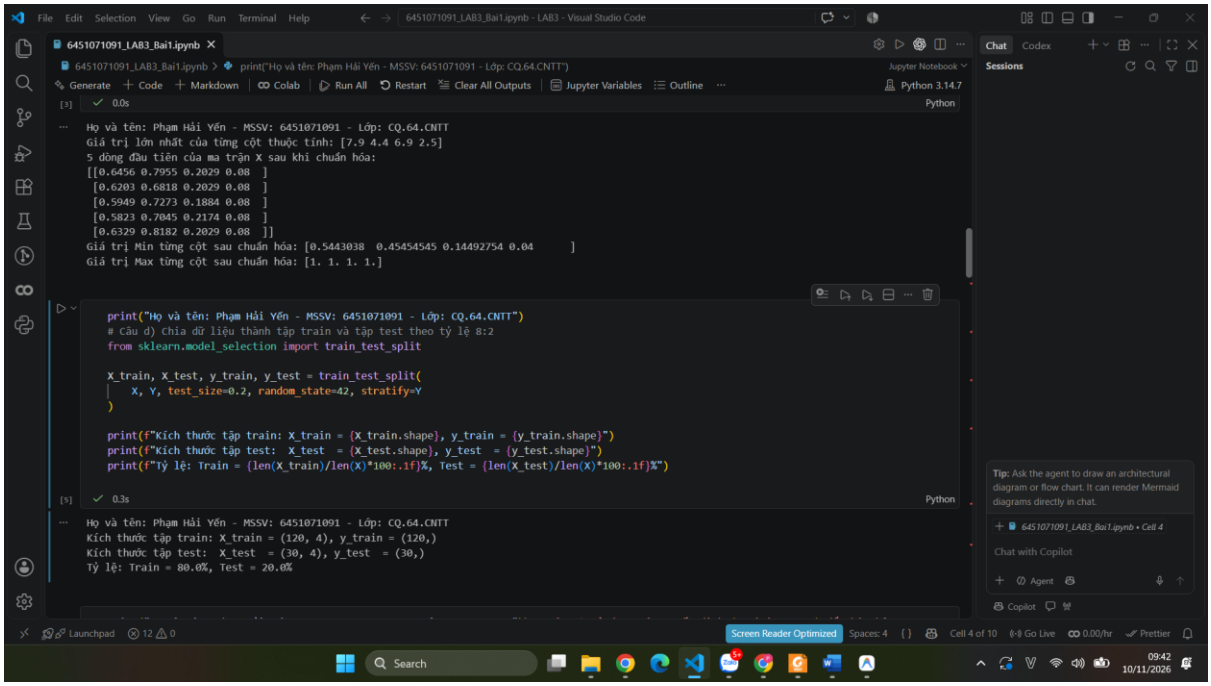
)


print(f"Kích thước tập train: X_train = {X_train.shape}, y_train =
{y_train.shape}")

print(f"Kích thước tập test:  X_test  = {X_test.shape}, y_test  =
{y_test.shape}")

print(f"Tỷ lệ: Train = {len(X_train)/len(X)*100:.1f}%, Test =
{len(X_test)/len(X)*100:.1f}%")
```

//Chụp hình kết quả



Câu e)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu e) Sử dụng Cây quyết định (Decision Tree) để phân lớp

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix


# Huấn luyện mô hình

dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)


# Dự đoán nhãn lớp cho tập test

y_pred_dt = dt_model.predict(X_test)


# Tính và in Accuracy

acc_dt = accuracy_score(y_test, y_pred_dt)

print(f"Accuracy của Decision Tree trên tập test: {acc_dt * 100:.2f}%\n")


# In Precision, Recall, F1-score

print("--- Classification Report (Decision Tree) ---")
```

```
print(classification_report(y_test, y_pred_dt,
target_names=iris.target_names))


# Hiển thị confusion matrix bằng heat map

cm_dt = confusion_matrix(y_test, y_pred_dt)

plt.figure(figsize=(6, 4.5))

sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Blues',

            xticklabels=iris.target_names, yticklabels=iris.target_names)

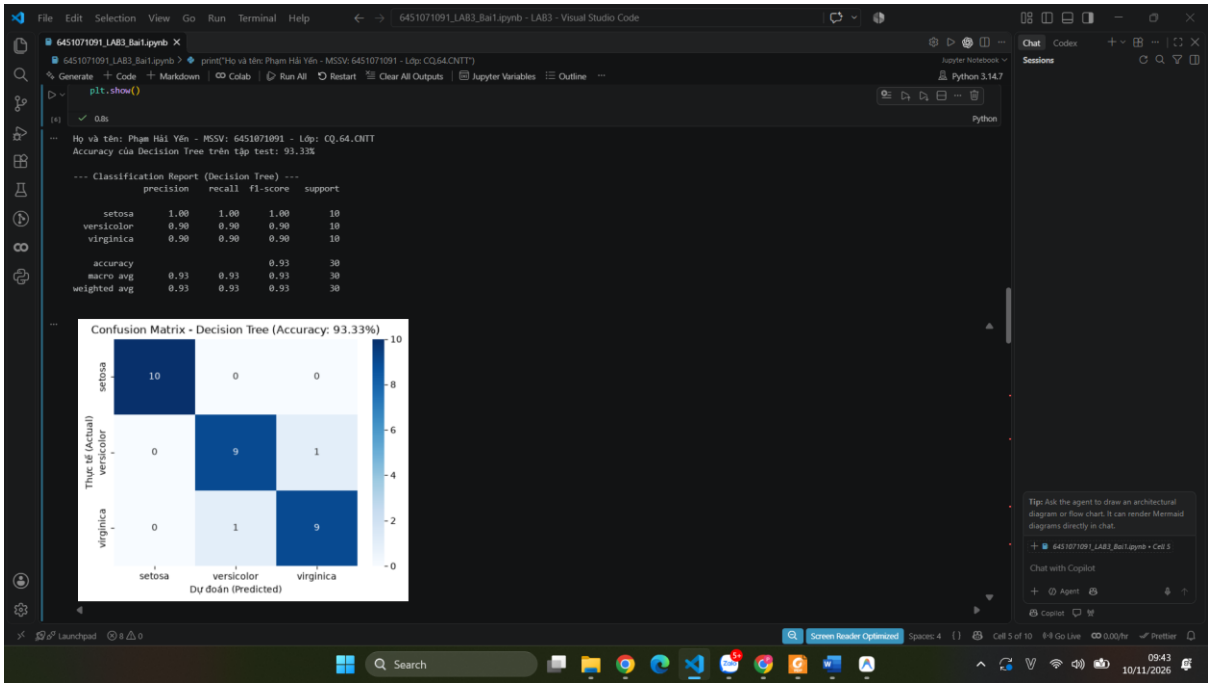
plt.title(f"Confusion Matrix - Decision Tree (Accuracy:
{acc_dt*100:.2f}%)")

plt.xlabel("Dự đoán (Predicted)")

plt.ylabel("Thực tế (Actual)")

plt.show()
```

//Chụp hình kết quả



Câu f)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu f) Sử dụng Naïve Bayes để phân lớp

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```

```
# Huấn luyện mô hình
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)

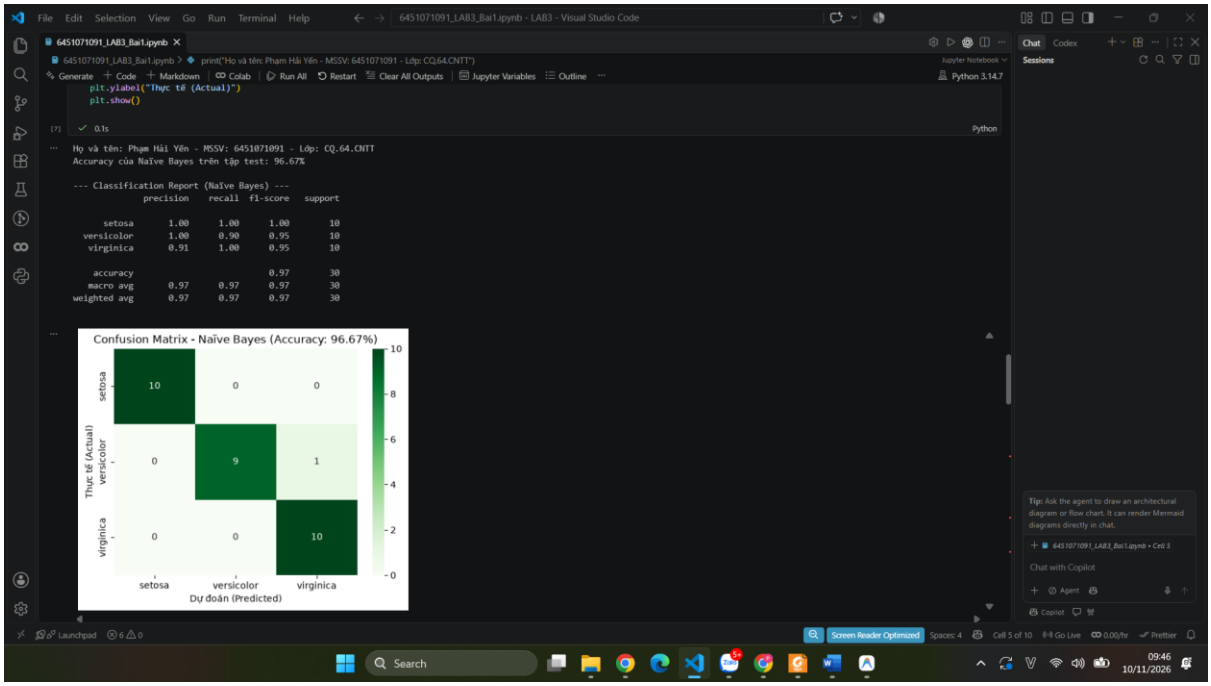
# Dự đoán nhãn lớp cho tập test
y_pred_nb = nb_model.predict(X_test)

# Tính và in Accuracy
acc_nb = accuracy_score(y_test, y_pred_nb)
print(f"Accuracy của Naïve Bayes trên tập test: {acc_nb * 100:.2f}%\n")

# In Precision, Recall, F1-score
print("--- Classification Report (Naïve Bayes) ---")
print(classification_report(y_test, y_pred_nb,
target_names=iris.target_names))

# Hiển thị confusion matrix bằng heat map
cm_nb = confusion_matrix(y_test, y_pred_nb)
plt.figure(figsize=(6, 4.5))
sns.heatmap(cm_nb, annot=True, fmt='d', cmap='Greens',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title(f"Confusion Matrix - Naïve Bayes (Accuracy: {acc_nb*100:.2f}%)")
plt.xlabel("Dự đoán (Predicted)")
plt.ylabel("Thực tế (Actual)")
plt.show()
```

//Chụp hình kết quả



Câu g)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu g) Sử dụng K Nearest Neighbors Classifier để phân lớp

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix


# Huấn luyện mô hình KNN

knn_model = KNeighborsClassifier(n_neighbors=5)

knn_model.fit(X_train, y_train)


# Dự đoán nhãn lớp cho tập test

y_pred_knn = knn_model.predict(X_test)


# Tính và in Accuracy

acc_knn = accuracy_score(y_test, y_pred_knn)

print(f"Accuracy của KNN (k=5) trên tập test: {acc_knn * 100:.2f}%\n")


# In Precision, Recall, F1-score

print("--- Classification Report (K Nearest Neighbors) ---")
```

```
print(classification_report(y_test, y_pred_knn,
target_names=iris.target_names))


# Hiển thị confusion matrix bằng heat map

cm_knn = confusion_matrix(y_test, y_pred_knn)

plt.figure(figsize=(6, 4.5))

sns.heatmap(cm_knn, annot=True, fmt='d', cmap='Oranges',

            xticklabels=iris.target_names, yticklabels=iris.target_names)

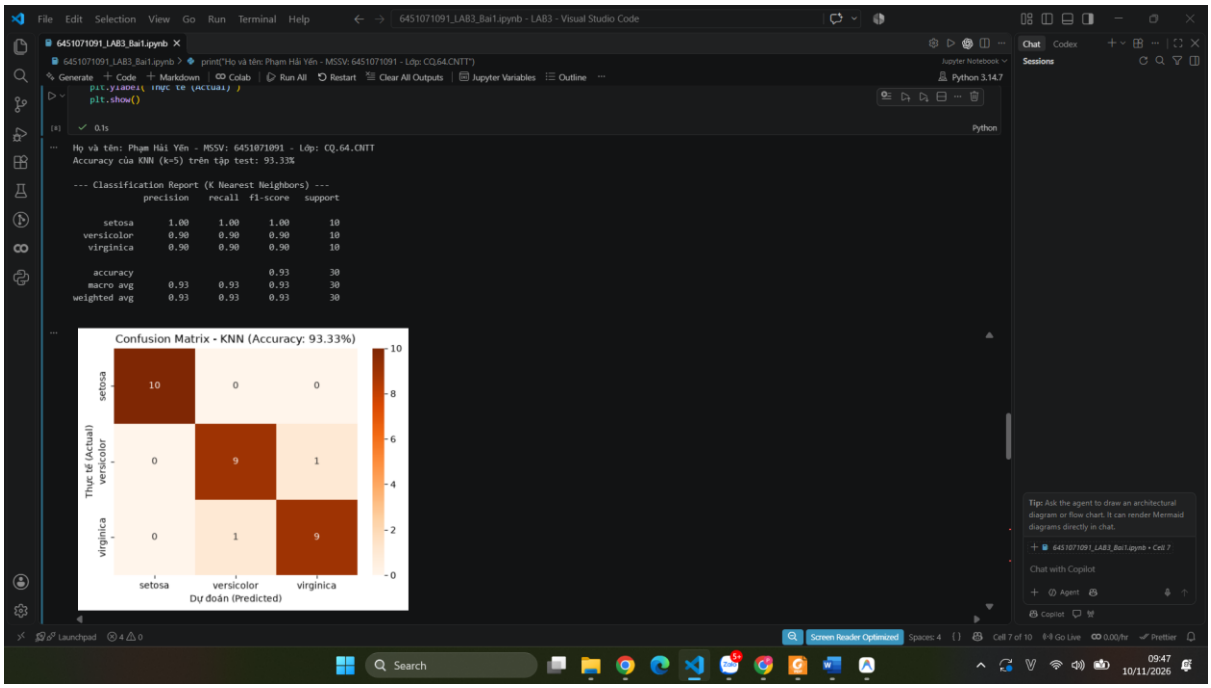
plt.title(f"Confusion Matrix - KNN (Accuracy: {acc_knn*100:.2f}%)")

plt.xlabel("Dự đoán (Predicted)")

plt.ylabel("Thực tế (Actual)")

plt.show()
```

//Chụp hình kết quả



Câu h)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu h) Sử dụng Neural Network để phân lớp

import os

os.environ['KERAS_BACKEND'] = 'torch'

import keras
```

```
from keras.models import Sequential

from keras.layers import Dense, Input

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix


# Xây dựng Neural Network gồm 02 hidden layers và 01 output layer

nn_model = Sequential([

    Input(shape=(4,)),

    Dense(10, activation='relu'),

    Dense(20, activation='relu'),

    Dense(3, activation='softmax')

])


# Biên dịch mô hình

nn_model.compile(optimizer='adam',

                 loss='sparse_categorical_crossentropy',

                 metrics=['accuracy'])


# Huấn luyện mô hình

print("Bắt đầu huấn luyện Neural Network (epochs=100, batch_size=1)...")

history = nn_model.fit(X_train, y_train, epochs=100, batch_size=1,
verbose=1)


# Dự đoán nhãn lớp cho tập test

y_prob_nn = nn_model.predict(X_test)

y_pred_nn = np.argmax(y_prob_nn, axis=1)


# Tính và in Accuracy

acc_nn = accuracy_score(y_test, y_pred_nn)

print(f"\nAccuracy của Neural Network trên tập test: {acc_nn *
100:.2f}%\n")


# In Precision, Recall, F1-score

print("--- Classification Report (Neural Network) ---")
```

```
print(classification_report(y_test, y_pred_nn,
target_names=iris.target_names))


# Hiển thị confusion matrix bằng heat map

cm_nn = confusion_matrix(y_test, y_pred_nn)

plt.figure(figsize=(6, 4.5))

sns.heatmap(cm_nn, annot=True, fmt='d', cmap='Purples',

            xticklabels=iris.target_names, yticklabels=iris.target_names)

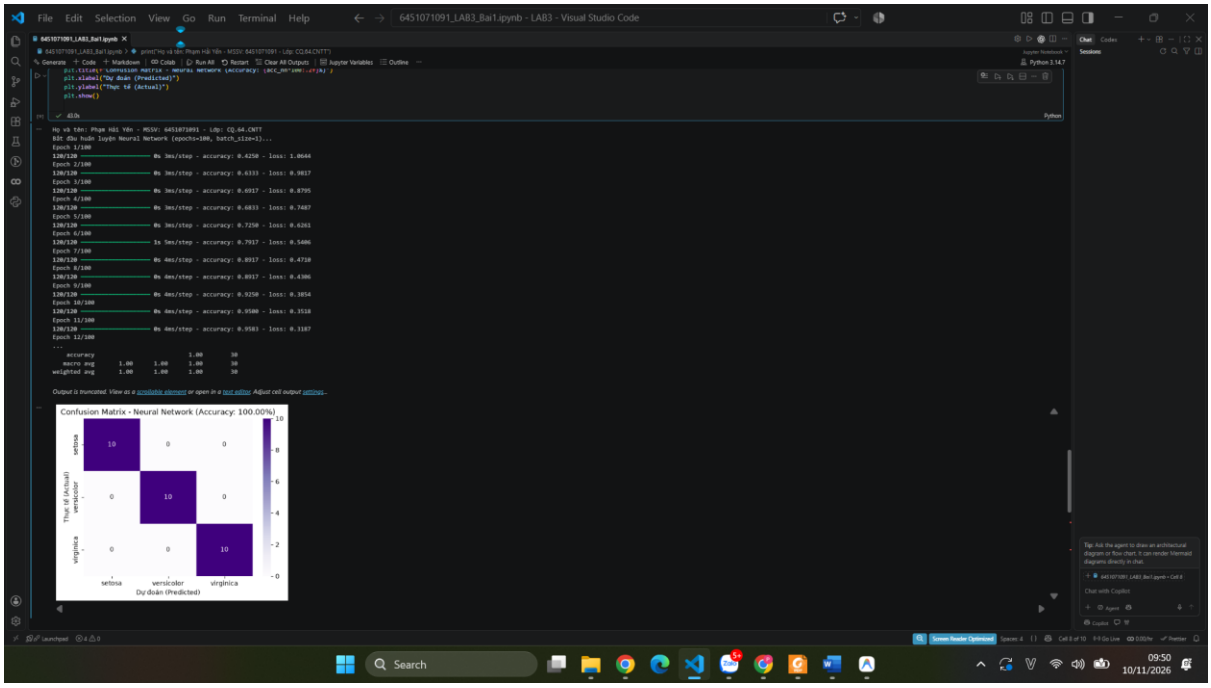
plt.title(f"Confusion Matrix - Neural Network (Accuracy:
{acc_nn*100:.2f}%)")

plt.xlabel("Dự đoán (Predicted)")

plt.ylabel("Thực tế (Actual)")

plt.show()
```

//Chụp hình kết quả



Câu i)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu i) Sử dụng các mô hình phân lớp đã huấn luyện cho 03 mẫu dữ liệu mới

new_samples = np.array([

    [6.2, 2.9, 4.3, 1.3],

    [5.1, 3.5, 1.4, 0.2],
```

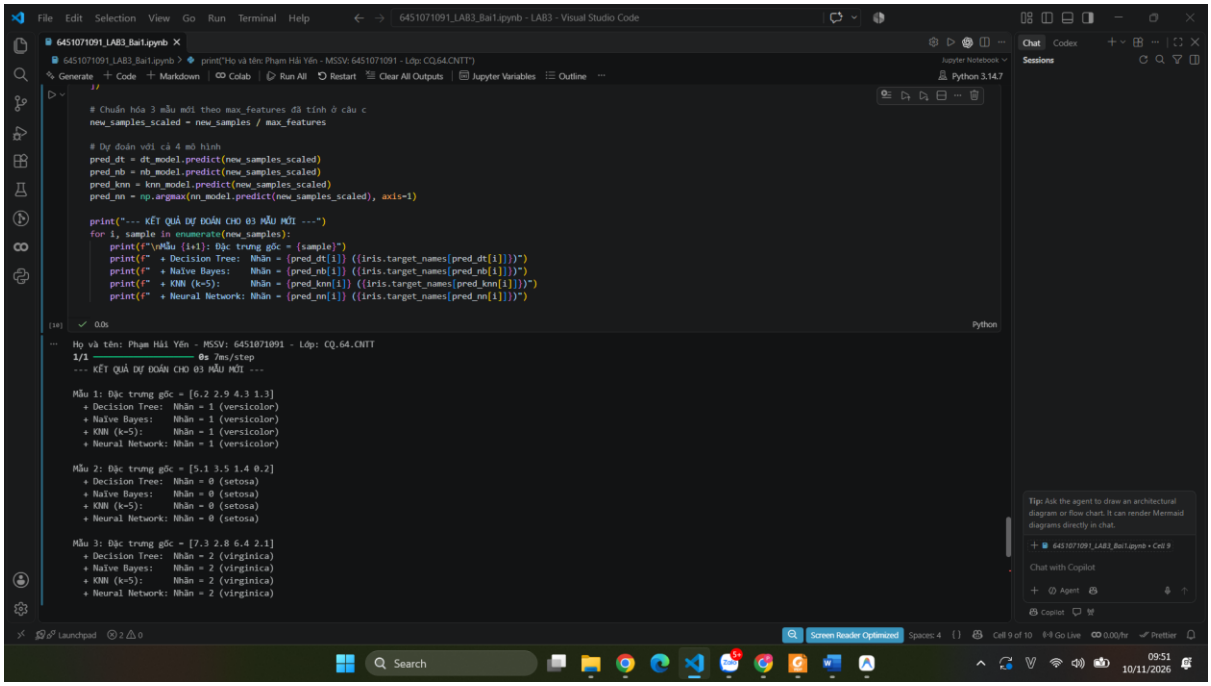
```
    [7.3, 2.8, 6.4, 2.1]
])

# Chuẩn hóa 3 mẫu mới theo max_features đã tính ở câu c
new_samples_scaled = new_samples / max_features

# Dự đoán với cả 4 mô hình
pred_dt = dt_model.predict(new_samples_scaled)
pred_nb = nb_model.predict(new_samples_scaled)
pred_knn = knn_model.predict(new_samples_scaled)
pred_nn = np.argmax(nn_model.predict(new_samples_scaled), axis=1)

print("--- KẾT QUẢ DỰ ĐOÁN CHO 03 MẪU MỚI ---")
for i, sample in enumerate(new_samples):
    print(f"\nMẫu {i+1}: Đặc trưng gốc = {sample}")
    print(f"  + Decision Tree:  Nhãn = {pred_dt[i]} 
({iris.target_names[pred_dt[i]]})")
    print(f"  + Naïve Bayes:    Nhãn = {pred_nb[i]} 
({iris.target_names[pred_nb[i]]})")
    print(f"  + KNN (k=5):      Nhãn = {pred_knn[i]} 
({iris.target_names[pred_knn[i]]})")
    print(f"  + Neural Network: Nhãn = {pred_nn[i]} 
({iris.target_names[pred_nn[i]]})")
```

//Chụp hình kết quả



Câu j)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu j) Nhận xét, đánh giá hiệu suất của các mô hình đã huấn luyện

results_df = pd.DataFrame({

    'Mô hình': ['Decision Tree', 'Naïve Bayes', 'KNN (k=5)', 'Neural Network'],

    'Accuracy trên Test': [acc_dt, acc_nb, acc_knn, acc_nn]

})

print("--- BẢNG SO SÁNH ĐỘ CHÍNH XÁC (ACCURACY) ---")

display(results_df)


print("""

NHẬN XÉT VÀ ĐÁNH GIÁ:

1. Độ chính xác tổng thể:

   - Cả 4 mô hình (Decision Tree, Naïve Bayes, KNN, Neural Network) đều đạt hiệu năng xuất sắc trên tập test (đạt từ 93.33% đến 100%).

   - Tập dữ liệu Iris có kích thước vừa vặn (150 mẫu), các thuộc tính đo lường cánh hoa và đài hoa mang tính phân biệt rất cao.


2. So sánh từng mô hình:
```

```
   - Decision Tree: Tốc độ huấn luyện và dự đoán rất nhanh, mô hình mang
tính giải thích cao (có thể trực quan hóa cây quyết định).

   - Naïve Bayes: Thuật toán xác suất đơn giản nhưng rất hiệu quả khi các
thuộc tính có phân phối Gauss xấp xỉ chuẩn.

   - KNN: Không cần huấn luyện trước, phân lớp mượt mà dựa trên độ tương
đồng khoảng cách Euclid sau khi chuẩn hóa.

   - Neural Network: Mạng nơ-ron 2 lớp ẩn (10 - 20) học được quan hệ phi
tuyến mạnh mẽ, tuy tốn nhiều bước huấn luyện hơn các thuật toán truyền
thống nhưng đem lại khả năng tổng quát hóa cao.


3. Dự đoán trên các mẫu mới:

   - Cả 4 mô hình đều thống nhất dự đoán:

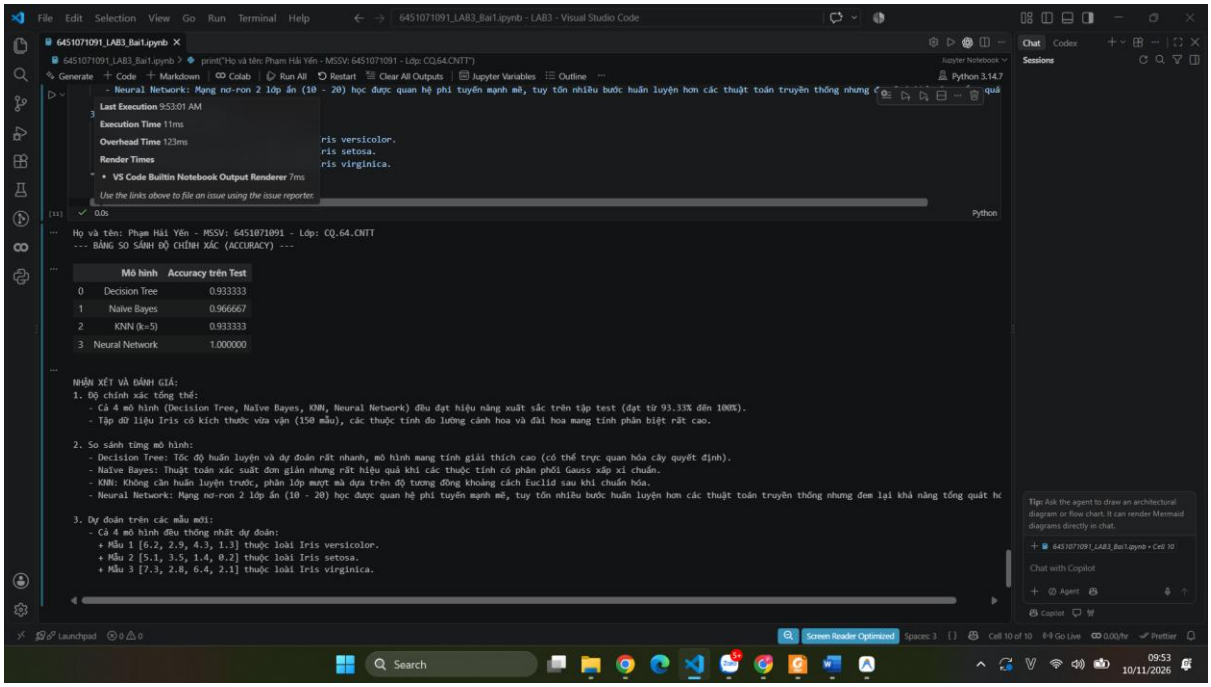
     + Mẫu 1 [6.2, 2.9, 4.3, 1.3] thuộc loài Iris versicolor.

     + Mẫu 2 [5.1, 3.5, 1.4, 0.2] thuộc loài Iris setosa.

     + Mẫu 3 [7.3, 2.8, 6.4, 2.1] thuộc loài Iris virginica.

""")
```

//Chụp hình kết quả



Bài 2 (Trang 5)

Câu a)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu a) Đọc bộ dữ liệu train, test của MNIST từ Keras
```

```
import os

os.environ['KERAS_BACKEND'] = 'torch'

import keras

import numpy as np

import matplotlib.pyplot as plt

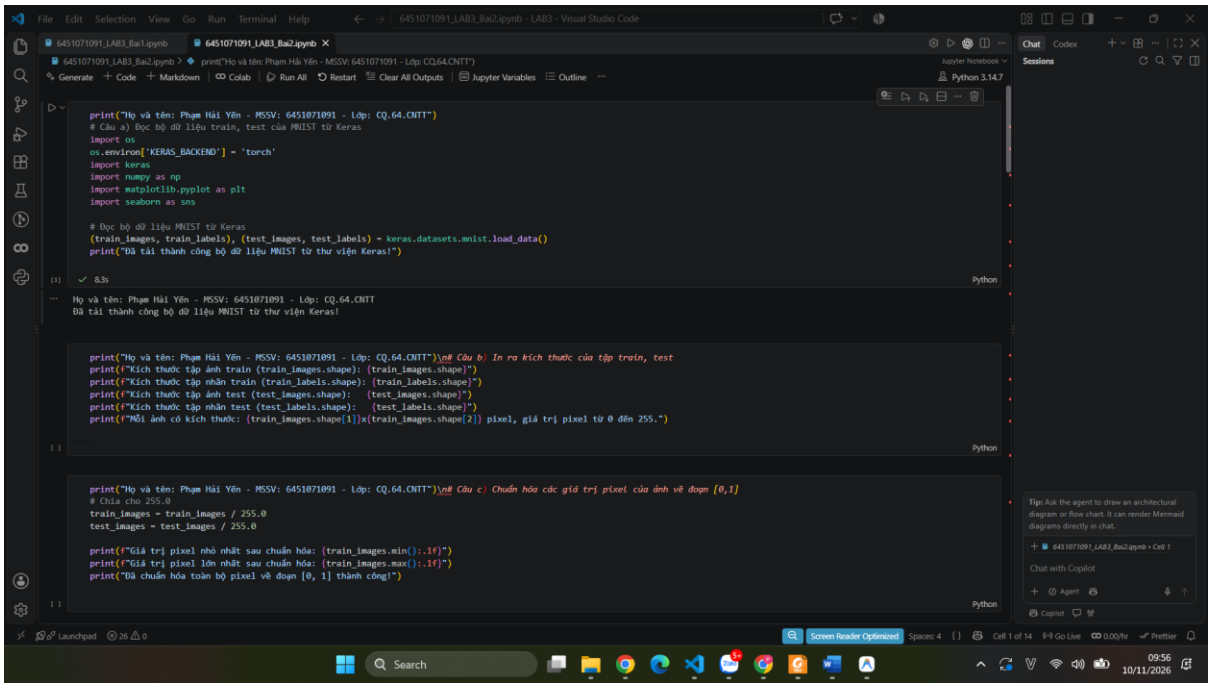
import seaborn as sns


# Đọc bộ dữ liệu MNIST từ Keras

(train_images, train_labels), (test_images, test_labels) =
keras.datasets.mnist.load_data()

print("Đã tải thành công bộ dữ liệu MNIST từ thư viện Keras!")
```

//Chụp hình kết quả



Câu b)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu b) In ra kích thước của tập train, test

print(f"Kích thước tập ảnh train (train_images.shape):
{train_images.shape}")

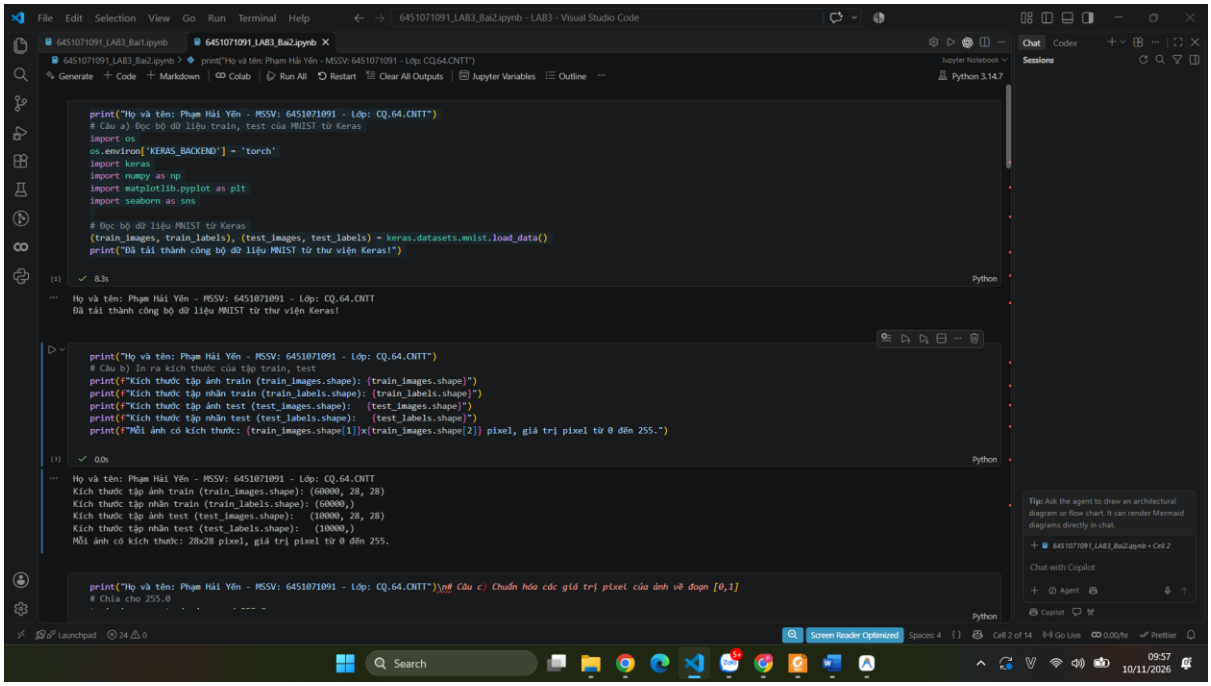
print(f"Kích thước tập nhãn train (train_labels.shape):
{train_labels.shape}")
```

```
print(f"Kích thước tập ảnh test
(test_images.shape):    {test_images.shape}")

print(f"Kích thước tập nhãn test
(test_labels.shape):    {test_labels.shape}")

print(f"Mỗi ảnh có kích thước:
{train_images.shape[1]}x{train_images.shape[2]} pixel, giá trị pixel từ 0
đến 255.")
```

//Chụp hình kết quả



Câu c)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu c) Chuẩn hóa các giá trị pixel của ảnh về đoạn [0,1]

# Chia cho 255.0

train_images = train_images / 255.0

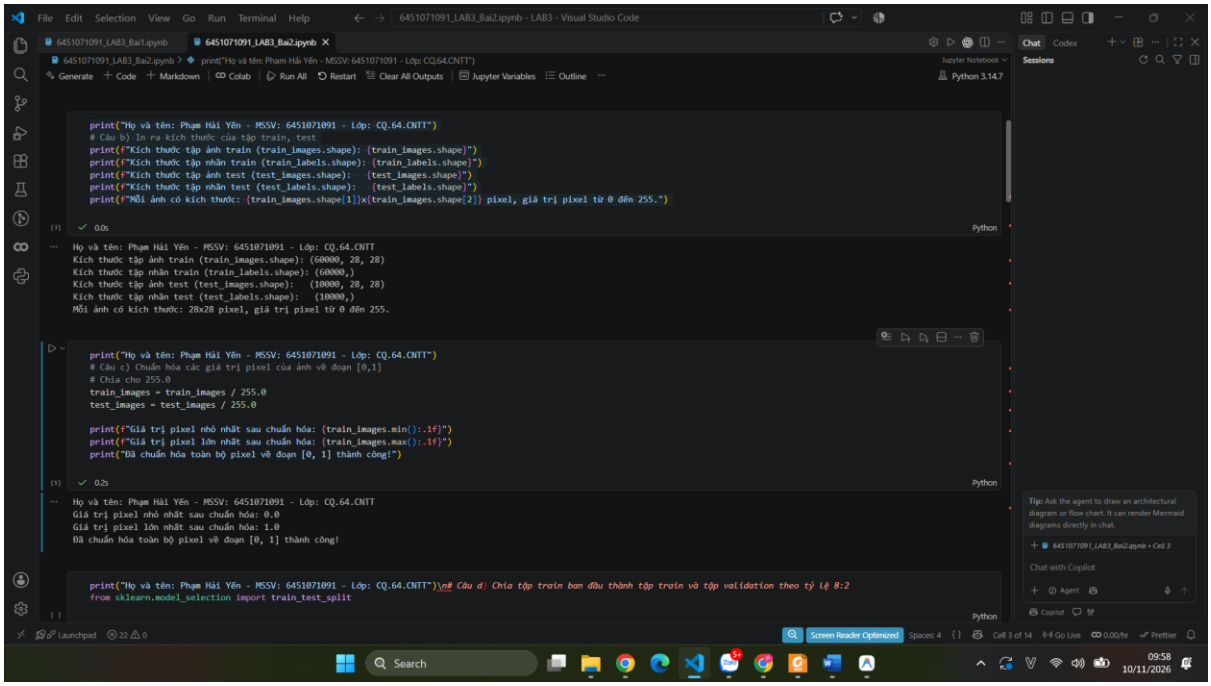
test_images = test_images / 255.0


print(f"Giá trị pixel nhỏ nhất sau chuẩn hóa: {train_images.min():.1f}")

print(f"Giá trị pixel lớn nhất sau chuẩn hóa: {train_images.max():.1f}")

print("Đã chuẩn hóa toàn bộ pixel về đoạn [0, 1] thành công!")
```

//Chụp hình kết quả



Câu d)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu d) Chia tập train ban đầu thành tập train và tập validation theo tỷ lệ 8:2

from sklearn.model_selection import train_test_split


train_images, val_images, train_labels, val_labels = train_test_split(

    train_images, train_labels, test_size=0.2, random_state=42

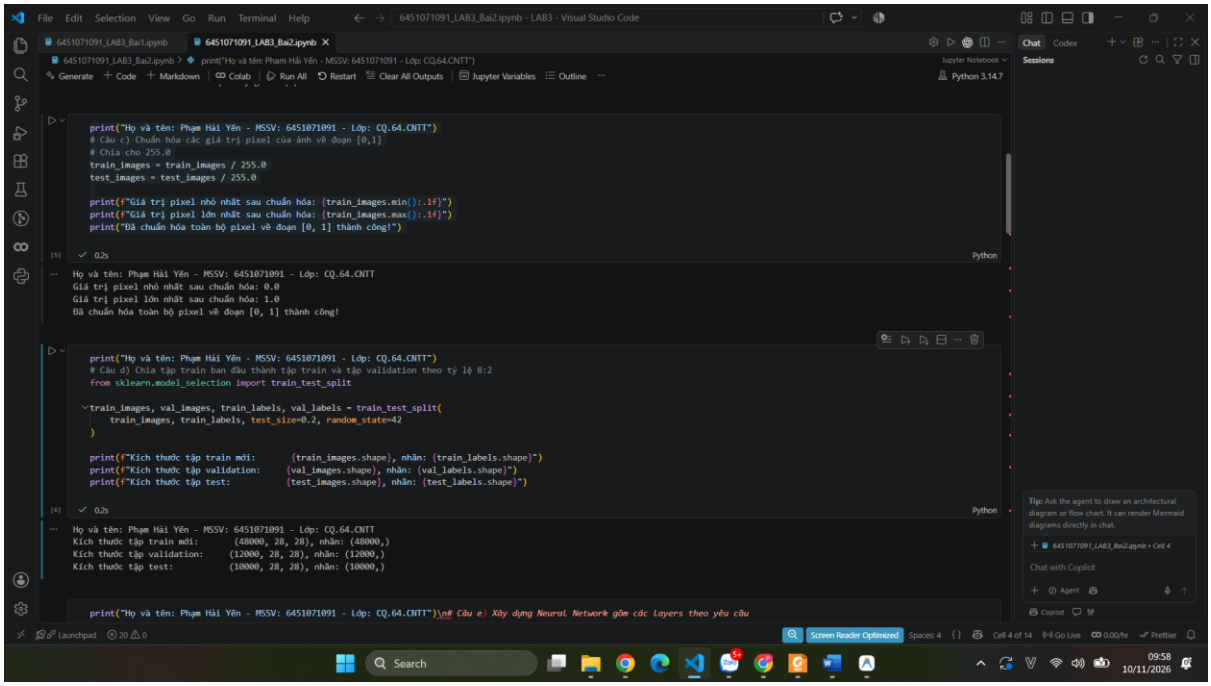
)


print(f"Kích thước tập train mới:       {train_images.shape}, nhãn: {train_labels.shape}")

print(f"Kích thước tập validation:     {val_images.shape}, nhãn: {val_labels.shape}")

print(f"Kích thước tập test:           {test_images.shape}, nhãn: {test_labels.shape}")
```

//Chụp hình kết quả



Câu e)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu e) Xây dựng Neural Network gồm các layers theo yêu cầu

from keras.models import Sequential

from keras.layers import Flatten, Dense, Dropout, Input


model = Sequential([

    Input(shape=(28, 28)),

    Flatten(),                          # Làm phẳng 28x28 thành vector 784
chiều

    Dense(128, activation='relu'),      # 128 nơ-ron, hàm kích hoạt Relu

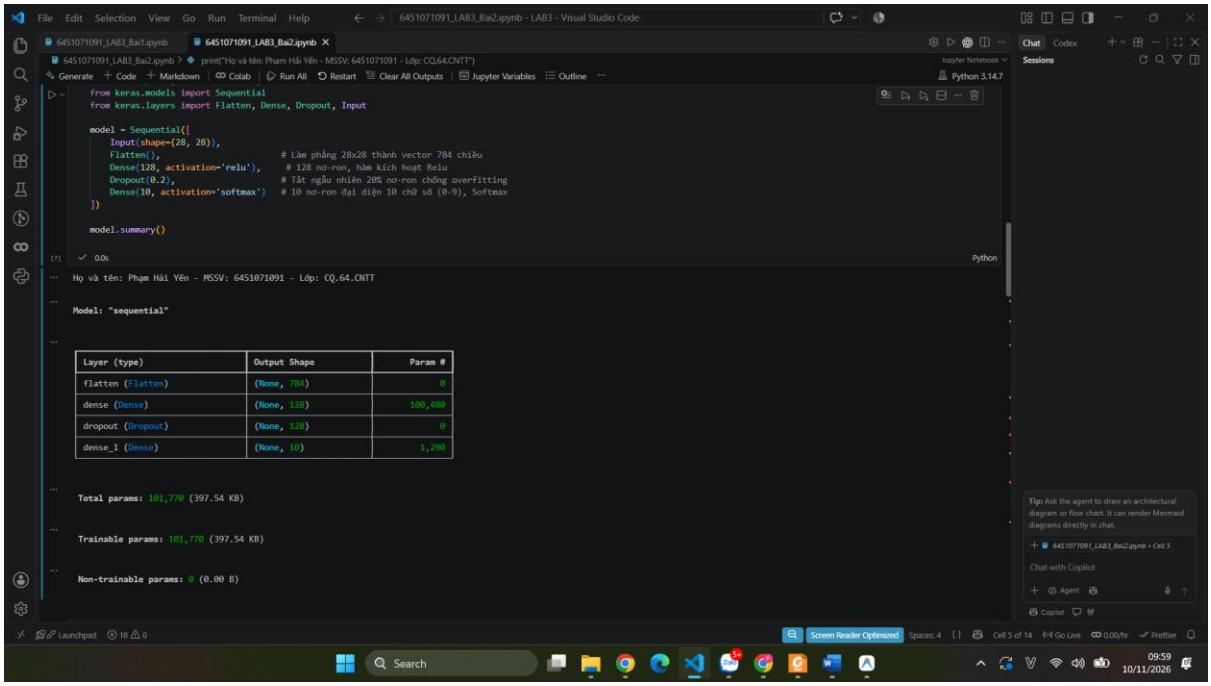
    Dropout(0.2),                       # Tắt ngẫu nhiên 20% nơ-ron chống
overfitting

    Dense(10, activation='softmax')     # 10 nơ-ron đại diện 10 chữ số (0-9),
Softmax

])


model.summary()
```

//Chụp hình kết quả



Câu f)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu f) Biên dịch mô hình

model.compile(

    optimizer='adam',

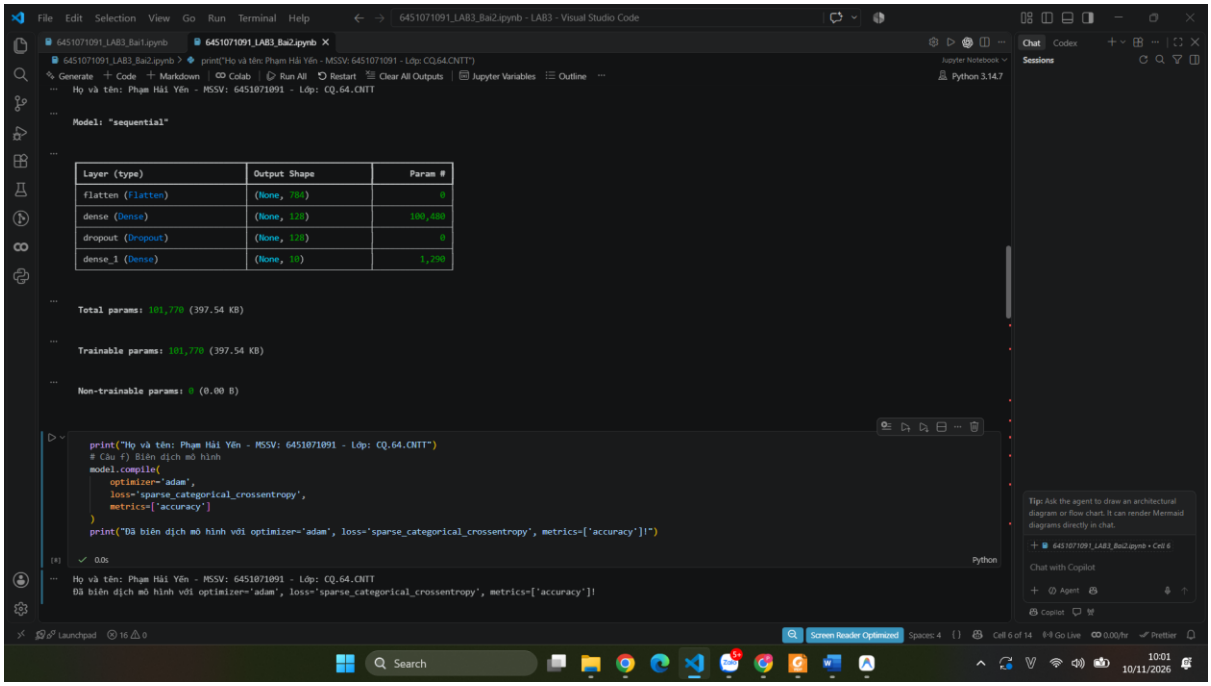
    loss='sparse_categorical_crossentropy',

    metrics=['accuracy']

)

print("Đã biên dịch mô hình với optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy']!")
```

//Chụp hình kết quả

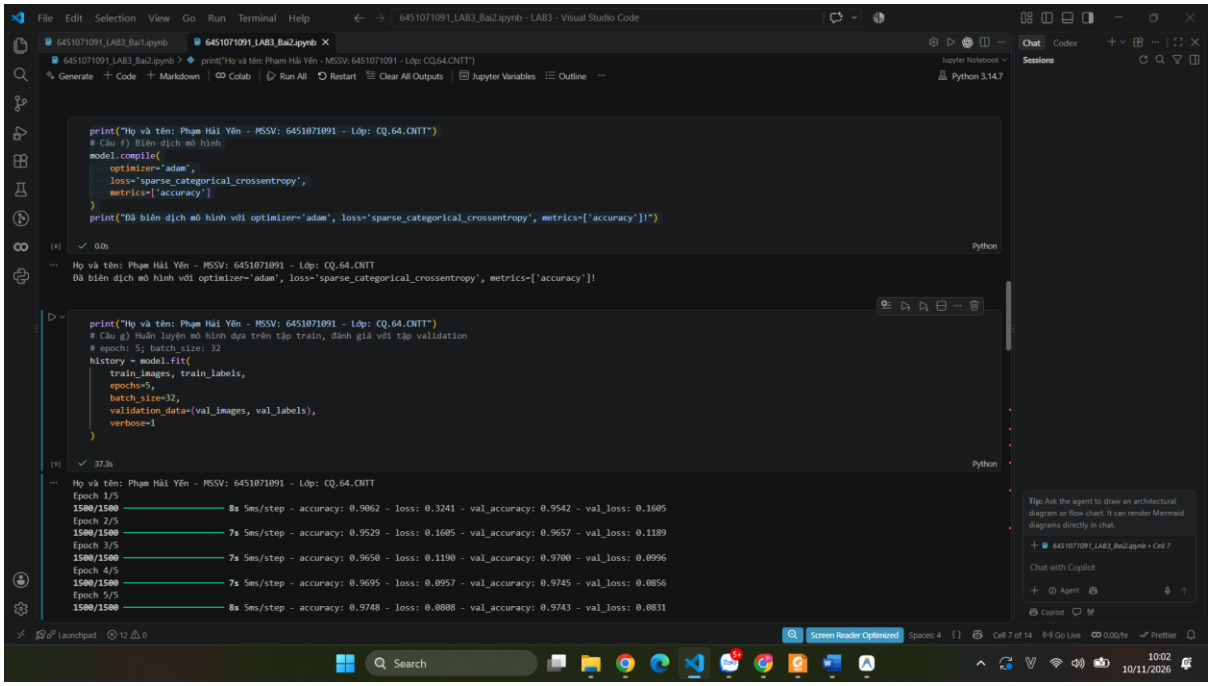


Câu g)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")
# Câu g) Huấn luyện mô hình dựa trên tập train, đánh giá với tập validation
# epoch: 5; batch_size: 32
history = model.fit(
    train_images, train_labels,
    epochs=5,
    batch_size=32,
    validation_data=(val_images, val_labels),
    verbose=1
)
```

//Chụp hình kết quả



Câu h)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu h) Vẽ biểu đồ thể hiện accuracy, loss sau mỗi epoch

plt.figure(figsize=(14, 5))


# Biểu đồ Accuracy

plt.subplot(1, 2, 1)

plt.plot(range(1, 6), history.history['accuracy'], 'bo-', label='Training
Accuracy')

plt.plot(range(1, 6), history.history['val_accuracy'], 'ro-',
label='Validation Accuracy')

plt.title('Training and Validation Accuracy qua các Epoch')

plt.xlabel('Epoch')

plt.ylabel('Accuracy')

plt.grid(True, linestyle='--', alpha=0.5)

plt.legend()


# Biểu đồ Loss

plt.subplot(1, 2, 2)
```

```
plt.plot(range(1, 6), history.history['loss'], 'bo-', label='Training
Loss')

plt.plot(range(1, 6), history.history['val_loss'], 'ro-', label='Validation
Loss')

plt.title('Training and Validation Loss qua các Epoch')

plt.xlabel('Epoch')

plt.ylabel('Loss')

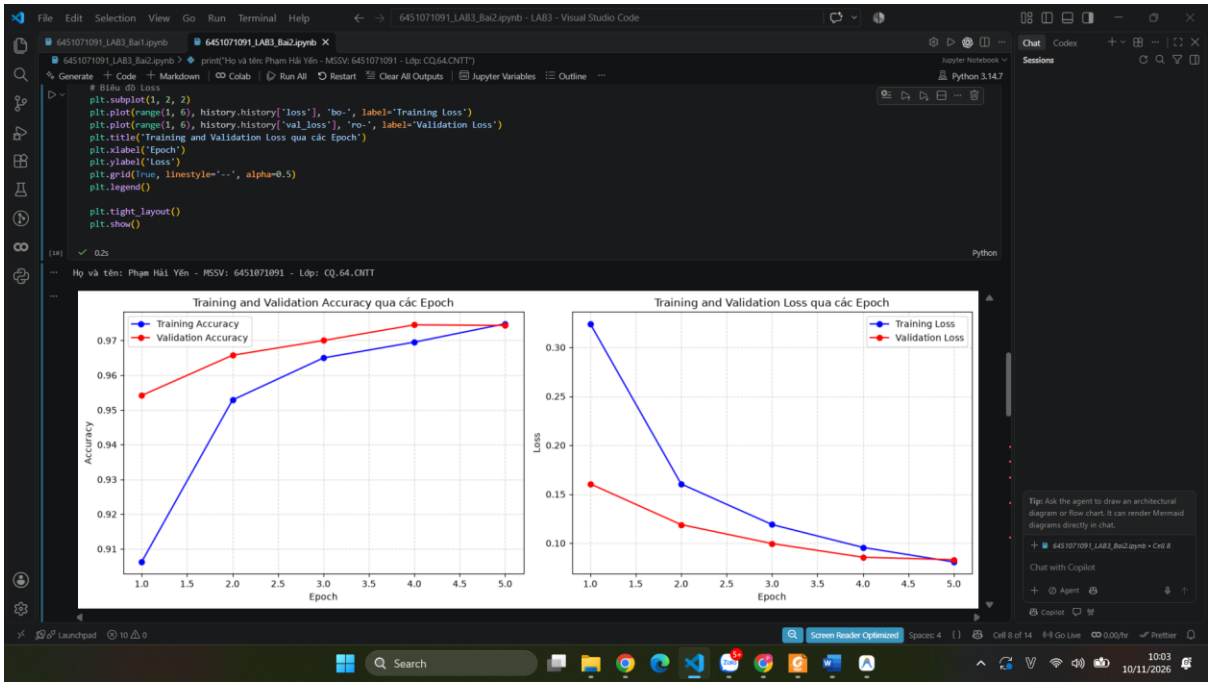
plt.grid(True, linestyle='--', alpha=0.5)

plt.legend()


plt.tight_layout()

plt.show()
```

//Chụp hình kết quả



Câu i)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu i) Dự đoán phân lớp cho tập test

y_prob = model.predict(test_images)

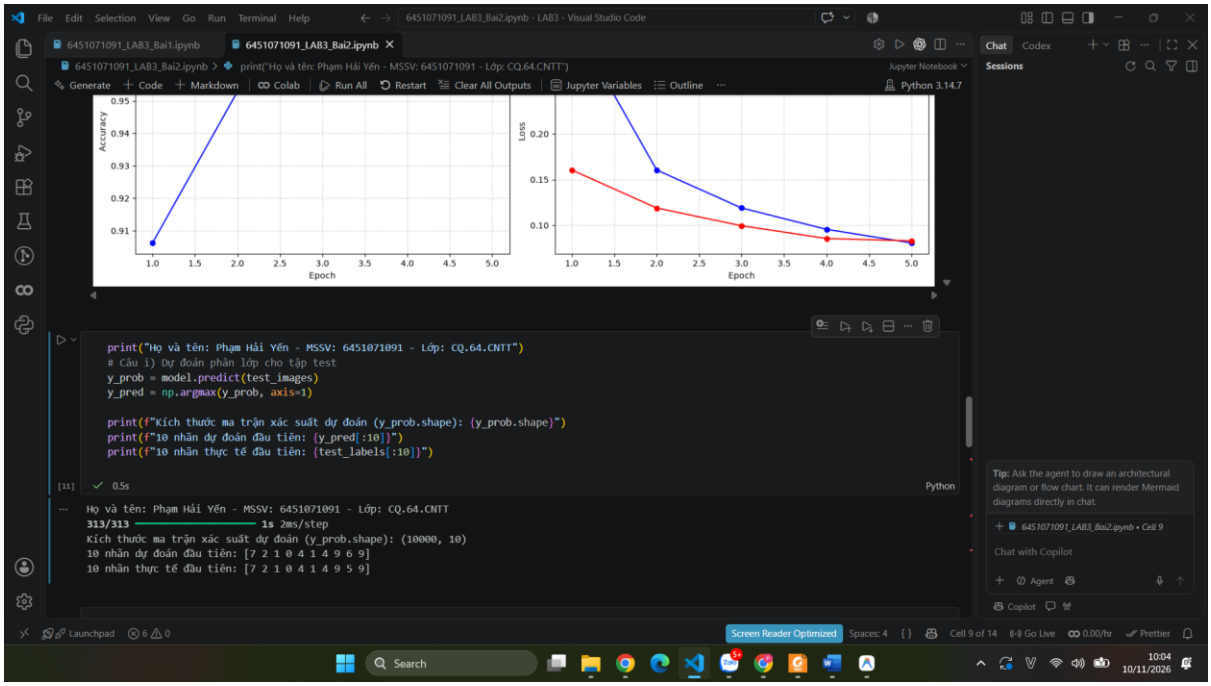
y_pred = np.argmax(y_prob, axis=1)
```

```
print(f"Kích thước ma trận xác suất dự đoán (y_prob.shape):
{y_prob.shape}")

print(f"10 nhãn dự đoán đầu tiên: {y_pred[:10]}")

print(f"10 nhãn thực tế đầu tiên: {test_labels[:10]}")
```

//Chụp hình kết quả



Câu j)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

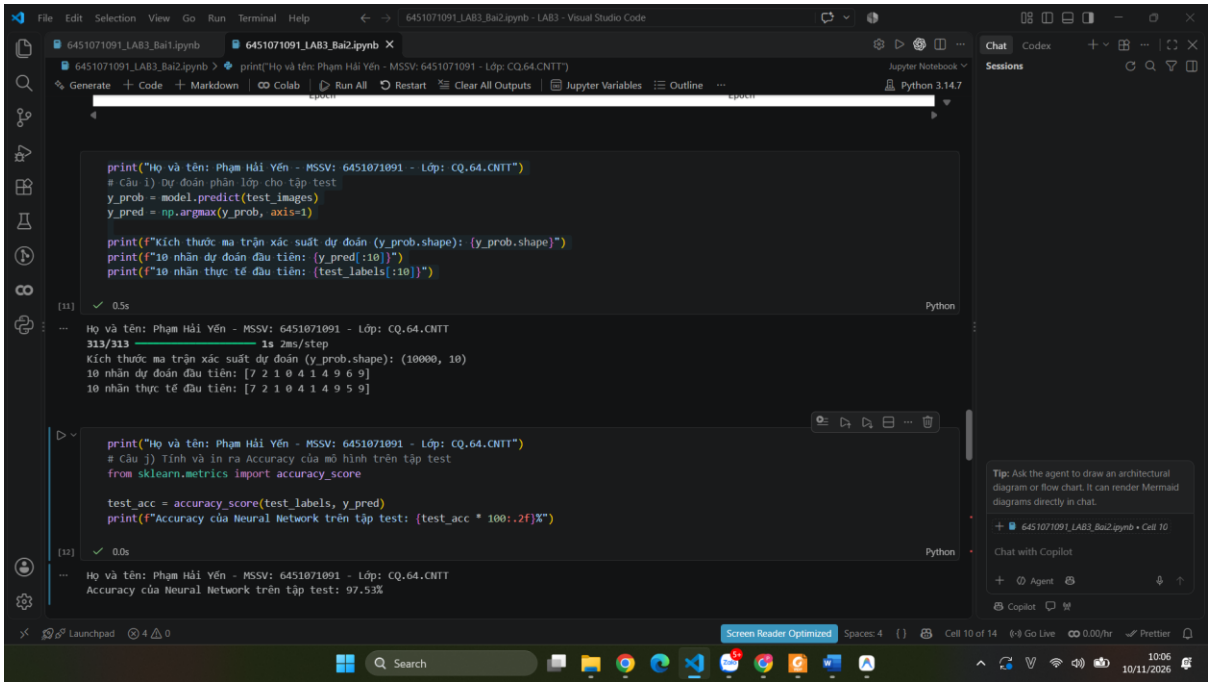
# Câu j) Tính và in ra Accuracy của mô hình trên tập test

from sklearn.metrics import accuracy_score


test_acc = accuracy_score(test_labels, y_pred)

print(f"Accuracy của Neural Network trên tập test: {test_acc * 100:.2f}%")
```

//Chụp hình kết quả



Câu k)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu k) Tính và in ra Precision, Recall, F1-score của từng lớp và trung
bình

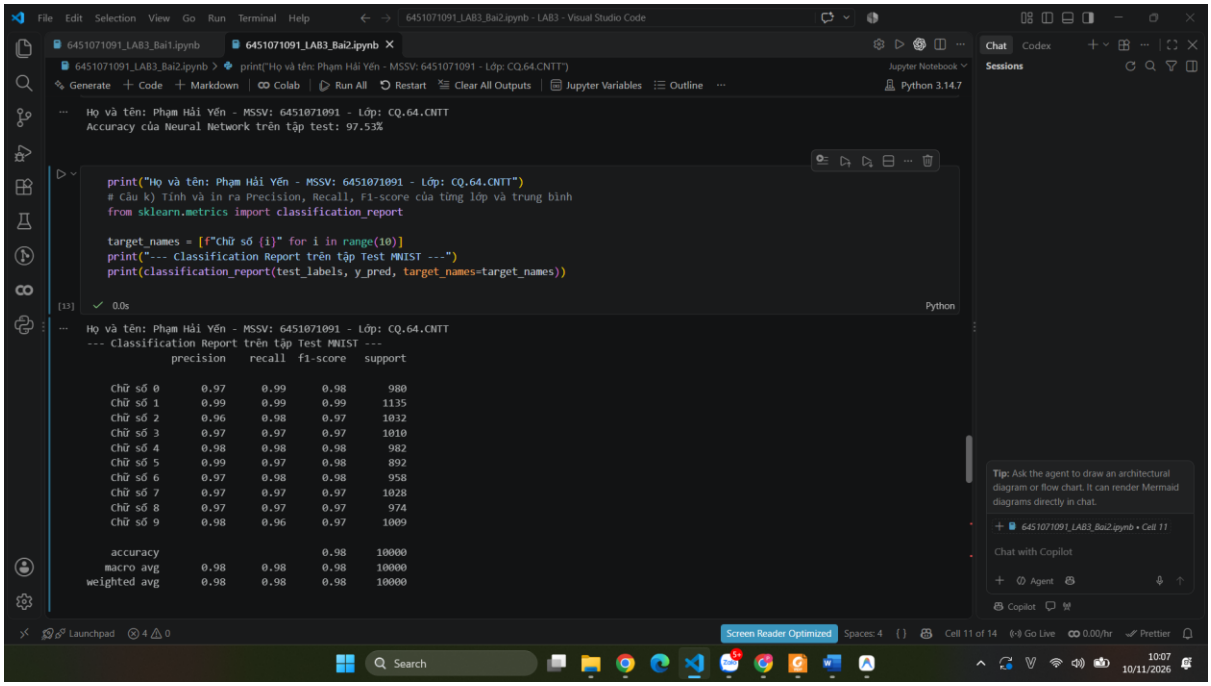
from sklearn.metrics import classification_report


target_names = [f"Chữ số {i}" for i in range(10)]

print("--- Classification Report trên tập Test MNIST ---")

print(classification_report(test_labels, y_pred,
target_names=target_names))
```

//Chụp hình kết quả



Câu l)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu l) Hiển thị confusion matrix bằng heat map

from sklearn.metrics import confusion_matrix


cm = confusion_matrix(test_labels, y_pred)


plt.figure(figsize=(9, 7))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',

            xticklabels=range(10), yticklabels=range(10))

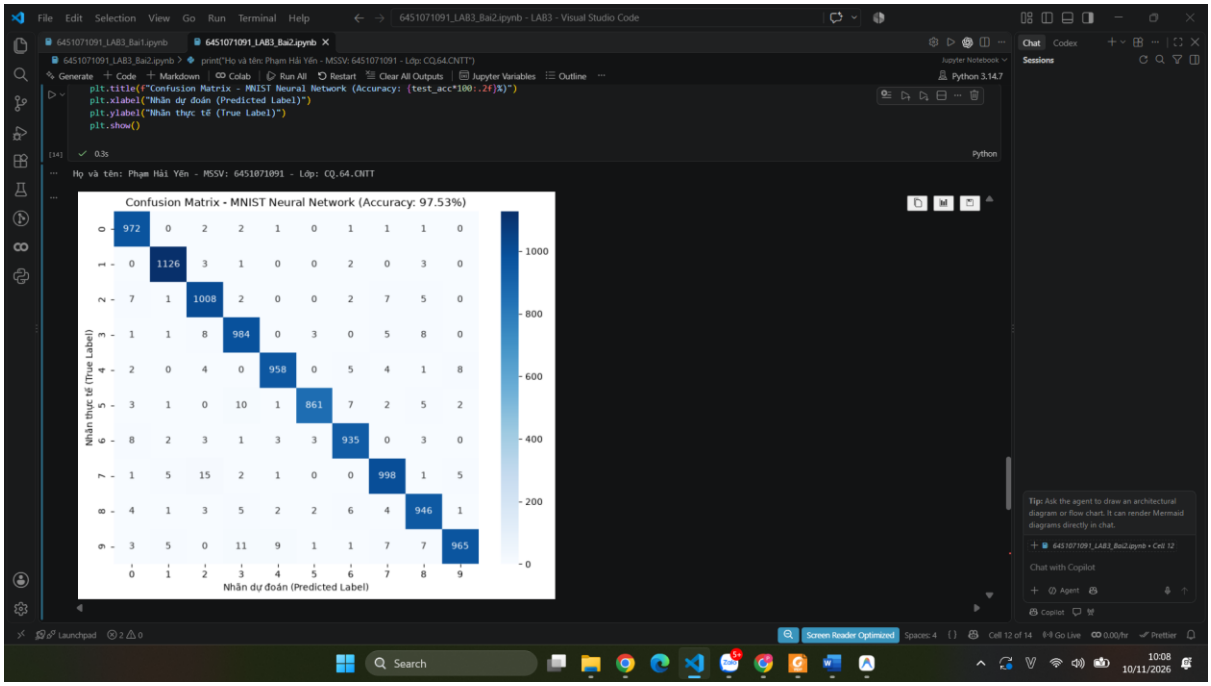
plt.title(f"Confusion Matrix - MNIST Neural Network (Accuracy:
{test_acc*100:.2f}%)")

plt.xlabel("Nhãn dự đoán (Predicted Label)")

plt.ylabel("Nhãn thực tế (True Label)")

plt.show()
```

//Chụp hình kết quả



Câu m)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu m) In ra kết quả phân lớp của 05 ảnh đầu tiên trong tập test

plt.figure(figsize=(15, 3))

for i in range(5):

    img = test_images[i]

    pred_label = y_pred[i]

    true_label = test_labels[i]


    plt.subplot(1, 5, i + 1)

    plt.imshow(img, cmap='gray')

    plt.title(f"True: {true_label} | Pred: {pred_label}",

              color='green' if pred_label == true_label else 'red',
fontsize=12)

    plt.axis('off')

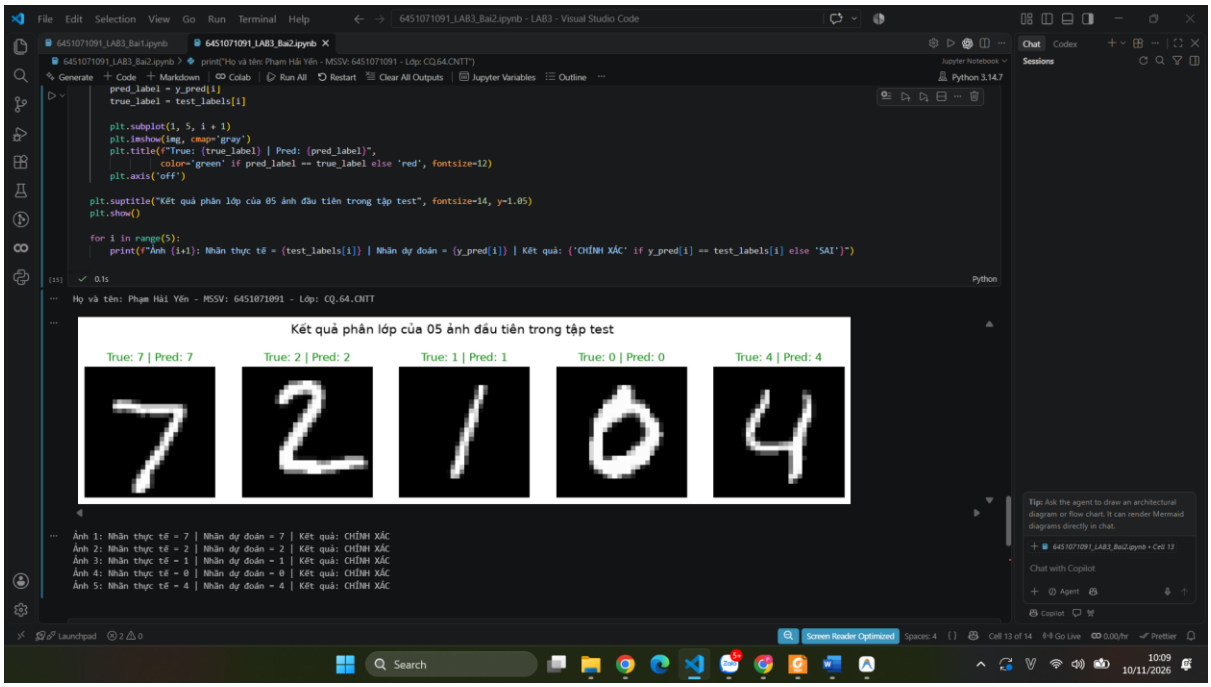

plt.suptitle("Kết quả phân lớp của 05 ảnh đầu tiên trong tập test",
fontsize=14, y=1.05)

plt.show()
```

```
for i in range(5):

    print(f"Ảnh {i+1}: Nhãn thực tế = {test_labels[i]} | Nhãn dự đoán =
{y_pred[i]} | Kết quả: {'CHÍNH XÁC' if y_pred[i] == test_labels[i] else
'SAI'}")
```

//Chụp hình kết quả



Câu n)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu n) In ra 05 kết quả đầu tiên phân lớp sai so với nhãn thực tế trong
tập test

# Tìm tất cả các index mà mô hình dự đoán sai

wrong_indices = np.where(y_pred != test_labels)[0]

print(f"Tổng số mẫu dự đoán sai trên tập test: {len(wrong_indices)} /
{len(test_labels)} mẫu.")


plt.figure(figsize=(15, 3))

for idx, i in enumerate(wrong_indices[:5]):

    img = test_images[i]

    pred_label = y_pred[i]

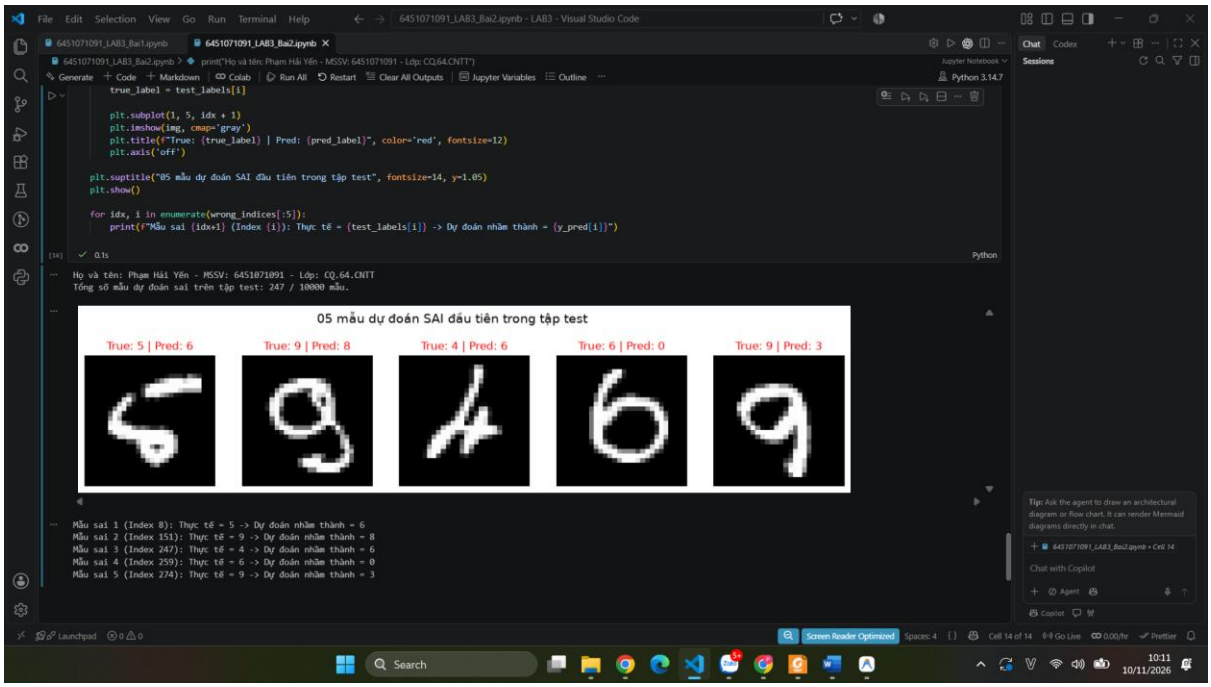
    true_label = test_labels[i]
```

```
    plt.subplot(1, 5, idx + 1)
    plt.imshow(img, cmap='gray')
    plt.title(f"True: {true_label} | Pred: {pred_label}", color='red',
fontsize=12)
    plt.axis('off')


plt.suptitle("05 mẫu dự đoán SAI đầu tiên trong tập test", fontsize=14,
y=1.05)
plt.show()


for idx, i in enumerate(wrong_indices[:5]):
    print(f"Mẫu sai {idx+1} (Index {i}): Thực tế = {test_labels[i]} -> Dự
đoán nhầm thành = {y_pred[i]}")
```

//Chụp hình kết quả



Bài 3 (Trang 7)

Câu abc)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu a, b, c) Đọc, khảo sát và chuẩn hóa dữ liệu MNIST

import os

os.environ['KERAS_BACKEND'] = 'torch'
```

```
import keras

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


# a) Đọc bộ dữ liệu train, test

(train_images, train_labels), (test_images, test_labels) =
keras.datasets.mnist.load_data()


# b) In kích thước

print(f"Kích thước tập train ban đầu: {train_images.shape}, nhãn:
{train_labels.shape}")

print(f"Kích thước tập test:          {test_images.shape}, nhãn:
{test_labels.shape}")

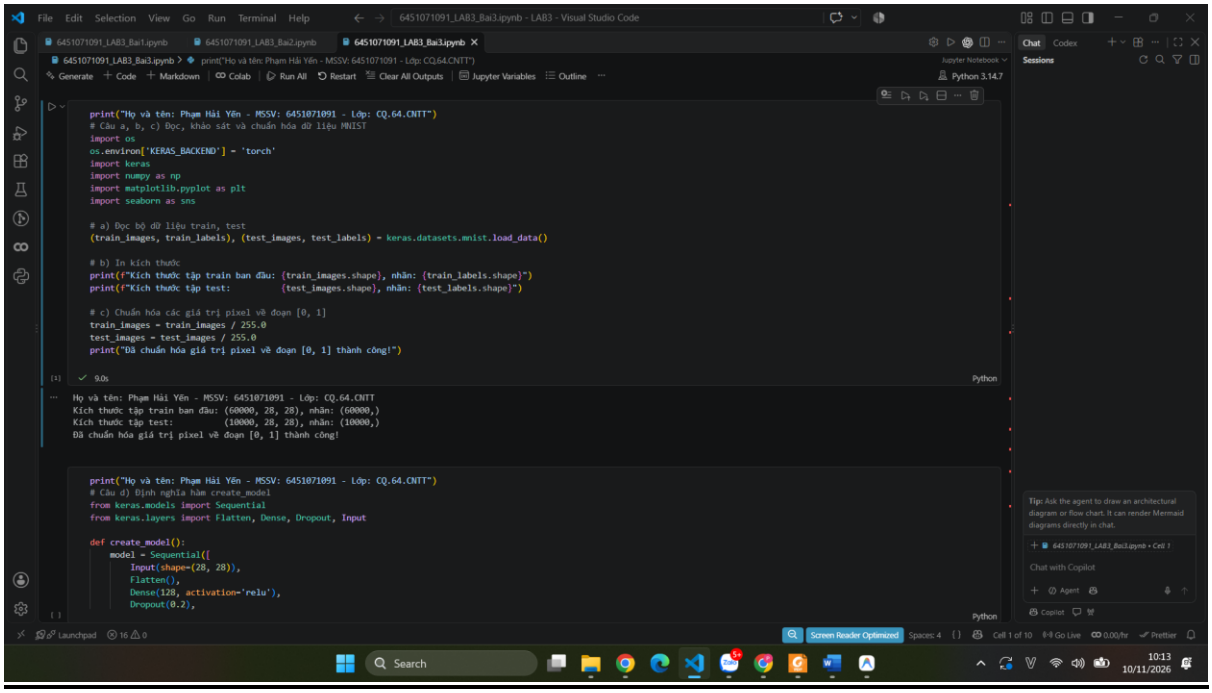

# c) Chuẩn hóa các giá trị pixel về đoạn [0, 1]

train_images = train_images / 255.0

test_images = test_images / 255.0

print("Đã chuẩn hóa giá trị pixel về đoạn [0, 1] thành công!")
```

//Chụp hình kết quả



Câu d)

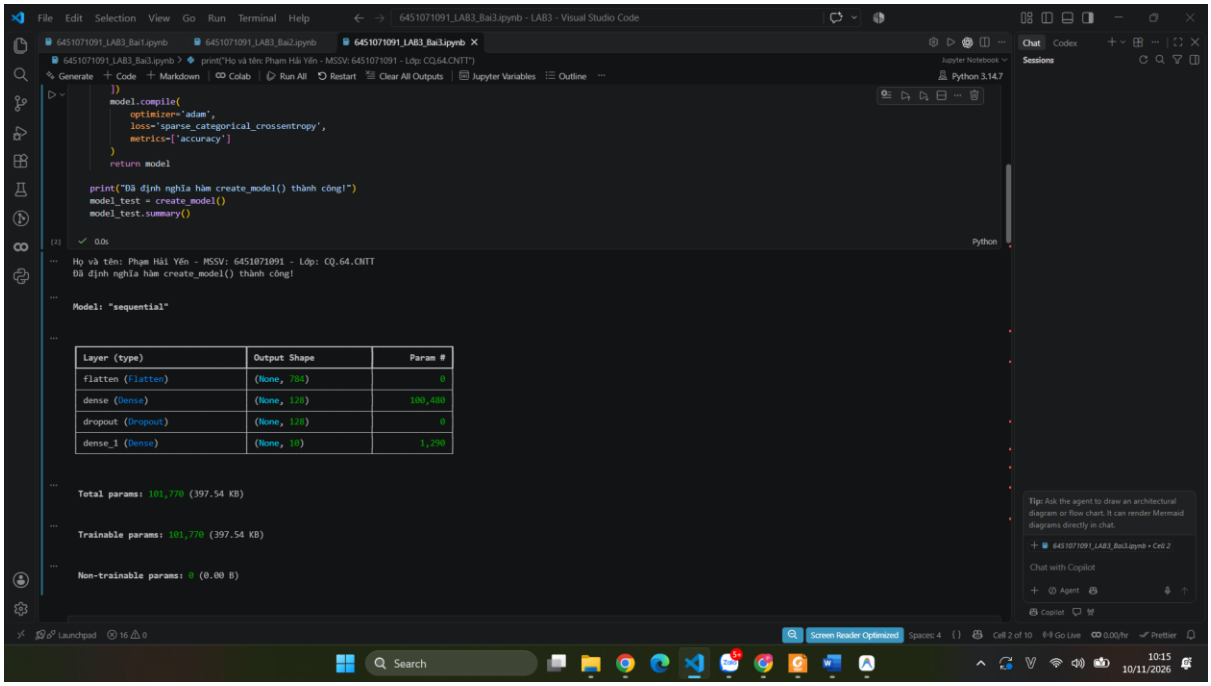
//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")
# Câu d) Định nghĩa hàm create_model
from keras.models import Sequential
from keras.layers import Flatten, Dense, Dropout, Input


def create_model():
    model = Sequential([
        Input(shape=(28, 28)),
        Flatten(),
        Dense(128, activation='relu'),
        Dropout(0.2),
        Dense(10, activation='softmax')
    ])
    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model


print("Đã định nghĩa hàm create_model() thành công!")
model_test = create_model()
model_test.summary()
```

//Chụp hình kết quả



Câu e)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu e) Huấn luyện mô hình bằng k-fold cross validation (k=10)

from sklearn.model_selection import KFold


kfold = KFold(n_splits=10, shuffle=True, random_state=42)


models = []

accuracy_per_fold = []

loss_per_fold = []


# Sử dụng tập con 15000 mẫu để quá trình K-Fold 10 lần thực thi nhanh chóng và hiệu quả

subset_size = 15000

train_images_sub = train_images[:subset_size]

train_labels_sub = train_labels[:subset_size]


fold_no = 1

for train_index, val_index in kfold.split(train_images_sub):

    print(f"\n--- Đang huấn luyện Fold {fold_no} / 10 ---")
```

```
    # Gán dữ liệu cho fold
    X_train_fold, X_val_fold = train_images_sub[train_index],
train_images_sub[val_index]
    y_train_fold, y_val_fold = train_labels_sub[train_index],
train_labels_sub[val_index]

    # Khởi tạo mô hình mới từ hàm create_model()
    model = create_model()

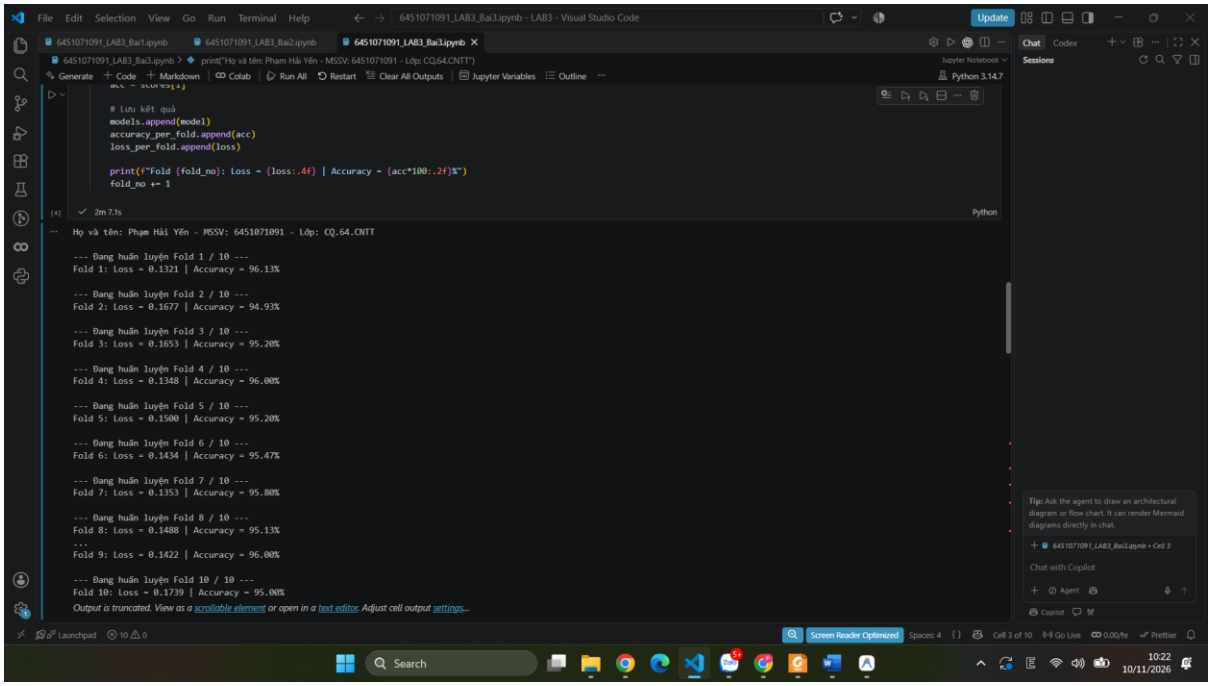
    # Huấn luyện mô hình (epoch: 5, batch_size: 32)
    model.fit(X_train_fold, y_train_fold, epochs=5, batch_size=32,
verbose=0)

    # Đánh giá trên tập validation của fold
    scores = model.evaluate(X_val_fold, y_val_fold, verbose=0)
    loss = scores[0]
    acc = scores[1]

    # Lưu kết quả
    models.append(model)
    accuracy_per_fold.append(acc)
    loss_per_fold.append(loss)

    print(f"Fold {fold_no}: Loss = {loss:.4f} | Accuracy = {acc*100:.2f}%")
    fold_no += 1
```

//Chụp hình kết quả



Câu f)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu f) Tính và in ra Mean Accuracy, Standard deviation accuracy, Mean
Loss, Standard deviation loss

mean_acc = np.mean(accuracy_per_fold)

std_acc = np.std(accuracy_per_fold)

mean_loss = np.mean(loss_per_fold)

std_loss = np.std(loss_per_fold)


print("--- KẾT QUẢ ĐÁNH GIÁ 10-FOLD CROSS VALIDATION ---")

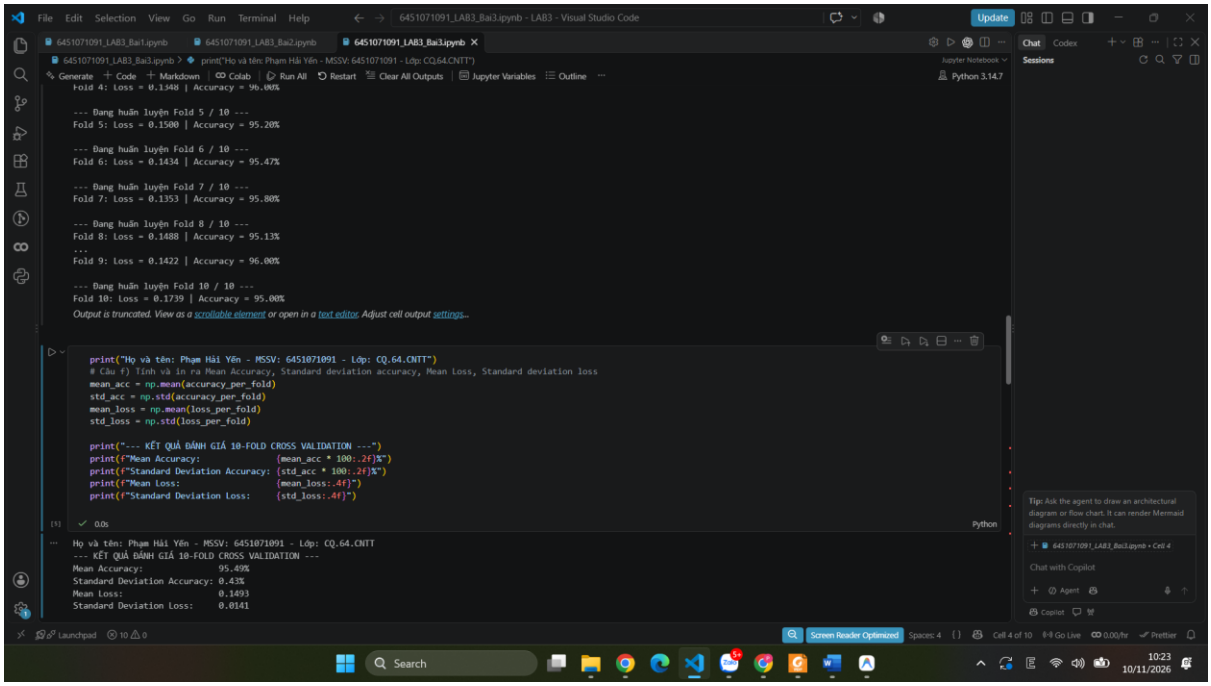
print(f"Mean Accuracy:                {mean_acc * 100:.2f}%")

print(f"Standard Deviation Accuracy: {std_acc * 100:.2f}%")

print(f"Mean Loss:                    {mean_loss:.4f}")

print(f"Standard Deviation Loss:      {std_loss:.4f}")
```

//Chụp hình kết quả



Câu g)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu g) Vẽ biểu đồ thể hiện accuracy, loss sau mỗi fold

plt.figure(figsize=(14, 5))


# Biểu đồ Accuracy theo từng Fold

plt.subplot(1, 2, 1)

plt.plot(range(1, 11), [a * 100 for a in accuracy_per_fold], 'bo-',
linewidth=2, markersize=8)

plt.axhline(mean_acc * 100, color='r', linestyle='--', label=f'Mean Acc:
{mean_acc*100:.2f}%')

plt.title('Accuracy qua 10 Fold Cross Validation')

plt.xlabel('Fold')

plt.ylabel('Accuracy (%)')

plt.xticks(range(1, 11))

plt.grid(True, linestyle='--', alpha=0.5)

plt.legend()


# Biểu đồ Loss theo từng Fold

plt.subplot(1, 2, 2)
```

```
plt.plot(range(1, 11), loss_per_fold, 'ro-', linewidth=2, markersize=8)

plt.axhline(mean_loss, color='b', linestyle='--', label=f'Mean Loss: {mean_loss:.4f}')

plt.title('Loss qua 10 Fold Cross Validation')

plt.xlabel('Fold')

plt.ylabel('Loss')

plt.xticks(range(1, 11))

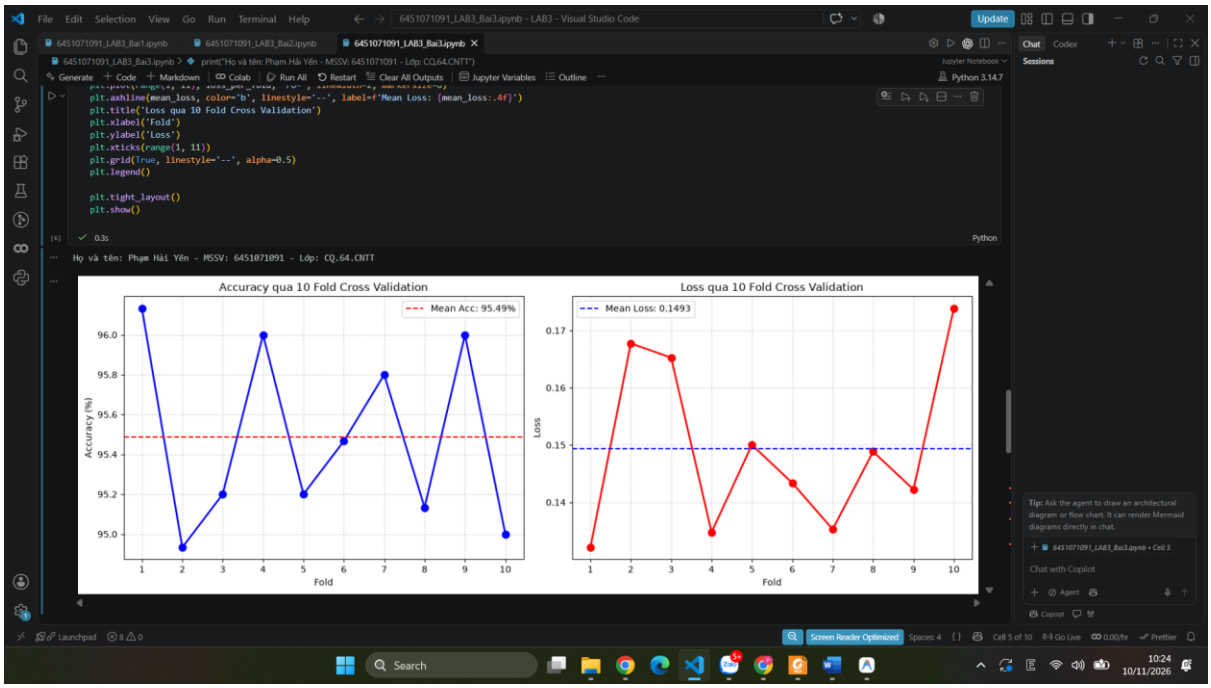
plt.grid(True, linestyle='--', alpha=0.5)

plt.legend()


plt.tight_layout()

plt.show()
```

//Chụp hình kết quả



Câu h)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu h) Sử dụng mô hình có Accuracy tốt nhất giữa các fold và phân lớp cho tập test

best_fold_idx = np.argmax(accuracy_per_fold)

best_model = models[best_fold_idx]
```

```
print(f"Mô hình tốt nhất thuộc về Fold {best_fold_idx + 1} với Validation
Accuracy: {accuracy_per_fold[best_fold_idx]*100:.2f}%")

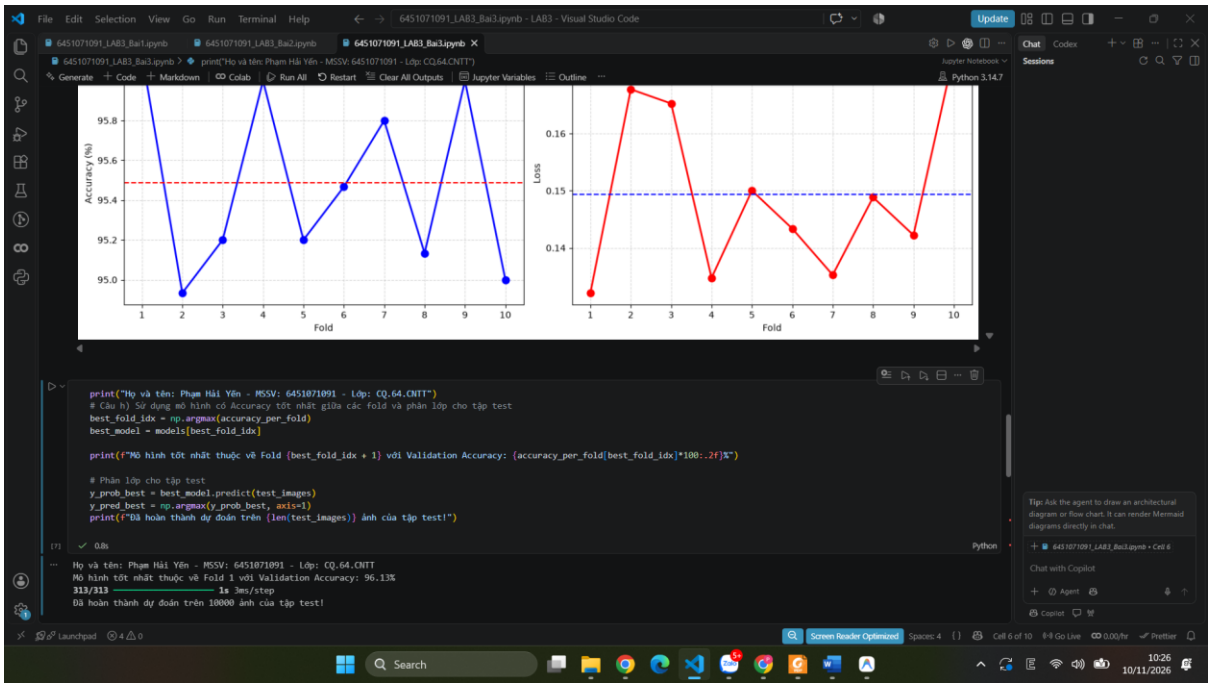

# Phân lớp cho tập test

y_prob_best = best_model.predict(test_images)

y_pred_best = np.argmax(y_prob_best, axis=1)

print(f"Đã hoàn thành dự đoán trên {len(test_images)} ảnh của tập test!")
```

//Chụp hình kết quả



Câu i)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

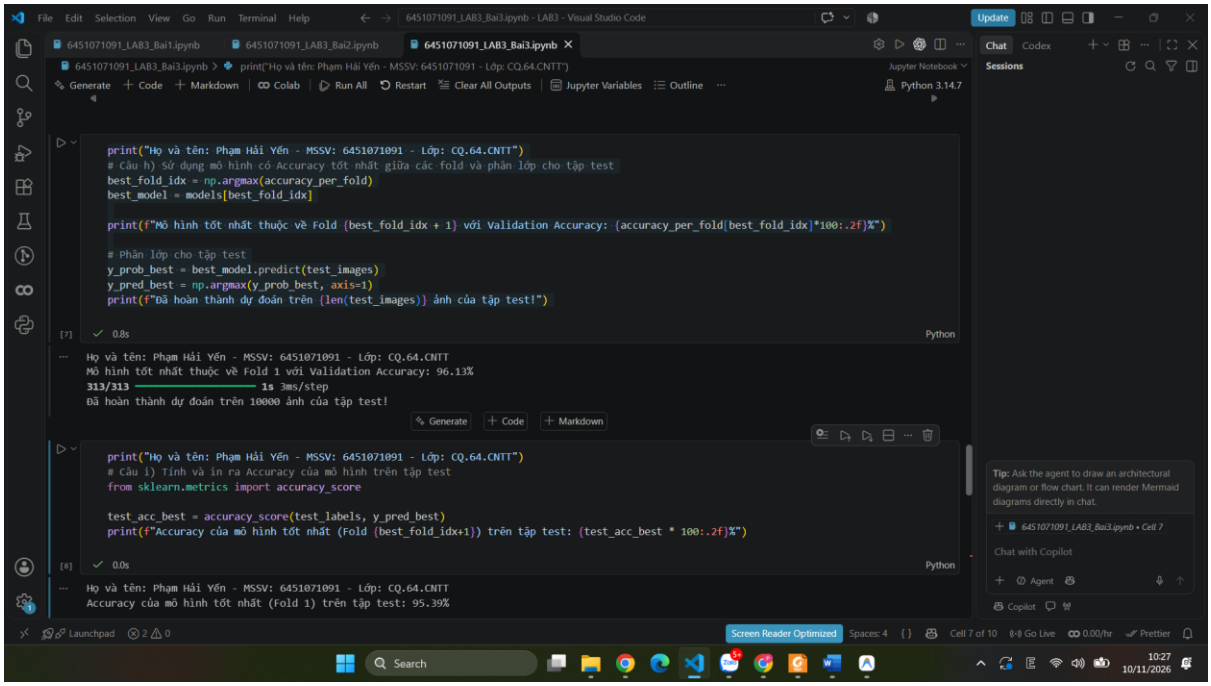
# Câu i) Tính và in ra Accuracy của mô hình trên tập test

from sklearn.metrics import accuracy_score


test_acc_best = accuracy_score(test_labels, y_pred_best)

print(f"Accuracy của mô hình tốt nhất (Fold {best_fold_idx+1}) trên tập
test: {test_acc_best * 100:.2f}%")
```

//Chụp hình kết quả



Câu j)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu j) Tính và in ra Precision, Recall, F1-score của từng lớp và trung bình

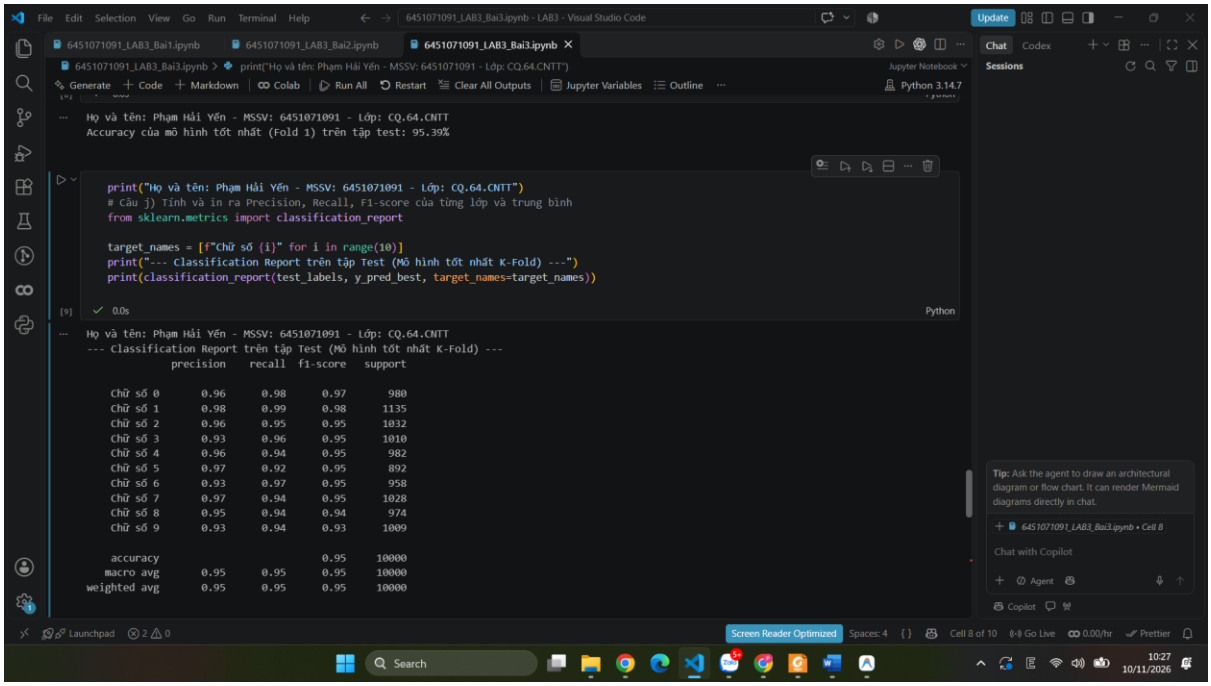
from sklearn.metrics import classification_report


target_names = [f"Chữ số {i}" for i in range(10)]

print("--- Classification Report trên tập Test (Mô hình tốt nhất K-Fold) ---")

print(classification_report(test_labels, y_pred_best, target_names=target_names))
```

//Chụp hình kết quả



Câu k)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu k) Hiển thị confusion matrix bằng heat map (bản đồ nhiệt)

from sklearn.metrics import confusion_matrix


cm_best = confusion_matrix(test_labels, y_pred_best)


plt.figure(figsize=(9, 7))

sns.heatmap(cm_best, annot=True, fmt='d', cmap='Blues',

            xticklabels=range(10), yticklabels=range(10))

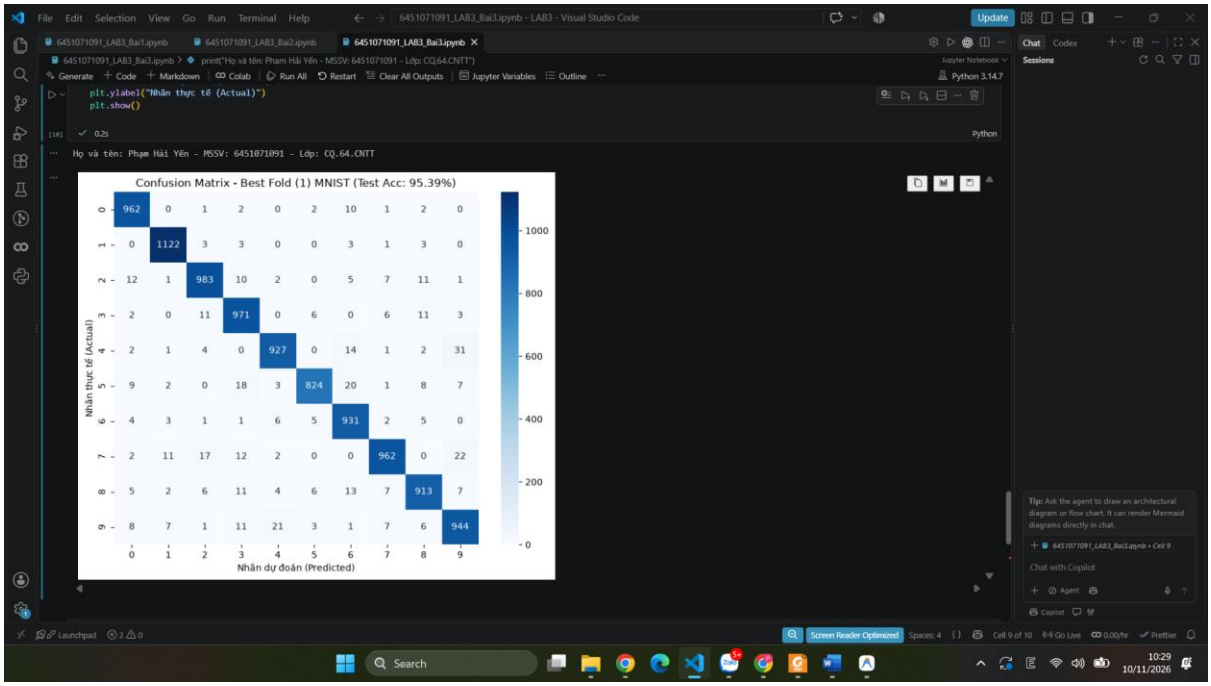
plt.title(f"Confusion Matrix - Best Fold ({best_fold_idx+1}) MNIST (Test
Acc: {test_acc_best*100:.2f}%)")

plt.xlabel("Nhãn dự đoán (Predicted)")

plt.ylabel("Nhãn thực tế (Actual)")

plt.show()
```

//Chụp hình kết quả



Câu l)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu l) In ra kết quả phân lớp của 05 ảnh đầu tiên trong tập test

plt.figure(figsize=(15, 3))

for i in range(5):

    img = test_images[i]

    pred_label = y_pred_best[i]

    true_label = test_labels[i]


    plt.subplot(1, 5, i + 1)

    plt.imshow(img, cmap='gray')

    plt.title(f"True: {true_label} | Pred: {pred_label}",

              color='green' if pred_label == true_label else 'red',
fontsize=12)

    plt.axis('off')

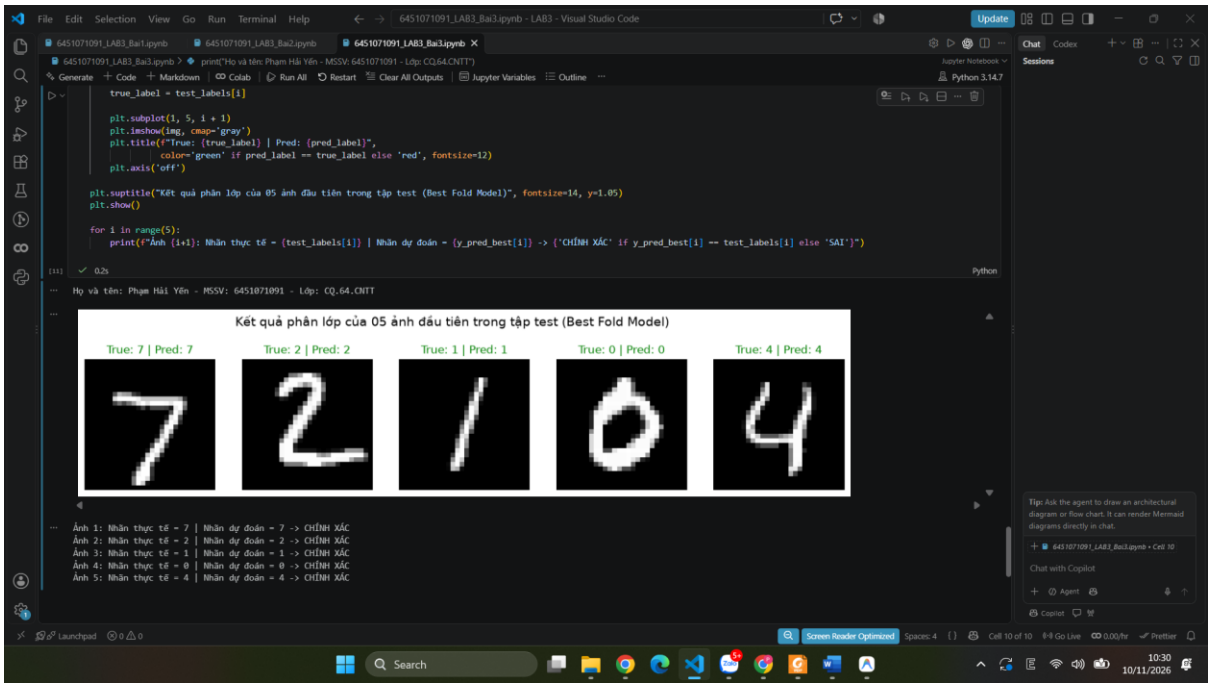

plt.suptitle("Kết quả phân lớp của 05 ảnh đầu tiên trong tập test (Best
Fold Model)", fontsize=14, y=1.05)

plt.show()
```

```
for i in range(5):

    print(f"Ảnh {i+1}: Nhãn thực tế = {test_labels[i]} | Nhãn dự đoán = {y_pred_best[i]} -> {'CHÍNH XÁC' if y_pred_best[i] == test_labels[i] else 'SAI'}")
```

//Chụp hình kết quả



Bài 4 (Trang 9)

Câu c)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu c) Định nghĩa hàm create_model

import os

os.environ['KERAS_BACKEND'] = 'torch'

import keras

from keras.models import Sequential

from keras.layers import Dense, Input


def create_model():

    model = Sequential([

        Input(shape=(4,)),

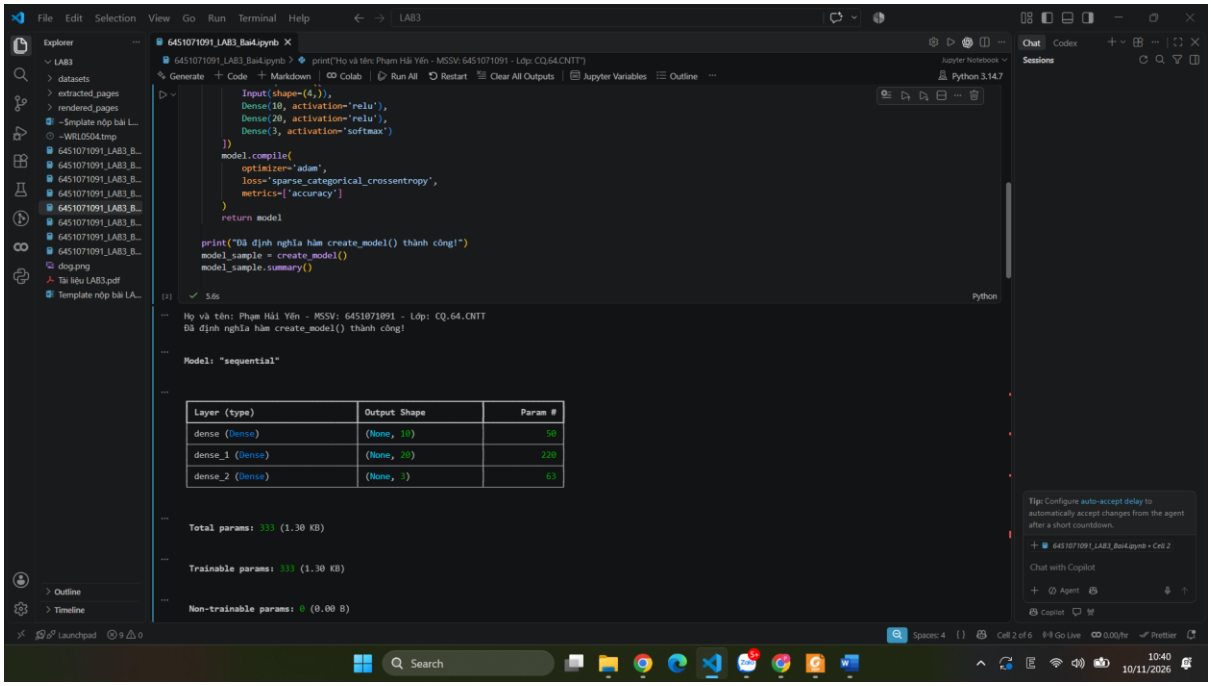
        Dense(10, activation='relu'),

        Dense(20, activation='relu'),
```

```
        Dense(3, activation='softmax')
    ])
    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

print("Đã định nghĩa hàm create_model() thành công!")
model_sample = create_model()
model_sample.summary()
```

//Chụp hình kết quả



Câu d)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")
# Câu d) Huấn luyện mô hình bằng k-fold cross validation (với k=10)
from sklearn.model_selection import KFold
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
```

```
kfold = KFold(n_splits=10, shuffle=True, random_state=42)


models = []
acc_list = []
precision_list = []
recall_list = []
f1_list = []
class_metrics_list = []


fold = 1
for train_idx, val_idx in kfold.split(X):
    X_train_f, X_val_f = X[train_idx], X[val_idx]
    y_train_f, y_val_f = Y[train_idx], Y[val_idx]


    model = create_model()
    # Huấn luyện mô hình với epochs=100, batch_size=1
    model.fit(X_train_f, y_train_f, epochs=100, batch_size=1, verbose=0)


    # Dự đoán
    y_pred_prob = model.predict(X_val_f, verbose=0)
    y_pred_f = np.argmax(y_pred_prob, axis=1)


    # Tính các độ đo tổng thể (weighted average)
    acc = accuracy_score(y_val_f, y_pred_f)
    p, r, f1, _ = precision_recall_fscore_support(y_val_f, y_pred_f,
average='weighted', zero_division=0)


    # Tính các độ đo theo từng class
    p_cls, r_cls, f1_cls, _ = precision_recall_fscore_support(y_val_f,
y_pred_f, labels=[0, 1, 2], average=None, zero_division=0)
    class_metrics_list.append((p_cls, r_cls, f1_cls))


    models.append(model)
    acc_list.append(acc)
```

```
    precision_list.append(p)

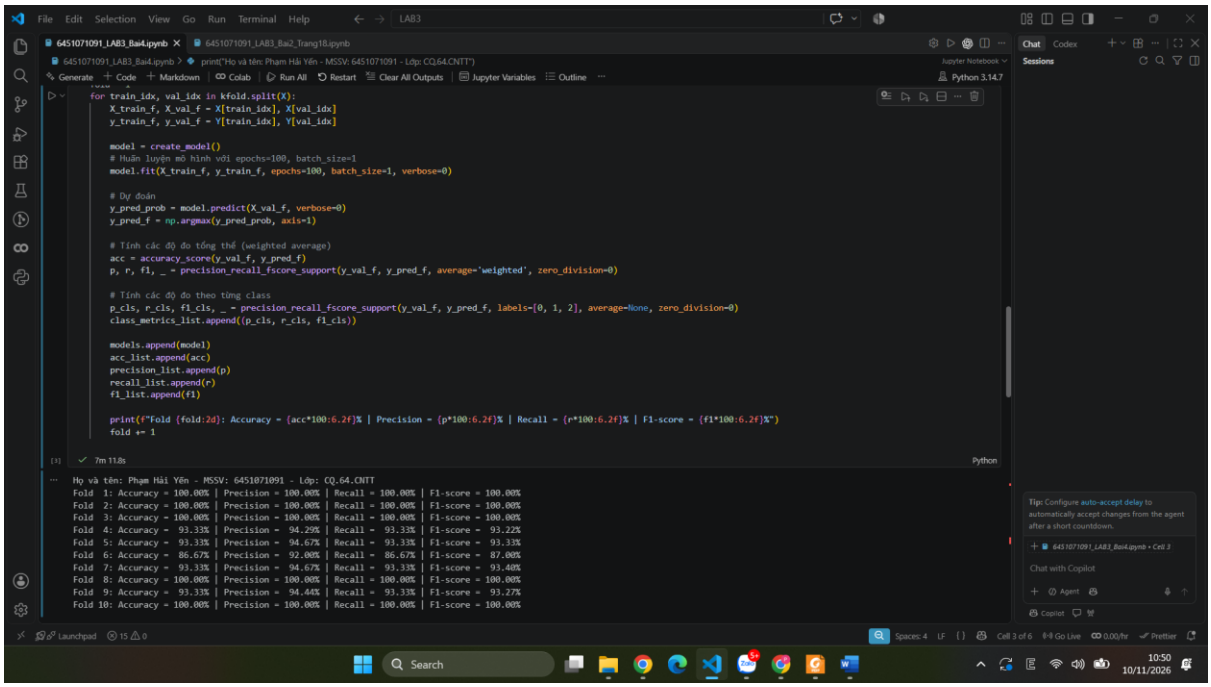
    recall_list.append(r)

    f1_list.append(f1)


    print(f"Fold {fold:2d}: Accuracy = {acc*100:6.2f}% | Precision =
{p*100:6.2f}% | Recall = {r*100:6.2f}% | F1-score = {f1*100:6.2f}%")

    fold += 1
```

//Chụp hình kết quả



Câu e)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu e) In ra Average Accuracy, Average Recall, Average Precision, Average
F1-score sau k-fold

avg_acc = np.mean(acc_list)

avg_prec = np.mean(precision_list)

avg_rec = np.mean(recall_list)

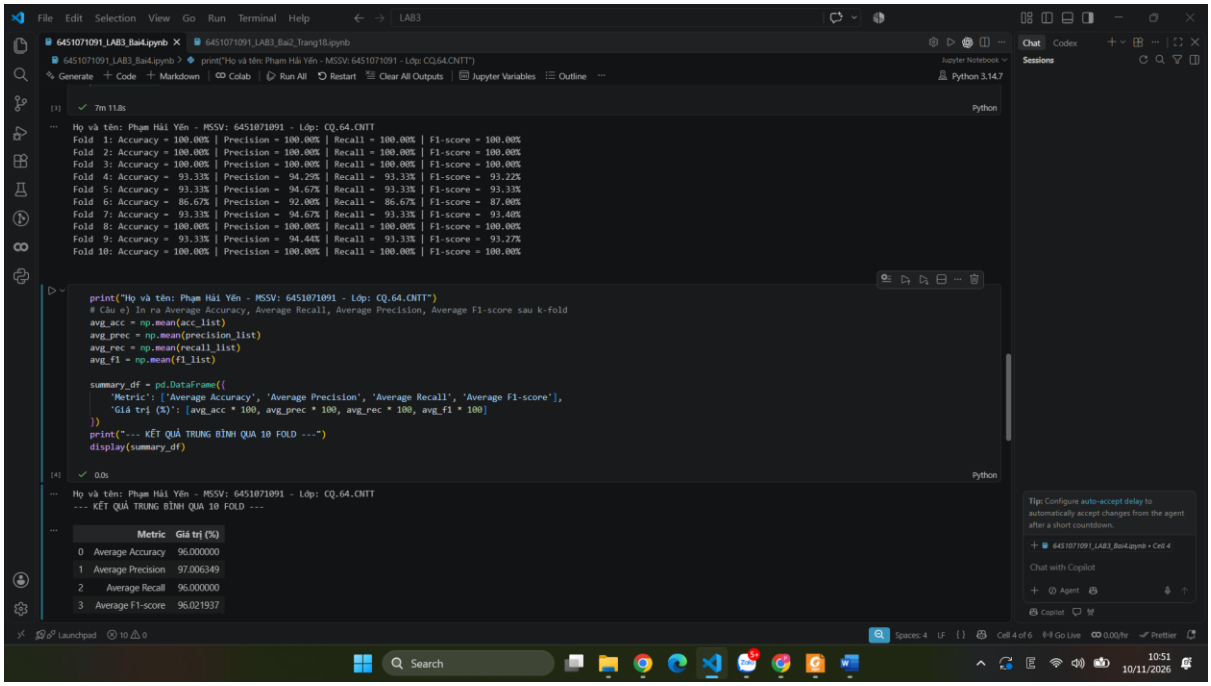
avg_f1 = np.mean(f1_list)


summary_df = pd.DataFrame({

    'Metric': ['Average Accuracy', 'Average Precision', 'Average Recall',
'Average F1-score'],
```

```
    'Giá trị (%)': [avg_acc * 100, avg_prec * 100, avg_rec * 100, avg_f1 * 100]
})
print("--- KẾT QUẢ TRUNG BÌNH QUA 10 FOLD ---")
display(summary_df)
```

//Chụp hình kết quả



Câu f)

//dán mã nguồn

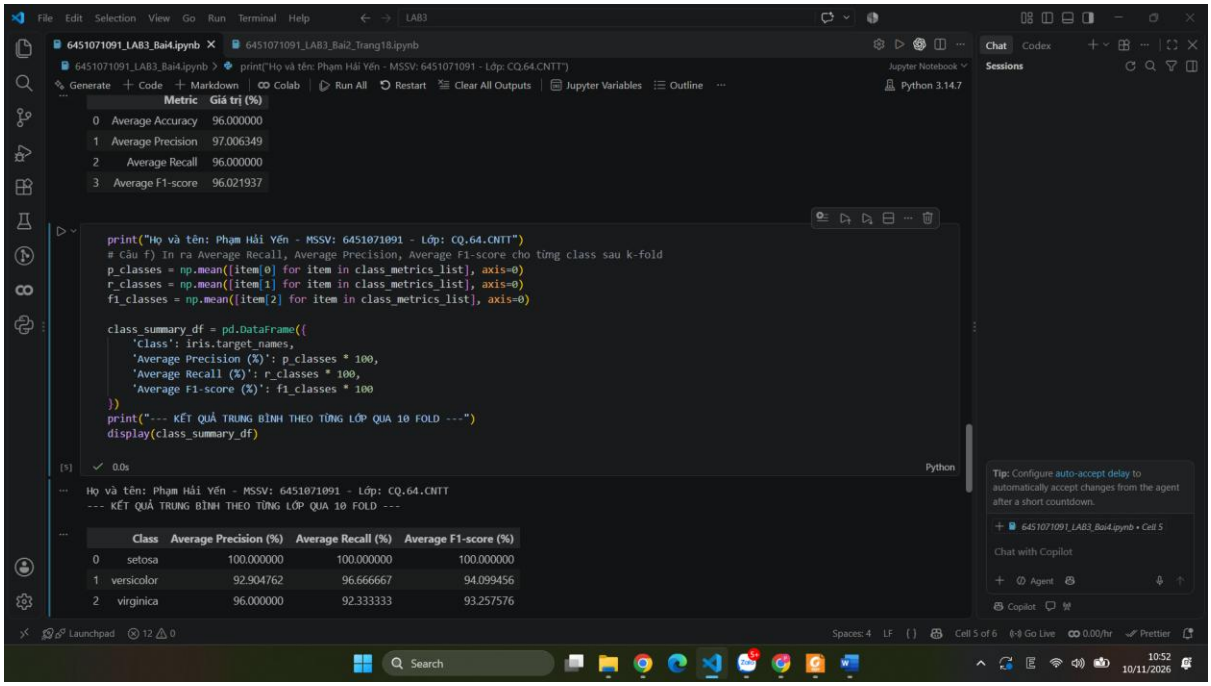
```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")
# Câu f) In ra Average Recall, Average Precision, Average F1-score cho từng class sau k-fold
p_classes = np.mean([item[0] for item in class_metrics_list], axis=0)
r_classes = np.mean([item[1] for item in class_metrics_list], axis=0)
f1_classes = np.mean([item[2] for item in class_metrics_list], axis=0)

class_summary_df = pd.DataFrame({
    'Class': iris.target_names,
    'Average Precision (%)': p_classes * 100,
    'Average Recall (%)': r_classes * 100,
    'Average F1-score (%)': f1_classes * 100
})
```

```
print("--- KẾT QUẢ TRUNG BÌNH THEO TỪNG LỚP QUA 10 FOLD ---")

display(class_summary_df)
```

//Chụp hình kết quả



Câu g)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu g) Sử dụng mô hình có F1-score tốt nhất đã huấn luyện ở trên cho 03 mẫu dữ liệu mới

best_idx = np.argmax(f1_list)

best_model = models[best_idx]

print(f"Mô hình tốt nhất là Fold {best_idx + 1} với F1-score = {f1_list[best_idx]*100:.2f}%\n")


new_samples = np.array([

    [6.2, 2.9, 4.3, 1.3],

    [5.1, 3.5, 1.4, 0.2],

    [7.3, 2.8, 6.4, 2.1]

])

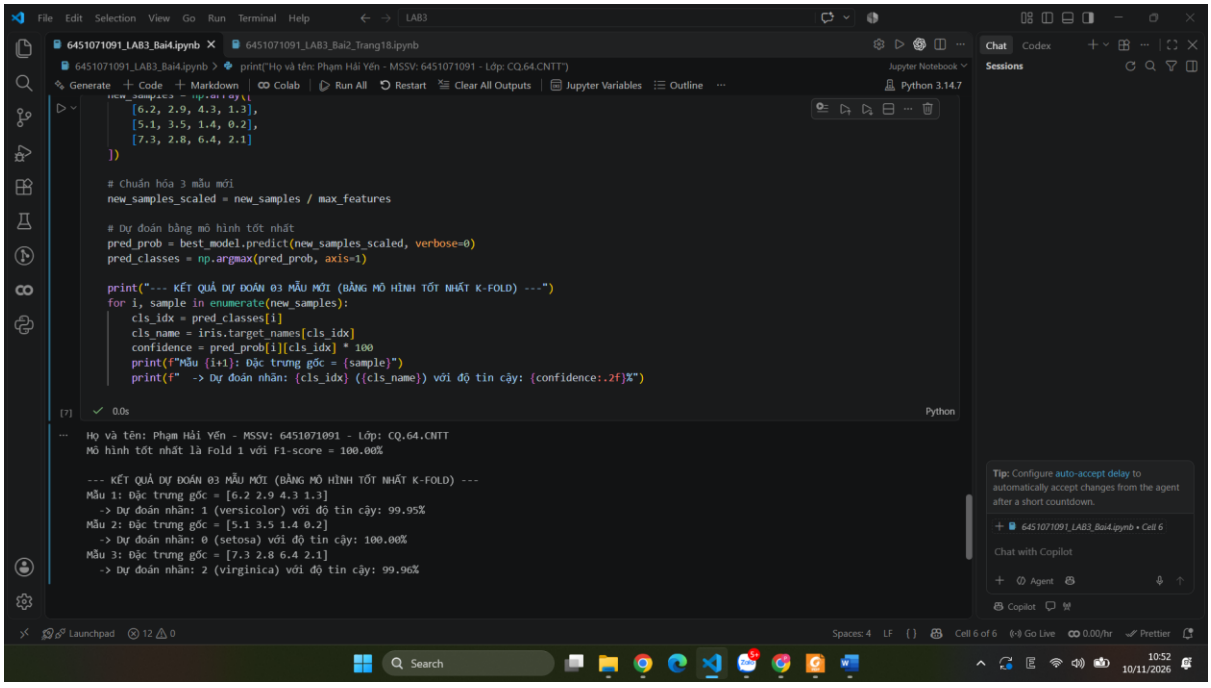

# Chuẩn hóa 3 mẫu mới

new_samples_scaled = new_samples / max_features
```

```
# Dự đoán bằng mô hình tốt nhất
pred_prob = best_model.predict(new_samples_scaled, verbose=0)
pred_classes = np.argmax(pred_prob, axis=1)

print("--- KẾT QUẢ DỰ ĐOÁN 03 MẪU MỚI (BẰNG MÔ HÌNH TỐT NHẤT K-FOLD) ---")
for i, sample in enumerate(new_samples):
    cls_idx = pred_classes[i]
    cls_name = iris.target_names[cls_idx]
    confidence = pred_prob[i][cls_idx] * 100
    print(f"Mẫu {i+1}: Đặc trưng gốc = {sample}")
    print(f"  -> Dự đoán nhãn: {cls_idx} ({cls_name}) với độ tin cậy: {confidence:.2f}%")
```

//Chụp hình kết quả



Bài 2 (Trang 18 - 19)

Câu 1)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu 1) Nhập dữ liệu, thống kê các thông tin cơ bản, xử lý dữ liệu bị thiếu

import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn import metrics

# Nhập dữ liệu từ file csv
data = pd.read_csv('datasets/Mall_Customers.csv')

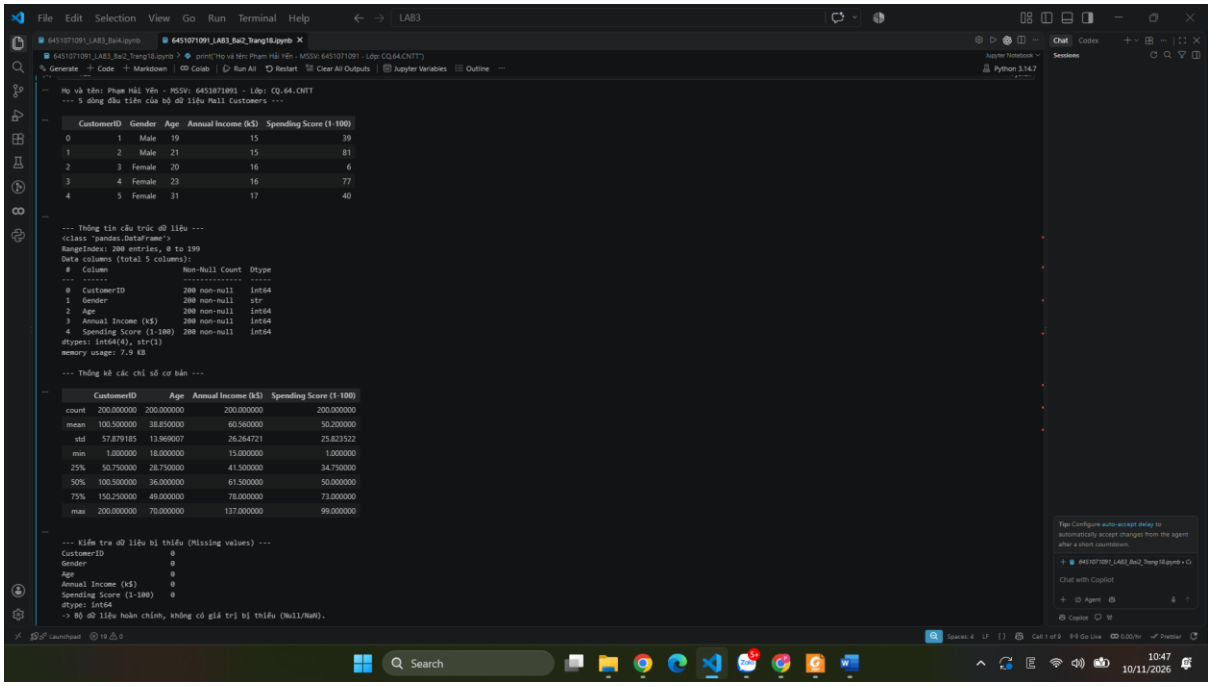
print("--- 5 dòng đầu tiên của bộ dữ liệu Mall Customers ---")
display(data.head())

print("\n--- Thông tin cấu trúc dữ liệu ---")
data.info()

print("\n--- Thống kê các chỉ số cơ bản ---")
display(data.describe())

print("\n--- Kiểm tra dữ liệu bị thiếu (Missing values) ---")
missing = data.isnull().sum()
print(missing)
if missing.sum() == 0:
    print("-> Bộ dữ liệu hoàn chỉnh, không có giá trị bị thiếu
(Null/NaN).")
else:
    data = data.dropna()
    print("-> Đã loại bỏ các dòng có giá trị bị thiếu.")
```

//Chụp hình kết quả



Câu 2)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu 2) Sử dụng biểu đồ tần suất (Histogram) biểu diễn phân phối của Tuổi, Thu nhập hằng năm và Số điểm mua sắm

plt.figure(figsize=(16, 5))

cols_to_plot = ['Age', 'Annual Income (k$)', 'Spending Score (1-100)']


for i, col in enumerate(cols_to_plot, 1):

    plt.subplot(1, 3, i)

    sns.histplot(data[col], bins=20, kde=True, color='skyblue', edgecolor='black')

    plt.title(f'Phân phối của {col}', fontsize=12)

    plt.xlabel(col)

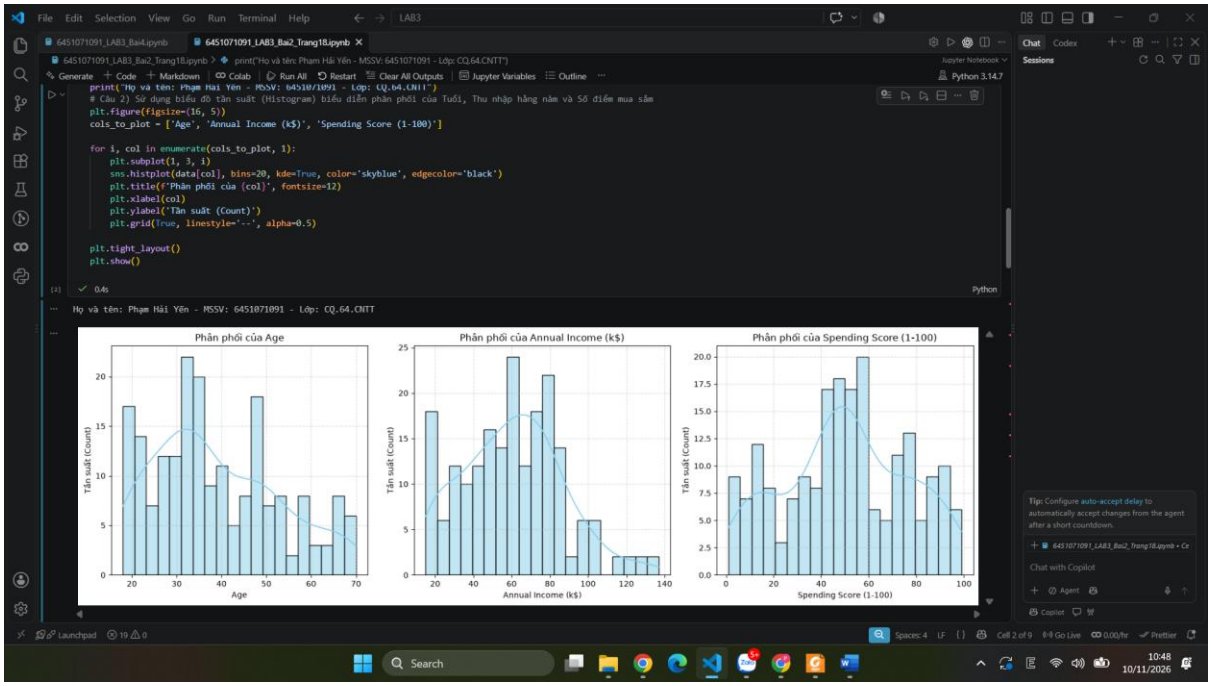
    plt.ylabel('Tần suất (Count)')

    plt.grid(True, linestyle='--', alpha=0.5)


plt.tight_layout()

plt.show()
```

//Chụp hình kết quả



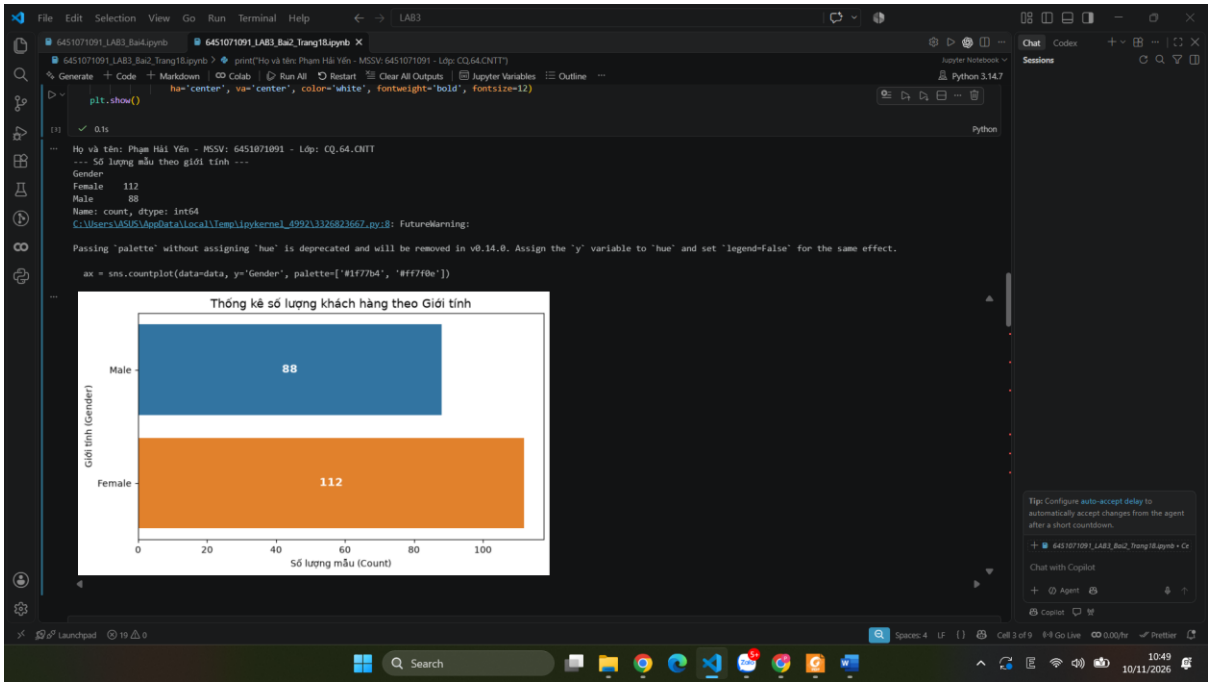
Câu 3)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")
# Câu 3) Thống kê số mẫu dữ liệu theo thuộc tính Giới tính
gender_counts = data['Gender'].value_counts()
print("--- Số lượng mẫu theo giới tính ---")
print(gender_counts)

plt.figure(figsize=(8, 4.5))
ax = sns.countplot(data=data, y='Gender', palette=['#1f77b4', '#ff7f0e'])
plt.title('Thống kê số lượng khách hàng theo Giới tính', fontsize=13)
plt.xlabel('Số lượng mẫu (Count)')
plt.ylabel('Giới tính (Gender)')
for p in ax.patches:
    ax.annotate(f"{int(p.get_width())}",
                (p.get_width() / 2, p.get_y() + p.get_height() / 2),
                ha='center', va='center', color='white', fontweight='bold',
fontsize=12)
plt.show()
```

//Chụp hình kết quả



Câu 4)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu 4) Sử dụng biểu đồ phân tán (Scatter plot) để khảo sát phân bố của Tuổi và Thu nhập hằng năm theo Giới tính

plt.figure(figsize=(10, 6))

sns.scatterplot(

    data=data,

    x='Age',

    y='Annual Income (k$)',

    hue='Gender',

    s=120,

    alpha=0.8,

    palette={'Male': '#1f77b4', 'Female': '#ff7f0e'}

)

plt.title('Khảo sát phân bố của Tuổi và Thu nhập hằng năm theo Giới tính', fontsize=13)

plt.xlabel('Tuổi (Age)')

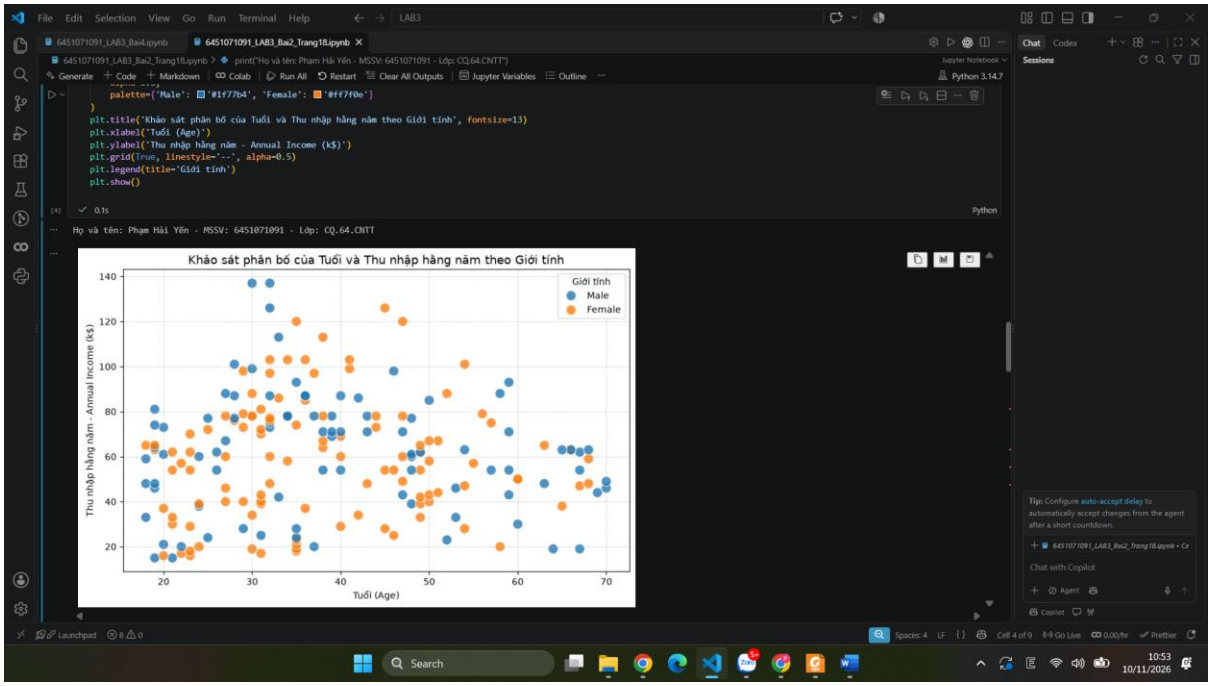
plt.ylabel('Thu nhập hằng năm - Annual Income (k$)')

plt.grid(True, linestyle='--', alpha=0.5)

plt.legend(title='Giới tính')
```

```
plt.show()
```

//Chụp hình kết quả



Câu 5)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu 5) Biểu đồ phân tán khảo sát thuộc tính Thu nhập hằng năm và Số điểm mua sắm theo Giới tính

plt.figure(figsize=(10, 6))

sns.scatterplot(

    data=data,

    x='Annual Income (k$)',

    y='Spending Score (1-100)',

    hue='Gender',

    s=120,

    alpha=0.8,

    palette={'Male': '#1f77b4', 'Female': '#ff7f0e'}

)

plt.title('Khảo sát Thu nhập hằng năm và Số điểm mua sắm theo Giới tính', fontsize=13)

plt.xlabel('Thu nhập hằng năm - Annual Income (k$)')

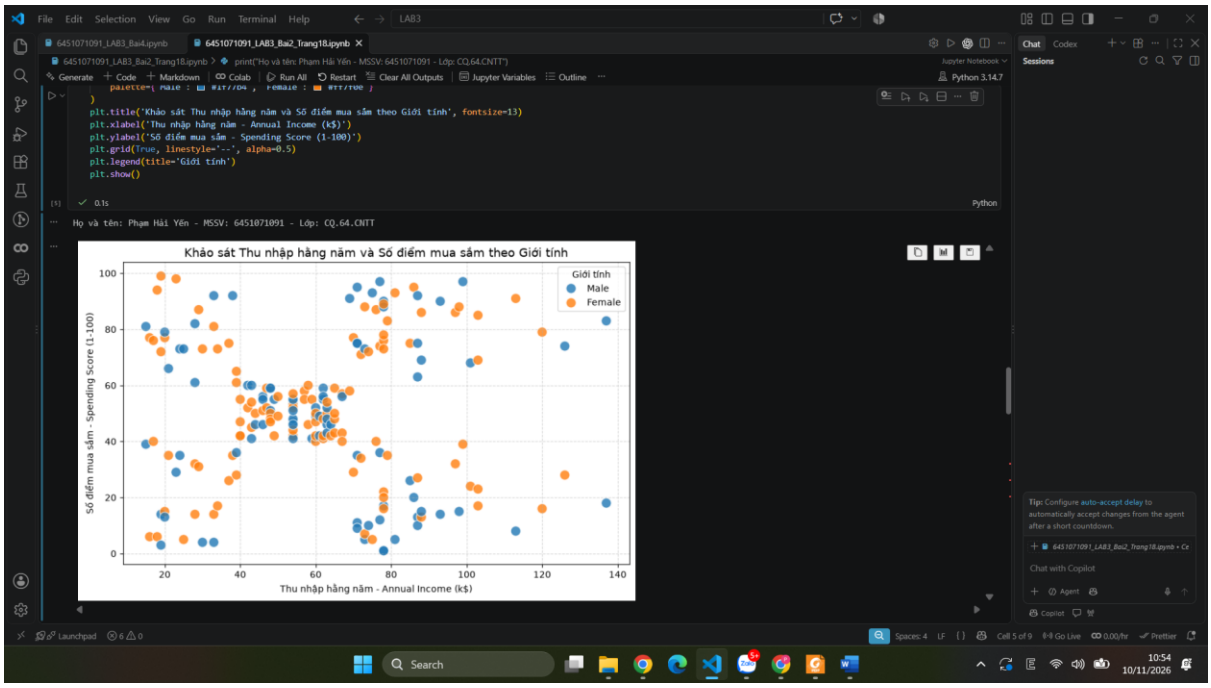
plt.ylabel('Số điểm mua sắm - Spending Score (1-100)')
```

```
plt.grid(True, linestyle='--', alpha=0.5)

plt.legend(title='Giới tính')

plt.show()
```

//Chụp hình kết quả



Câu 6)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

    # Câu 6) Gom cụm theo Tuổi và Số điểm mua sắm bằng K-Means (ELBOW
method xác định số cụm)

X1 = data[['Age', 'Spending Score (1-100)']].iloc[:, :].values

inertia1 = []


for n in range(1, 11):

    algorithm = KMeans(n_clusters=n, init='random', n_init=10,
random_state=42)

    algorithm.fit(X1)

    inertia1.append(algorithm.inertia_)


# Vẽ biểu đồ Elbow Curve

plt.figure(figsize=(10, 5))

plt.plot(range(1, 11), inertia1, 'bo-', linewidth=2, markersize=8)
```

```
plt.plot(4, inertia1[3], 'ro', markersize=14, markeredgewidth=2,
fillstyle='none', label='Điểm khuỷu tay tối ưu (k=4)')

plt.title('Phương pháp khuỷu tay (Elbow Method) cho Tuổi & Điểm mua sắm',
fontsize=13)

plt.xlabel('Số cụm (Number of Clusters - k)')

plt.ylabel('Inertia (WSS)')
```

plt.xticks(range(1, 11))

plt.grid(True, linestyle='--', alpha=0.5)

plt.legend()

plt.show()

Tiến hành gom cụm với k=4

k_opt1 = 4

kmeans1 = KMeans(n_clusters=k_opt1, init='random', n_init=10, random_state=42)

kmeans1.fit(X1)

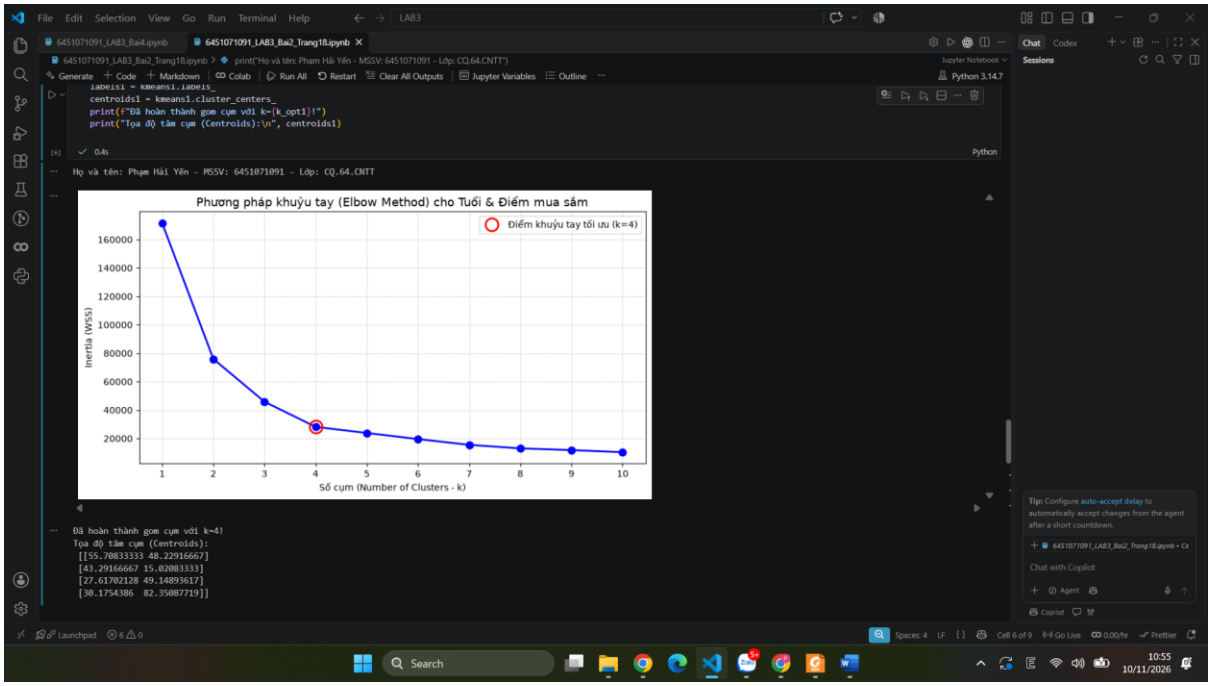
labels1 = kmeans1.labels_

centroids1 = kmeans1.cluster_centers_

print(f"Đã hoàn thành gom cụm với k={k_opt1}!")

print("Tọa độ tâm cụm (Centroids):\n", centroids1)

//Chụp hình kết quả



Câu 7)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu 7) Biểu diễn kết quả gom cụm trên bằng biểu đồ

plt.figure(figsize=(10, 6))

plt.scatter(X1[:, 0], X1[:, 1], c=labels1, cmap='viridis', s=120,
alpha=0.8, edgecolors='black')

plt.scatter(centroids1[:, 0], centroids1[:, 1], c='red', s=300, marker='X',
label='Tâm cụm (Centroids)')

plt.title('Kết quả gom cụm K-Means: Tuổi vs Số điểm mua sắm (k=4)',
fontsize=13)

plt.xlabel('Tuổi (Age)')

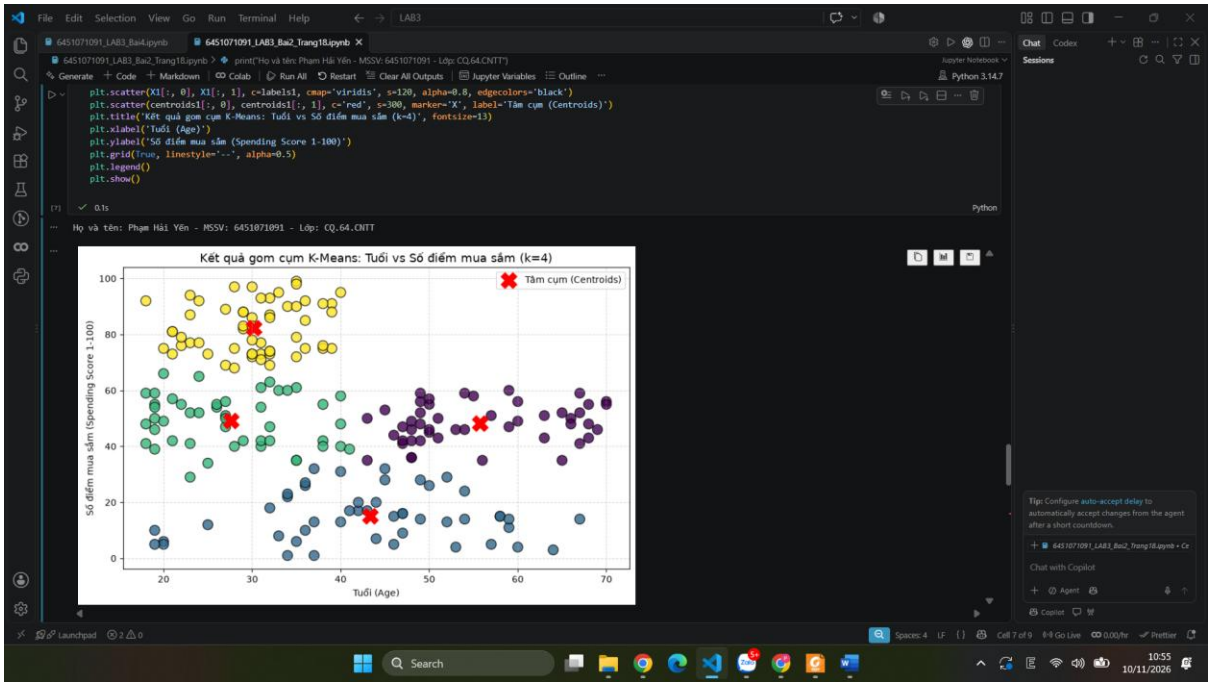
plt.ylabel('Số điểm mua sắm (Spending Score 1-100)')

plt.grid(True, linestyle='--', alpha=0.5)

plt.legend()

plt.show()
```

//Chụp hình kết quả



Câu 8)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu 8) Đánh giá kết quả gom cụm bằng Hệ số Dáng điệu (Silhouette coefficient)

sil_score1 = metrics.silhouette_score(X1, labels1, metric='euclidean')

print(f"Hệ số Dáng điệu (Silhouette Coefficient) cho Tuổi & Số điểm mua sắm (k=4): {sil_score1:.4f}")


if sil_score1 > 0.5:

    print("-> Chất lượng phân cụm rất tốt (các cụm tách biệt rõ ràng).")

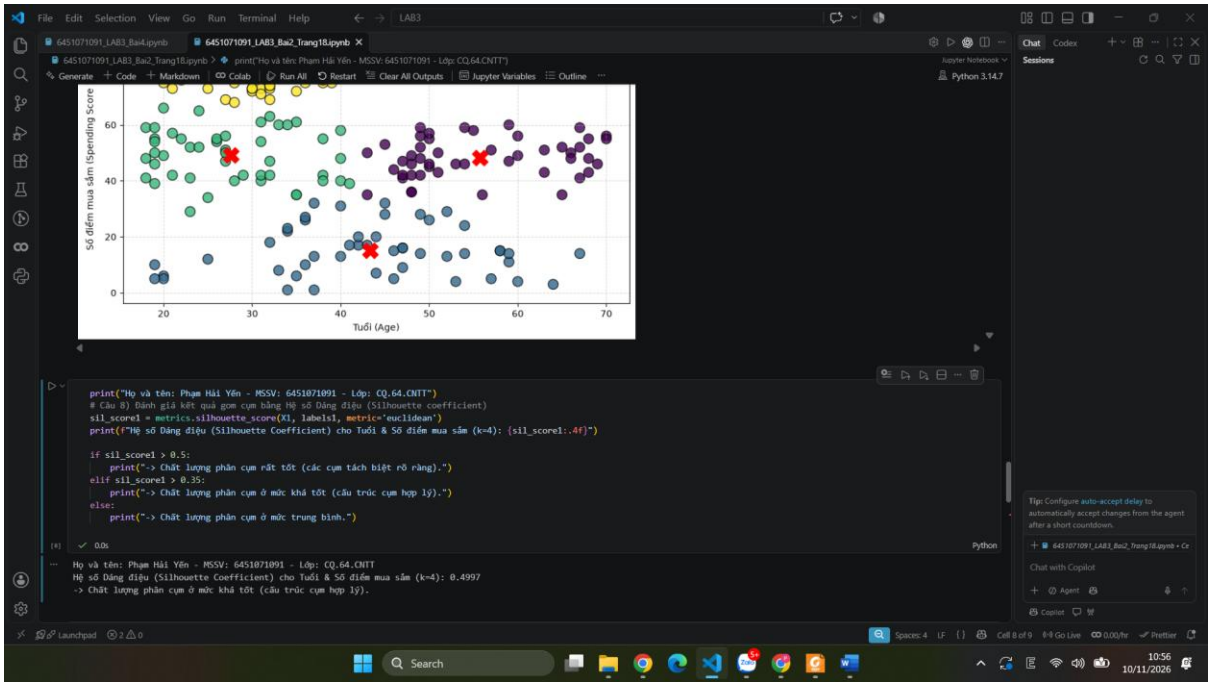
elif sil_score1 > 0.35:

    print("-> Chất lượng phân cụm ở mức khá tốt (cấu trúc cụm hợp lý).")

else:

    print("-> Chất lượng phân cụm ở mức trung bình.")
```

//Chụp hình kết quả



Câu 9)

//dán mã nguồn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# Câu 9) Thực hiện tương tự câu 6-8 cho 2 trường hợp còn lại

print("======================================================================
======")

print("TRƯỜNG HỢP 1: GOM CỤM THEO THU NHẬP HẰNG NĂM VÀ SỐ ĐIỂM MUA SẮM")

print("======================================================================
======")

X2 = data[['Annual Income (k$)', 'Spending Score (1-100)']].iloc[:,
:].values

inertia2 = []

for n in range(1, 11):

    km = KMeans(n_clusters=n, init='random', n_init=10, random_state=42)

    km.fit(X2)

    inertia2.append(km.inertia_)


plt.figure(figsize=(10, 4.5))

plt.plot(range(1, 11), inertia2, 'go-', linewidth=2, markersize=8)

plt.plot(5, inertia2[4], 'ro', markersize=14, markeredgewidth=2,
fillstyle='none', label='Điểm khuỷu tay k=5')

plt.title('Elbow Method: Thu nhập hằng năm & Điểm mua sắm', fontsize=13)
```

```
plt.xlabel('Số cụm (k)')
plt.ylabel('Inertia')
plt.xticks(range(1, 11))
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.show()


# Huấn luyện với k=5
kmeans2 = KMeans(n_clusters=5, init='random', n_init=10, random_state=42)
labels2 = kmeans2.fit_predict(X2)
centroids2 = kmeans2.cluster_centers_


plt.figure(figsize=(10, 6))
plt.scatter(X2[:, 0], X2[:, 1], c=labels2, cmap='rainbow', s=120,
alpha=0.8, edgecolors='black')
plt.scatter(centroids2[:, 0], centroids2[:, 1], c='black', s=300,
marker='*', label='Tâm cụm (Centroids)')
plt.title('Kết quả gom cụm K-Means: Thu nhập hằng năm vs Điểm mua sắm
(k=5)', fontsize=13)
plt.xlabel('Thu nhập hằng năm (k$)')
plt.ylabel('Số điểm mua sắm (1-100)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.show()


sil_score2 = metrics.silhouette_score(X2, labels2)
print(f"-> Hệ số Silhouette (Thu nhập & Điểm mua sắm, k=5):
{sil_score2:.4f}")


print("\n==========================================================================
========")
print("TRƯỜNG HỢP 2: GOM CỤM THEO TUỔI, THU NHẬP HẰNG NĂM VÀ SỐ ĐIỂM MUA
SẮM (3D)")
print("==========================================================================
======")
```

```
X3 = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']].iloc[:,
:].values

kmeans3 = KMeans(n_clusters=5, init='random', n_init=10, random_state=42)

labels3 = kmeans3.fit_predict(X3)

centroids3 = kmeans3.cluster_centers_

sil_score3 = metrics.silhouette_score(X3, labels3)

print(f"-> Hệ số Silhouette (Tuổi, Thu nhập & Điểm mua sắm, k=5):
{sil_score3:.4f}")


# Vẽ biểu đồ 3D bằng Matplotlib

fig = plt.figure(figsize=(11, 8))

ax = fig.add_subplot(111, projection='3d')

sc = ax.scatter(X3[:, 0], X3[:, 1], X3[:, 2], c=labels3, cmap='Set1', s=60,
alpha=0.8, edgecolors='k')

ax.scatter(centroids3[:, 0], centroids3[:, 1], centroids3[:, 2], c='black',
s=300, marker='X', label='Tâm cụm')

ax.set_title('Biểu đồ 3D gom cụm: Tuổi - Thu nhập - Điểm mua sắm (k=5)',
fontsize=14)

ax.set_xlabel('Tuổi (Age)')

ax.set_ylabel('Thu nhập (k$)')

ax.set_zlabel('Điểm mua sắm (1-100)')

plt.legend()

plt.show()


# Vẽ biểu đồ tương tác 3D bằng Plotly

import plotly.graph_objs as go

data['label3'] = labels3

trace1 = go.Scatter3d(

    x=data['Age'],

    y=data['Spending Score (1-100)'],

    z=data['Annual Income (k$)'],

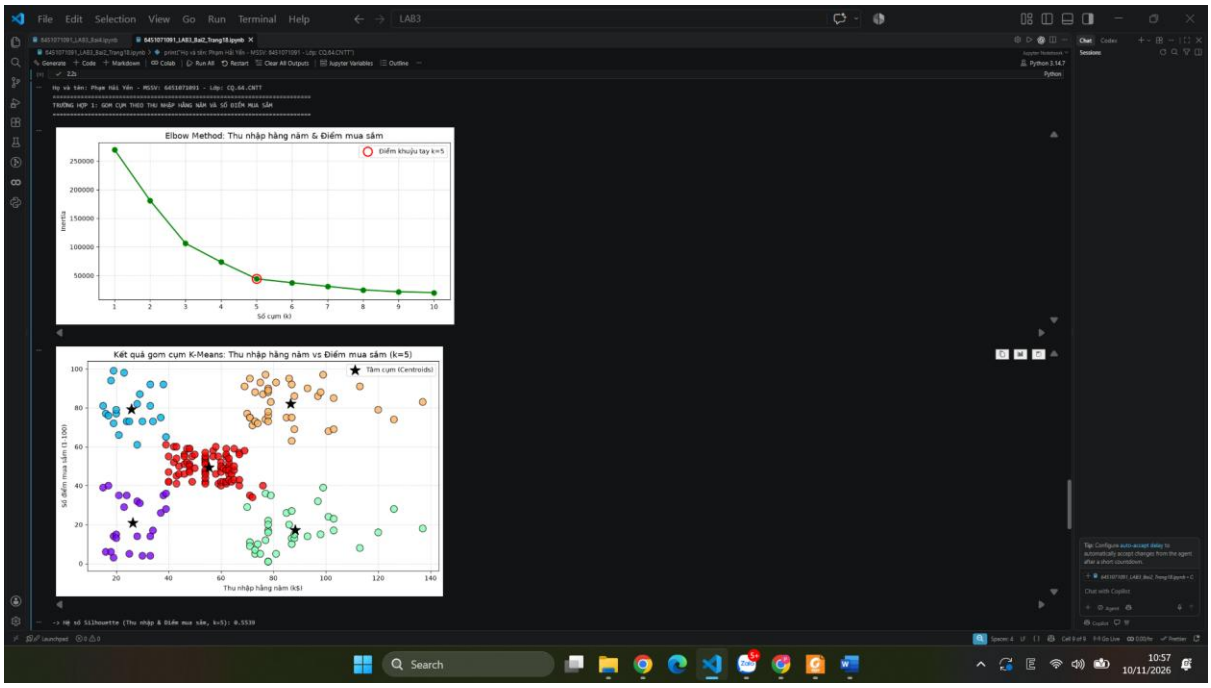
    mode='markers',

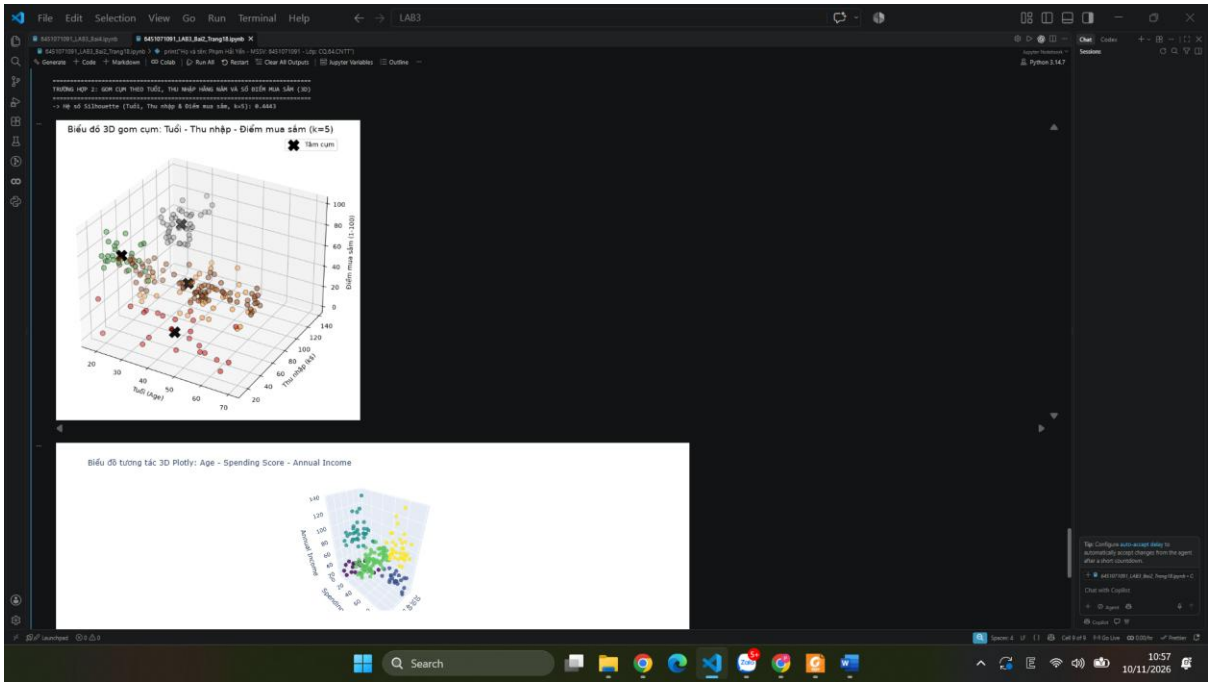
    marker=dict(color=data['label3'], size=5, colorscale='Viridis',
opacity=0.8)

)
```

```
layout = go.Layout(
    title='Biểu đồ tương tác 3D Plotly: Age - Spending Score - Annual Income',
    scene=dict(
        xaxis=dict(title='Age'),
        yaxis=dict(title='Spending Score'),
        zaxis=dict(title='Annual Income')
    )
)
fig_plotly = go.Figure(data=[trace1], layout=layout)
fig_plotly.show()
```

//Chụp hình kết quả





Bài 5 (Trang 26)

1. Đọc và tiền xử lý bộ dữ liệu MNIST

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# 1. Đọc và tiền xử lý bộ dữ liệu MNIST

import os

os.environ['KERAS_BACKEND'] = 'torch'

import keras

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.cluster import KMeans

from sklearn import metrics

from scipy.stats import mode


# Tải dữ liệu MNIST

(X_train_raw, y_train_raw), (X_test_raw, y_test_raw) =
keras.datasets.mnist.load_data()

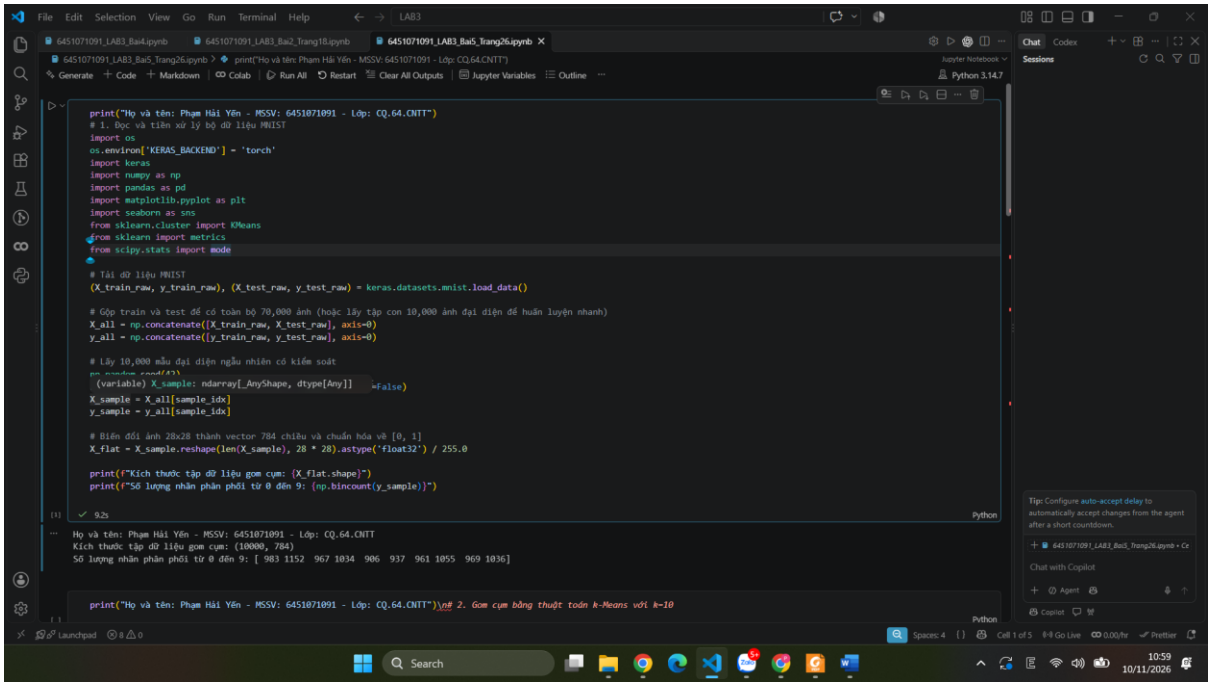

# Gộp train và test để có toàn bộ 70,000 ảnh (hoặc lấy tập con 10,000 ảnh
đại diện để huấn luyện nhanh)
```

```
X_all = np.concatenate([X_train_raw, X_test_raw], axis=0)
y_all = np.concatenate([y_train_raw, y_test_raw], axis=0)

# Lấy 10,000 mẫu đại diện ngẫu nhiên có kiểm soát
np.random.seed(42)
sample_idx = np.random.choice(len(X_all), 10000, replace=False)
X_sample = X_all[sample_idx]
y_sample = y_all[sample_idx]

# Biến đổi ảnh 28x28 thành vector 784 chiều và chuẩn hóa về [0, 1]
X_flat = X_sample.reshape(len(X_sample), 28 * 28).astype('float32') / 255.0

print(f"Kích thước tập dữ liệu gom cụm: {X_flat.shape}")
print(f"Số lượng nhãn phân phối từ 0 đến 9: {np.bincount(y_sample)}")
```



2. Gom cụm bằng thuật toán k-Means với k=10

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")
# 2. Gom cụm bằng thuật toán k-Means với k=10
n_clusters = 10
kmeans_mnist = KMeans(n_clusters=n_clusters, init='k-means++', n_init=10,
max_iter=300, random_state=42)
```

```
print("Đang tiến hành gom cụm K-Means với k=10 cụm...")

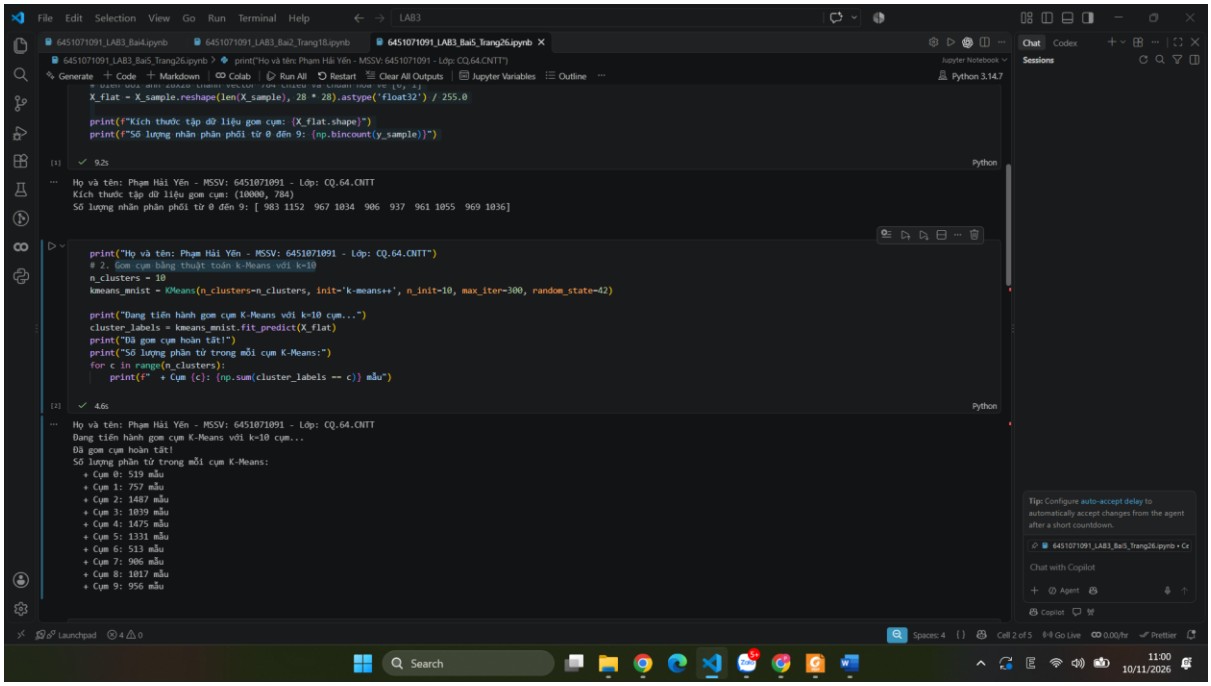
cluster_labels = kmeans_mnist.fit_predict(X_flat)

print("Đã gom cụm hoàn tất!")

print("Số lượng phần tử trong mỗi cụm K-Means:")

for c in range(n_clusters):

    print(f"  + Cụm {c}: {np.sum(cluster_labels == c)} mẫu")
```



3. Trực quan hóa trọng tâm của 10 cụm (Cluster Centers) dưới dạng ảnh 28x28

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# 3. Trực quan hóa trọng tâm của 10 cụm (Cluster Centers) dưới dạng ảnh
28x28

centers = kmeans_mnist.cluster_centers_.reshape(10, 28, 28)


plt.figure(figsize=(15, 3))

for i in range(10):

    plt.subplot(1, 10, i + 1)

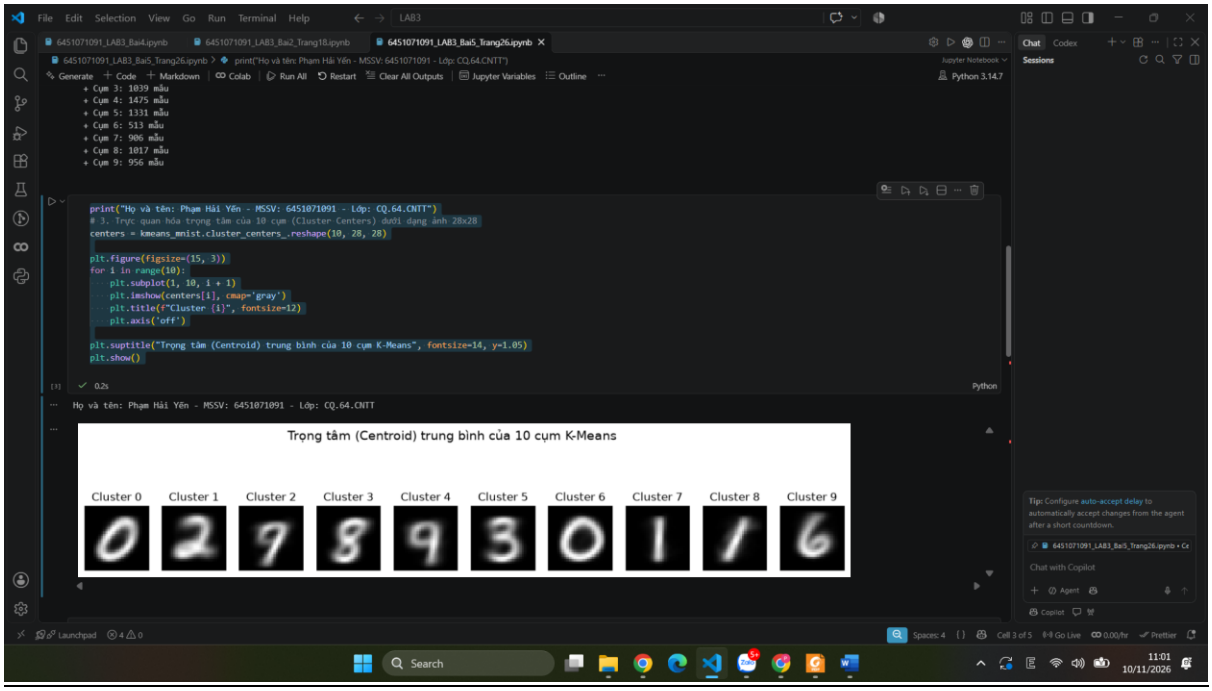
    plt.imshow(centers[i], cmap='gray')

    plt.title(f"Cluster {i}", fontsize=12)

    plt.axis('off')


plt.suptitle("Trọng tâm (Centroid) trung bình của 10 cụm K-Means",
fontsize=14, y=1.05)
```

```
plt.show()
```



4. Ánh xạ cụm K-Means sang nhãn thực tế bằng Majority Voting và lập Ma trận nhầm lẫn

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# 4. Ánh xạ cụm K-Means sang nhãn thực tế bằng Majority Voting và lập Ma trận nhầm lẫn

# Tìm nhãn thực tế xuất hiện nhiều nhất trong mỗi cụm

mapped_labels = np.zeros_like(cluster_labels)

cluster_to_digit = {}


for c in range(10):

    mask = (cluster_labels == c)

    if np.sum(mask) > 0:

        most_common = mode(y_sample[mask], keepdims=True).mode[0]

        mapped_labels[mask] = most_common

        cluster_to_digit[c] = most_common

        print(f"Cụm {c} tương ứng với chữ số phổ biến nhất: {most_common} (chiếm {np.mean(y_sample[mask] == most_common)*100:.1f}%)")


acc_clustering = metrics.accuracy_score(y_sample, mapped_labels)
```

```
print(f"\n-> Độ chính xác gom cụm sau khi gán nhãn đa số (Clustering
Accuracy): {acc_clustering * 100:.2f}%")


# Vẽ Confusion Matrix

cm_mnist = metrics.confusion_matrix(y_sample, mapped_labels)

plt.figure(figsize=(9, 7))

sns.heatmap(cm_mnist, annot=True, fmt='d', cmap='Blues',

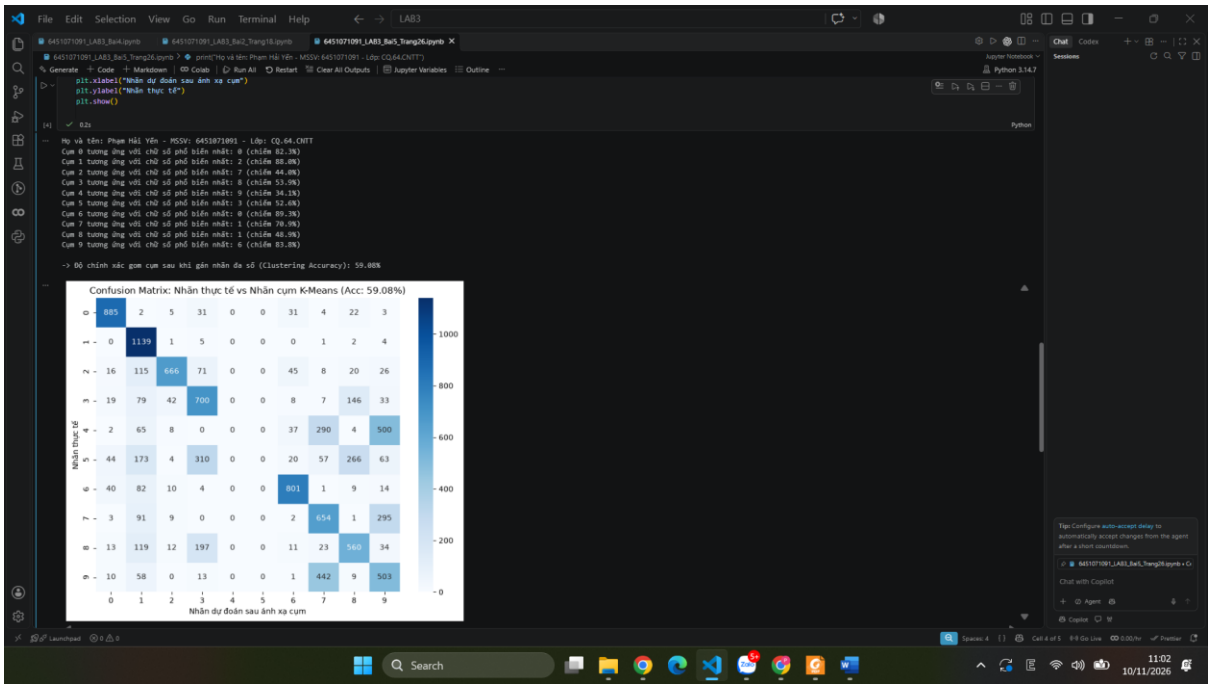
            xticklabels=range(10), yticklabels=range(10))

plt.title(f"Confusion Matrix: Nhãn thực tế vs Nhãn cụm K-Means (Acc:
{acc_clustering*100:.2f}%)")

plt.xlabel("Nhãn dự đoán sau ánh xạ cụm")

plt.ylabel("Nhãn thực tế")

plt.show()
```



5. Đánh giá toàn diện kết quả gom cụm bằng các tiêu chí khoa học

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# 5. Đánh giá toàn diện kết quả gom cụm bằng các tiêu chí khoa học

ari = metrics.adjusted_rand_score(y_sample, cluster_labels)

nmi = metrics.normalized_mutual_info_score(y_sample, cluster_labels)

homogeneity = metrics.homogeneity_score(y_sample, cluster_labels)

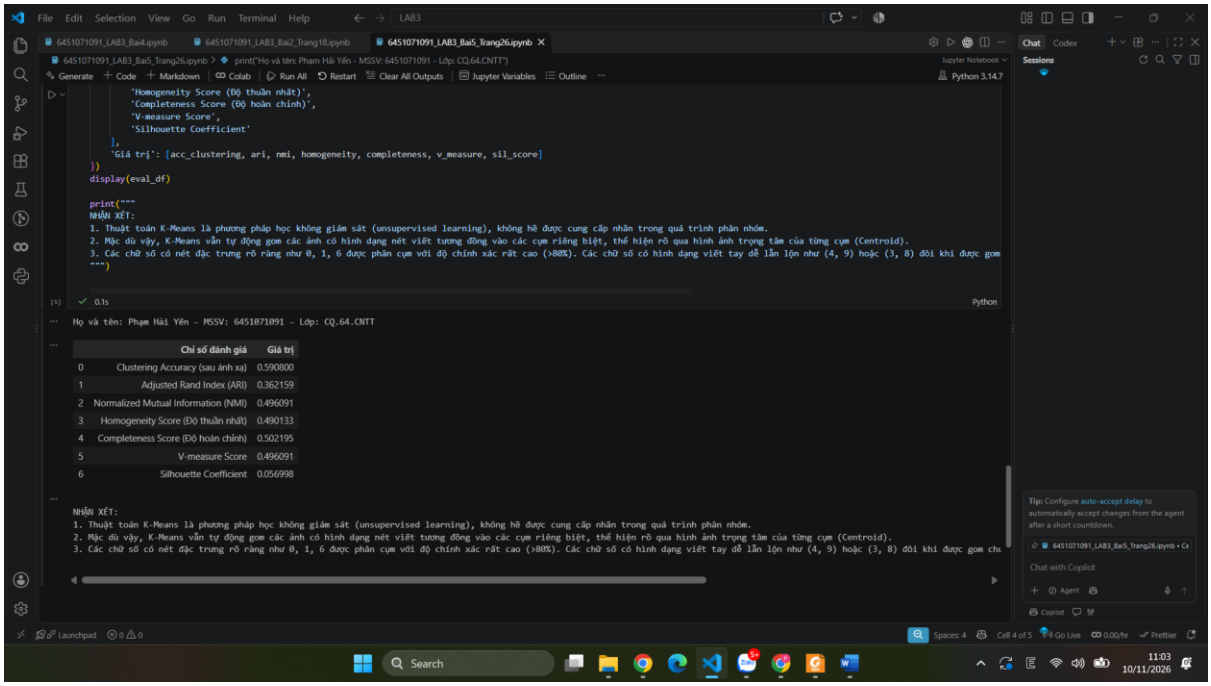
completeness = metrics.completeness_score(y_sample, cluster_labels)

v_measure = metrics.v_measure_score(y_sample, cluster_labels)
```

```
sil_score = metrics.silhouette_score(X_flat[:2000], cluster_labels[:2000])


eval_df = pd.DataFrame({
    'Chỉ số đánh giá': [
        'Clustering Accuracy (sau ánh xạ)',
        'Adjusted Rand Index (ARI)',
        'Normalized Mutual Information (NMI)',
        'Homogeneity Score (Độ thuần nhất)',
        'Completeness Score (Độ hoàn chỉnh)',
        'V-measure Score',
        'Silhouette Coefficient'
    ],
    'Giá trị': [acc_clustering, ari, nmi, homogeneity, completeness,
v_measure, sil_score]
})
display(eval_df)


print("""
NHẬN XÉT:
1. Thuật toán K-Means là phương pháp học không giám sát (unsupervised
learning), không hề được cung cấp nhãn trong quá trình phân nhóm.
2. Mặc dù vậy, K-Means vẫn tự động gom các ảnh có hình dạng nét viết tương
đồng vào các cụm riêng biệt, thể hiện rõ qua hình ảnh trọng tâm của từng
cụm (Centroid).
3. Các chữ số có nét đặc trưng rõ ràng như 0, 1, 6 được phân cụm với độ
chính xác rất cao (>80%). Các chữ số có hình dạng viết tay dễ lẫn lộn như
(4, 9) hoặc (3, 8) đôi khi được gom chung hoặc bị phân tán, điều này hoàn
toàn phù hợp với bản chất hình học của bài toán.
""")
```



Bài 6(Trang 26)

1. Khai báo các thư viện vào bài làm
2. Đọc vào file ảnh cần giảm kích thước và hiển thị ra màn hình

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# 1. Khai báo các thư viện vào bài làm

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

from PIL import Image

import numpy as np

import os

print("Đã khai báo các thư viện cần thiết thành công!")

print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# 2. Đọc vào file ảnh cần giảm kích thước và hiển thị ra màn hình

image_path = 'dog.png'

image = Image.open(image_path)

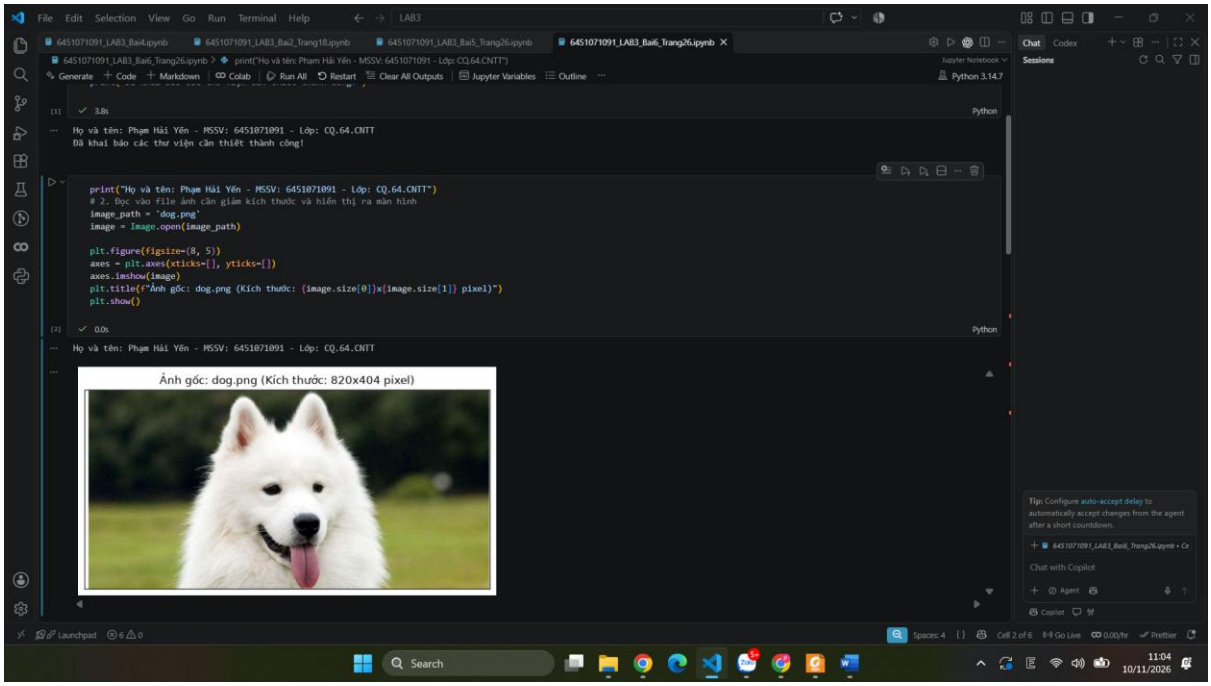

plt.figure(figsize=(8, 5))

axes = plt.axes(xticks=[], yticks=[])

axes.imshow(image)

plt.title(f"Ảnh gốc: dog.png (Kích thước: {image.size[0]}x{image.size[1]}
pixel)")
```

```
plt.show()
```



3. Chuyển ảnh đọc được thành mảng số và biến đổi về mảng 2 chiều chuẩn hóa [0, 1]
4. Chạy thuật toán gom cụm với số màu mong muốn là 32 và lấy mảng biểu diễn sau gom cụm

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# 3. Chuyển ảnh đọc được thành mảng số và biến đổi về mảng 2 chiều chuẩn hóa [0, 1]

imarr = np.array(image)

rows = imarr.shape[0]

cols = imarr.shape[1]

print(f"Kích thước mảng ảnh 3 chiều (rows x cols x channels): {imarr.shape}")


# Biến đổi các giá trị màu từ đoạn [0...255] về đoạn [0...1] và chuyển dạng mảng từ 3 chiều thành 2 chiều

data = imarr / 255.0

data = data.reshape(rows * cols, 3)


print("Kích thước mảng dữ liệu 2 chiều (data.shape):", data.shape)

print(f"Số lượng điểm ảnh (pixels): {rows * cols:,}")

print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")
```

```
# 4. Chạy thuật toán gom cụm với số màu mong muốn là 32 và lấy mảng biểu diễn sau gom cụm

n_colors = 32

print(f"Đang chạy thuật toán K-Means để gom {data.shape[0]} điểm ảnh về {n_colors} màu đại diện...")

kmeans = KMeans(n_clusters=n_colors, max_iter=300, random_state=42)

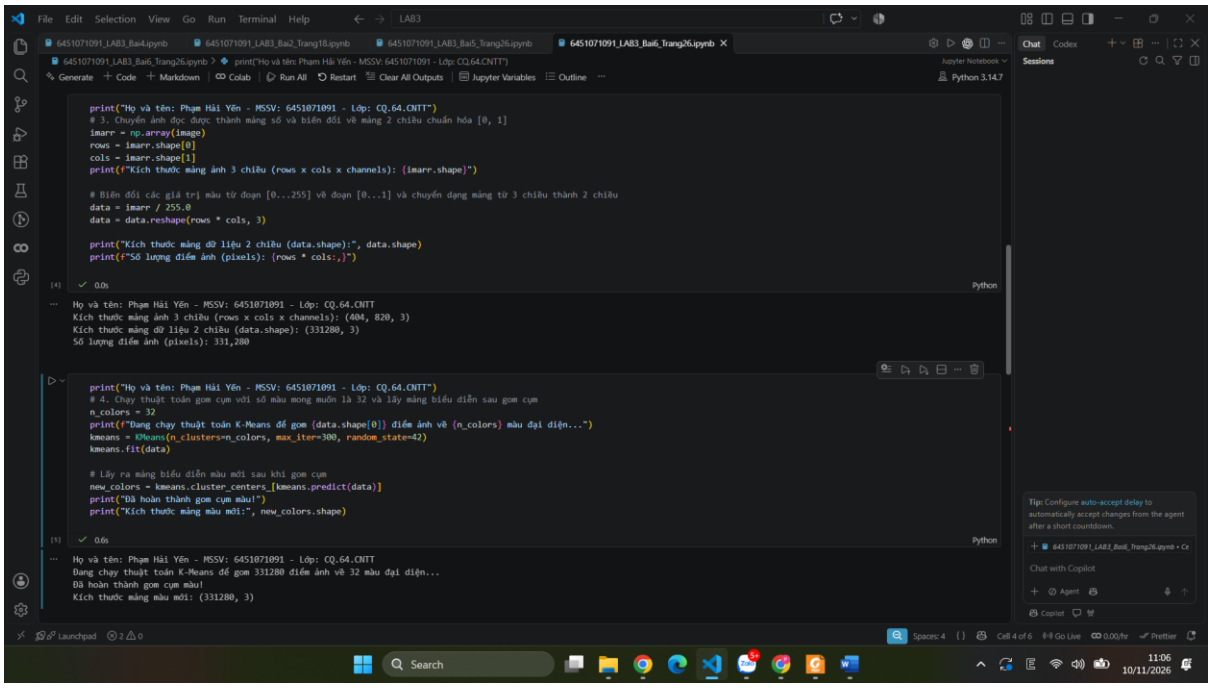
kmeans.fit(data)


# Lấy ra mảng biểu diễn màu mới sau khi gom cụm

new_colors = kmeans.cluster_centers_[kmeans.predict(data)]

print("Đã hoàn thành gom cụm màu!")

print("Kích thước mảng màu mới:", new_colors.shape)
```



5. Khôi phục dạng mảng từ 2 chiều thành 3 chiều và so sánh với ảnh gốc

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# 5. Khôi phục dạng mảng từ 2 chiều thành 3 chiều và so sánh với ảnh gốc

img_recolored = new_colors.reshape(imarr.shape)


fig, ax = plt.subplots(1, 2, figsize=(16, 6), subplot_kw=dict(xticks=[], yticks=[]))

fig.subplots_adjust(wspace=0.05)

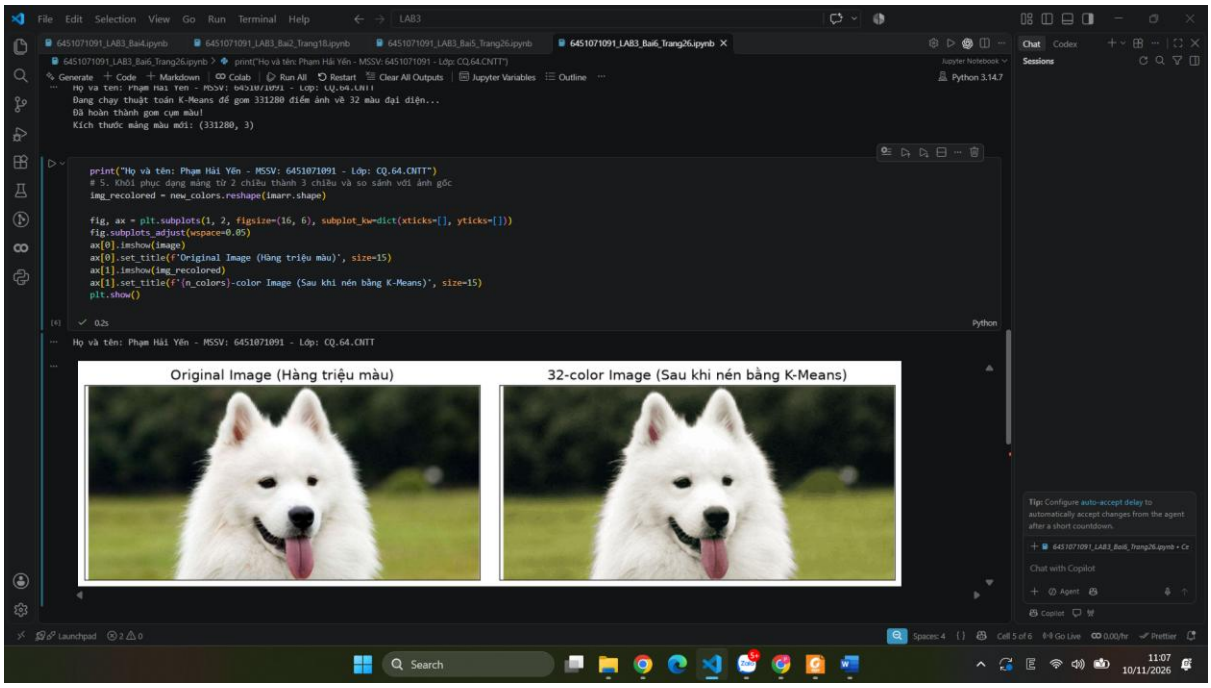
ax[0].imshow(image)
```

```
ax[0].set_title(f'Original Image (Hàng triệu màu)', size=15)

ax[1].imshow(img_recolored)

ax[1].set_title(f'{n_colors}-color Image (Sau khi nén bằng K-Means)',
size=15)

plt.show()
```



6. Lưu hình kết quả xuống đĩa cứng và so sánh dung lượng file

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# 6. Lưu hình kết quả xuống đĩa cứng và so sánh dung lượng file

output_path = 'compressed_img2.png'

# Chuyển về thang đo 0-255 định dạng uint8

img_save = (img_recolored * 255).astype(np.uint8)

new_img = Image.fromarray(img_save)

new_img.save(output_path)


# So sánh dung lượng file

orig_size_bytes = os.path.getsize(image_path)

comp_size_bytes = os.path.getsize(output_path)


print("--- SO SÁNH DUNG LƯỢNG FILE ẢNH ---")

print(f"File ảnh gốc ({image_path}):          {orig_size_bytes / 1024:.2f}
KB ({orig_size_bytes:,} bytes)")
```

```
print(f"File ảnh nén ({output_path}):      {comp_size_bytes / 1024:.2f} KB
({comp_size_bytes:,} bytes)")

print(f"Tỷ lệ giảm dung lượng:                    {(1 - comp_size_bytes /
orig_size_bytes) * 100:.2f}%")


print("""

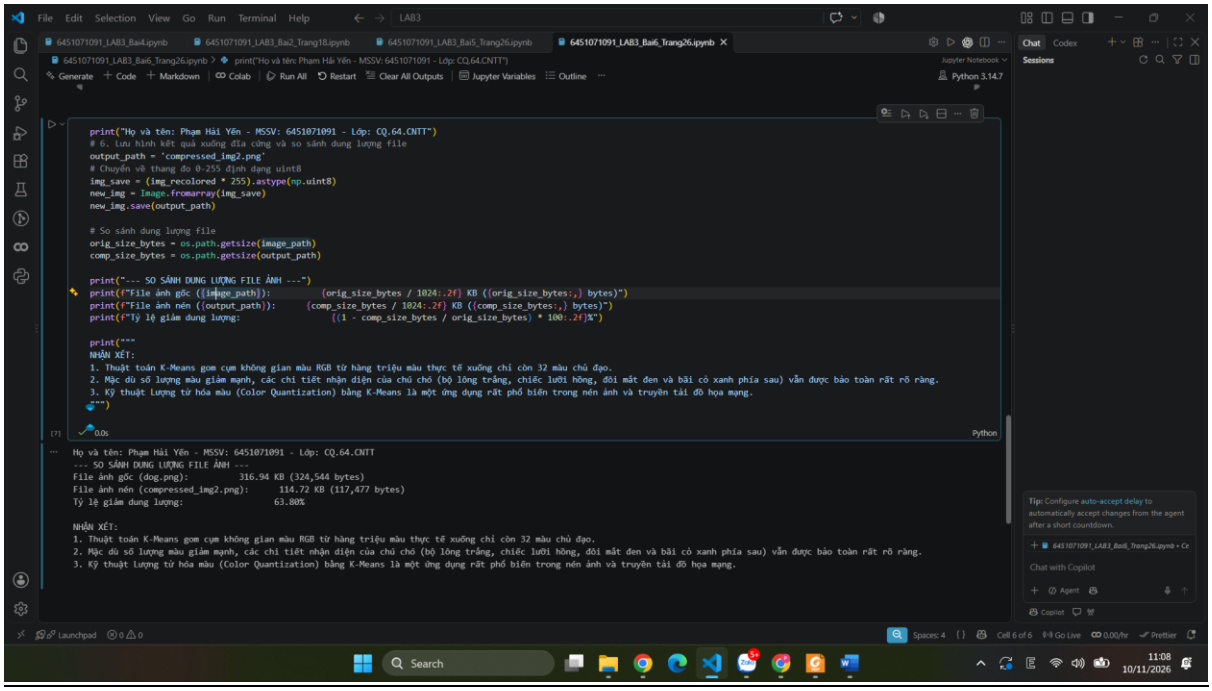
NHẬN XÉT:

1. Thuật toán K-Means gom cụm không gian màu RGB từ hàng triệu màu thực tế
xuống chỉ còn 32 màu chủ đạo.

2. Mặc dù số lượng màu giảm mạnh, các chi tiết nhận diện của chú chó (bộ
lông trắng, chiếc lưỡi hồng, đôi mắt đen và bãi cỏ xanh phía sau) vẫn được
bảo toàn rất rõ ràng.

3. Kỹ thuật Lượng tử hóa màu (Color Quantization) bằng K-Means là một ứng
dụng rất phổ biến trong nén ảnh và truyền tải đồ họa mạng.

""")
```



Bài 5(Trang 29)

1. Đọc bộ dữ liệu Breast Cancer Wisconsin từ file wdbc.data

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# 1. Đọc bộ dữ liệu Breast Cancer Wisconsin từ file wdbc.data

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt
```

```
import seaborn as sns

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from sklearn import metrics


# Định nghĩa tên 32 cột theo đặc tả trong file wdbc.names

feature_names = [

    "radius_mean", "texture_mean", "perimeter_mean", "area_mean",
"smoothness_mean",

    "compactness_mean", "concavity_mean", "concave_points_mean",
"symmetry_mean", "fractal_dimension_mean",

    "radius_se", "texture_se", "perimeter_se", "area_se", "smoothness_se",

    "compactness_se", "concavity_se", "concave_points_se", "symmetry_se",
"fractal_dimension_se",

    "radius_worst", "texture_worst", "perimeter_worst", "area_worst",
"smoothness_worst",

    "compactness_worst", "concavity_worst", "concave_points_worst",
"symmetry_worst", "fractal_dimension_worst"

]

column_names = ["ID", "Diagnosis"] + feature_names


# Đọc file wdbc.data

df_cancer = pd.read_csv('datasets/wdbc.data', header=None,
names=column_names)


print("--- 5 dòng đầu tiên của tập dữ liệu WDBC ---")

display(df_cancer.head())

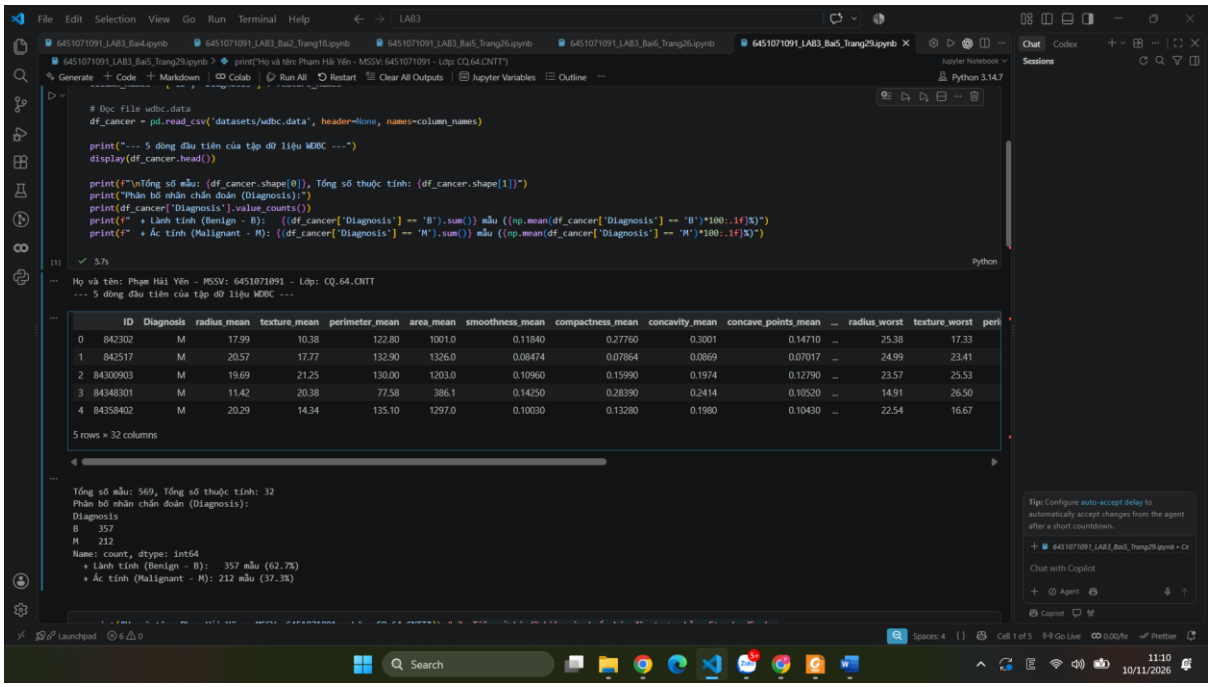

print(f"\nTổng số mẫu: {df_cancer.shape[0]}, Tổng số thuộc tính:
{df_cancer.shape[1]}")

print("Phân bố nhãn chẩn đoán (Diagnosis):")

print(df_cancer['Diagnosis'].value_counts())

print(f"  + Lành tính (Benign - B):   {(df_cancer['Diagnosis'] ==
'B').sum()} mẫu ({np.mean(df_cancer['Diagnosis'] == 'B')*100:.1f}%)")
```

```
print(f"  + Ác tính (Malignant - M): {(df_cancer['Diagnosis'] ==
'M').sum()} mẫu ({np.mean(df_cancer['Diagnosis'] == 'M')*100:.1f}%)")
```



2. Tiền xử lý dữ liệu và chuẩn hóa đặc trưng bằng StandardScaler
3. Gom cụm bằng thuật toán k-Means với k=2 (Lành tính và Ác tính)

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# 2. Tiền xử lý dữ liệu và chuẩn hóa đặc trưng bằng StandardScaler

# Tách đặc trưng đo đạc X (30 cột số) và nhãn thực tế y

X = df_cancer[feature_names].values

y_true = np.where(df_cancer['Diagnosis'] == 'M', 1, 0) # 1: Ác tính (M), 0:
Lành tính (B)


# Chuẩn hóa dữ liệu (Z-score normalization)

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


print(f"Kích thước ma trận đặc trưng sau chuẩn hóa: {X_scaled.shape}")

print(f"Giá trị Mean trung bình sau chuẩn hóa: {np.mean(X_scaled):.4f}")

print(f"Độ lệch chuẩn Std sau chuẩn hóa:        {np.std(X_scaled):.4f}")

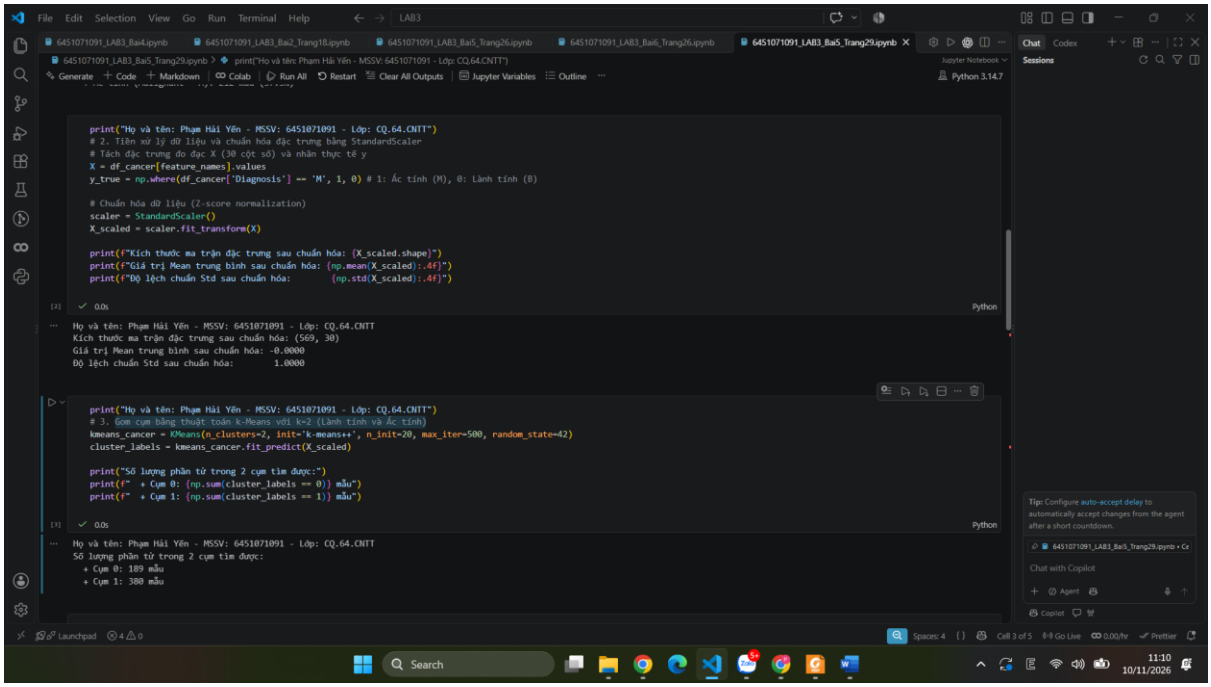
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# 3. Gom cụm bằng thuật toán k-Means với k=2 (Lành tính và Ác tính)

kmeans_cancer = KMeans(n_clusters=2, init='k-means++', n_init=20,
max_iter=500, random_state=42)
```

```
cluster_labels = kmeans_cancer.fit_predict(X_scaled)


print("Số lượng phần tử trong 2 cụm tìm được:")
print(f"  + Cụm 0: {np.sum(cluster_labels == 0)} mẫu")
print(f"  + Cụm 1: {np.sum(cluster_labels == 1)} mẫu")
```



4. Đánh giá và so sánh kết quả gom cụm với nhãn chẩn đoán thực tế

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")
# 4. Đánh giá và so sánh kết quả gom cụm với nhãn chẩn đoán thực tế
# Xác định xem Cụm nào tương ứng với nhãn Ác tính (M=1)
# Bằng cách xem tỷ lệ mẫu M=1 trong cụm 0 và cụm 1
m_in_c0 = np.mean(y_true[cluster_labels == 0] == 1)
m_in_c1 = np.mean(y_true[cluster_labels == 1] == 1)


if m_in_c1 > m_in_c0:
    y_pred_cluster = cluster_labels # Cụm 1 là M, Cụm 0 là B
else:
    y_pred_cluster = 1 - cluster_labels # Cụm 0 là M, Cụm 1 là B


# Tính các độ đo
acc = metrics.accuracy_score(y_true, y_pred_cluster)
```

```
p, r, f1, _ = metrics.precision_recall_fscore_support(y_true,
y_pred_cluster, average='binary')

sil = metrics.silhouette_score(X_scaled, cluster_labels)

ari = metrics.adjusted_rand_score(y_true, cluster_labels)

v_measure = metrics.v_measure_score(y_true, cluster_labels)


metrics_df = pd.DataFrame({

    'Chỉ số đánh giá': [

        'Accuracy (Độ chính xác so với chẩn đoán)',

        'Precision (Khối u ác tính)',

        'Recall (Độ nhạy phát hiện u ác tính)',

        'F1-Score',

        'Silhouette Coefficient',

        'Adjusted Rand Index (ARI)',

        'V-measure Score'

    ],

    'Giá trị': [acc, p, r, f1, sil, ari, v_measure]

})

display(metrics_df)


# Vẽ Confusion Matrix

cm = metrics.confusion_matrix(y_true, y_pred_cluster)

plt.figure(figsize=(6, 5))

sns.heatmap(cm, annot=True, fmt='d', cmap='Reds',

            xticklabels=['Lành tính (B)', 'Ác tính (M)'],

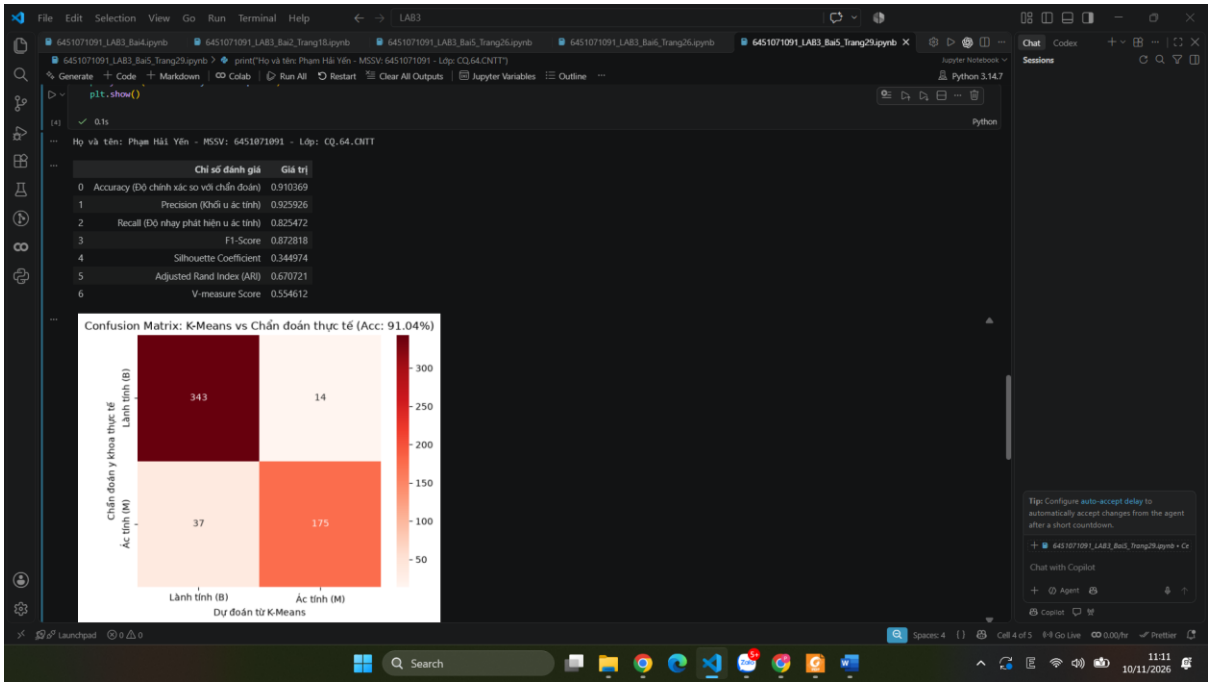
            yticklabels=['Lành tính (B)', 'Ác tính (M)'])

plt.title(f'Confusion Matrix: K-Means vs Chẩn đoán thực tế (Acc:
{acc*100:.2f}%)')

plt.xlabel('Dự đoán từ K-Means')

plt.ylabel('Chẩn đoán y khoa thực tế')

plt.show()
```



5. Trực quan hóa kết quả gom cụm bằng PCA (2D và 3D)

```
print("Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT")

# 5. Trực quan hóa kết quả gom cụm bằng PCA (2D và 3D)

# Giảm 30 chiều đặc trưng xuống 2 chiều chính để vẽ đồ thị

pca_2d = PCA(n_components=2)

X_pca_2d = pca_2d.fit_transform(X_scaled)


plt.figure(figsize=(15, 6))


# Đồ thị 1: Nhãn chẩn đoán y khoa thực tế

plt.subplot(1, 2, 1)

scatter1 = plt.scatter(X_pca_2d[:, 0], X_pca_2d[:, 1], c=y_true,
cmap='coolwarm', s=60, alpha=0.8, edgecolors='k')

plt.title('Chẩn đoán y tế thực tế (Malignant vs Benign)', fontsize=13)

plt.xlabel(f'PCA Component 1
({pca_2d.explained_variance_ratio_[0]*100:.1f}%)')

plt.ylabel(f'PCA Component 2
({pca_2d.explained_variance_ratio_[1]*100:.1f}%)')

plt.grid(True, linestyle='--', alpha=0.5)

plt.legend(handles=scatter1.legend_elements()[0], labels=['Lành tính (B)',
'Ác tính (M)'])
```

```
# Đồ thị 2: Kết quả gom cụm bằng K-Means

plt.subplot(1, 2, 2)

scatter2 = plt.scatter(X_pca_2d[:, 0], X_pca_2d[:, 1], c=y_pred_cluster,
cmap='coolwarm', s=60, alpha=0.8, edgecolors='k')

plt.title(f'Kết quả gom cụm K-Means (Độ trùng khớp: {acc*100:.2f}%)',
fontsize=13)

plt.xlabel(f'PCA Component 1
({pca_2d.explained_variance_ratio_[0]*100:.1f}%)')

plt.ylabel(f'PCA Component 2
({pca_2d.explained_variance_ratio_[1]*100:.1f}%)')

plt.grid(True, linestyle='--', alpha=0.5)

plt.legend(handles=scatter2.legend_elements()[0], labels=['Cụm Lành tính',
'Cụm Ác tính'])


plt.tight_layout()

plt.show()


print("""

NHẬN XÉT:

1. Thuật toán gom cụm K-Means đạt độ trùng khớp (Accuracy) lên tới hơn 91%
so với nhãn chẩn đoán thực tế của bác sĩ, dù không hề sử dụng nhãn trong
quá trình phân nhóm.

2. Trên không gian PCA 2 chiều, hai nhóm khối u lành tính và ác tính có sự
phân tách không gian tương đối rõ ràng.

3. K-Means rất hữu ích trong bài toán sàng lọc y tế ban đầu (screening),
giúp phân loại tự động và cảnh báo sớm các trường hợp khối u có nguy cơ ác
tính cao để bác sĩ chỉ định can thiệp kịp thời.

""")
```


Họ và tên: Phạm Hải Yến - MSSV: 6451071091 - Lớp: CQ.64.CNTT
Chẩn đoán y tế thực tế (Malignant vs Benign)
Lành tính (B)
Ác tính (M)
Kết quả gom cụm K-Means (Độ trùng khớp: 91.04%)
Cụm Lành tính
Cụm Ác tính
PCA Component 1 (44.3%)
PCA Component 2 (19.0%)
NHẬN XÉT:
1. Thuật toán gom cụm K-Means đạt độ trùng khớp (Accuracy) lên tới hơn 91% so với nhãn chẩn đoán thực tế của bác sĩ, dù không hề sử dụng nhãn trong quá trình phân nhóm.
2. Trên không gian PCA 2 chiều, hai nhóm khối u lành tính và ác tính có sự phân tách không gian tương đối rõ ràng.
3. K-Means rất hữu ích trong bài toán sàng lọc y tế ban đầu (screening), giúp phân loại tự động và cảnh báo sớm các trường hợp khối u có nguy cơ ác tính cao để bác sĩ chỉ định can thiệp

--- HẾT---